



智能计算系统

第三章 深度学习应用

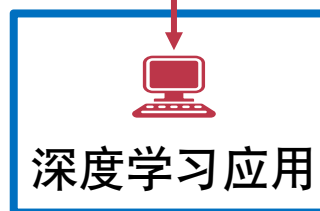
Driving Example



深度学习基础



上一章学习了神经网络的基本知识，多层感知机的正反向计算过程，以及基础优化方法。



本章通过分析经典深度学习算法，学习将基础神经网络应用到实际场景，并逐步优化实现工业级应用的过程。

让机器更好的理解和服务人类



图像信息

任务：理解图像内容

方法：卷积神经网络

Transformer

序列信息

任务：理解语音/文字/视频

方法：循环神经网络

Transformer

提纲

- ▶ 适合图像处理的卷积神经网络
- ▶ 适合语音/文本处理的循环神经网络
- ▶ 从深度学习到大模型
- ▶ 神经网络量化
- ▶ 本章小结

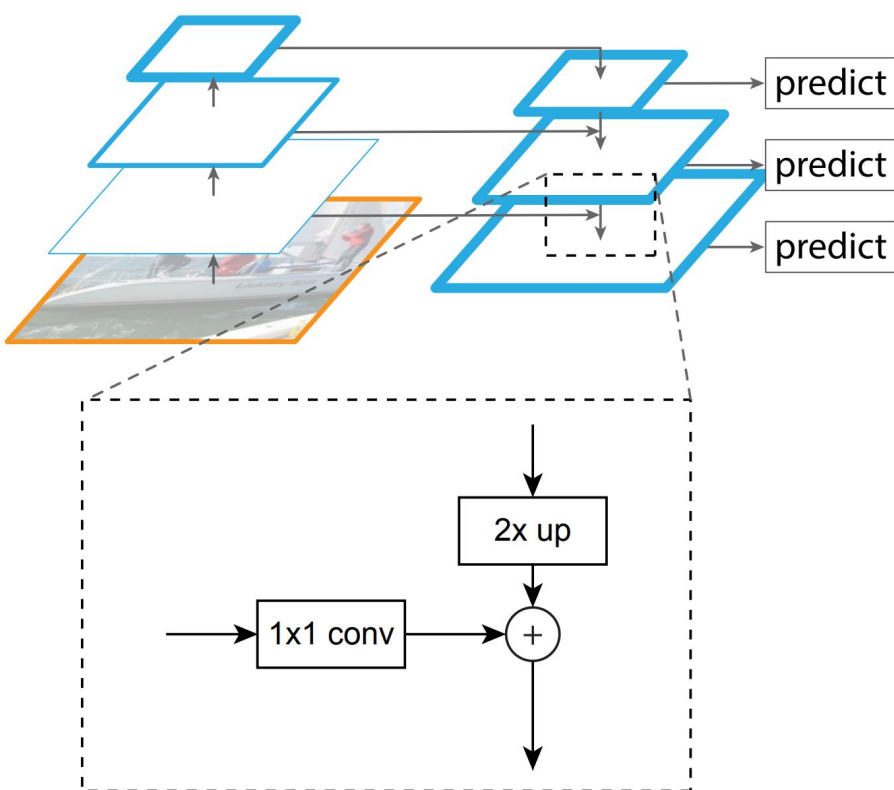
提纲

- ▶ 适合图像处理的卷积神经网络
 - ▶ 卷积神经网络总体结构
 - ▶ 神经网络的优化
 - ▶ 应用于图像分类的卷积神经网络
 - ▶ 应用于目标检测的卷积神经网络
 - ▶ Driving example

FPN

- FPN (Feature Pyramid Networks)

- Lin T Y, Dollár P, Girshick R, et al. Feature pyramid networks for object detection[C]. CVPR'2017.



- **问题：**卷积操作拥有固定的感受野，难以稳健地面对尺度多变的目标
- **难点：**
 - 1) 传统方法将图像采样到不同尺寸后输入检测器，计算量大
 - 2) 不同层级的特征图在语义层级存在差异，难以统一利用检测头进行预测
- **方法：**
 - 1) Backbone自下而上地生成多尺度的特征图，底层特征分辨率高，对小目标友好；顶层特征分辨率低，对大目标友好
 - 2) 顶层特征包含更丰富的语义信息，因此自上而下地融合高层特征与低层特征，补充多尺度语义信息

FPN

● FPN (Feature Pyramid Networks)

- Lin T Y, Dollár P, Girshick R, et al. Feature pyramid networks for object detection[C].

CVPR'2017

Faster R-CNN	proposals	feature	head	lateral?	top-down?	AP@0.5	AP	AP _s	AP _m	AP _l
(*) baseline from He <i>et al.</i> [16] [†]	RPN, C_4	C_4	conv5			47.3	26.3	-	-	-
(a) baseline on conv4	RPN, C_4	C_4	conv5			53.1	31.6	13.2	35.6	47.1
(b) baseline on conv5	RPN, C_5	C_5	2fc			51.7	28.0	9.6	31.9	43.1
(c) FPN	RPN, $\{P_k\}$	$\{P_k\}$	2fc	✓	✓	56.9	33.9	17.8	37.7	45.8

- 显著提升了Faster R-CNN在小目标和中目标上的性能

method	backbone	competition	image pyramid	test-dev					test-std				
				AP@.5	AP	AP _s	AP _m	AP _l	AP@.5	AP	AP _s	AP _m	AP _l
ours, Faster R-CNN on FPN	ResNet-101	-		59.1	36.2	18.2	39.0	48.2	58.5	35.8	17.5	38.7	47.8

Competition-winning *single-model* results follow:

G-RMI [†]	Inception-ResNet	2016		-	34.7	-	-	-	-	-	-	-	-
AttractionNet [‡] [10]	VGG16 + Wide ResNet [§]	2016	✓	53.4	35.7	15.6	38.0	52.7	52.9	35.3	14.7	37.6	51.9
Faster R-CNN +++ [16]	ResNet-101	2015	✓	55.7	34.9	15.6	38.7	50.9	-	-	-	-	-
Multipath [40] (on minival)	VGG-16	2015		49.6	31.5	-	-	-	-	-	-	-	-
ION [‡] [2]	VGG-16	2015		53.4	31.2	12.8	32.9	45.2	52.9	30.7	11.8	32.8	44.8

- 性能一举超越同时期检测器

YOLO

- YOLO(v1): Redmon J, Divvala S, Girshick R, et al. You Only Look Once: Unified, Real-Time Object Detection. CVPR'2016.
- **主要思想：**将目标检测问题转换为直接从图像中提取bounding boxes和类别概率的**单回归问题**，只需看一眼 (you only look once, YOLO) 就可以检测出目标的类别和位置。YOLO算法开创了one-stage检测的先河，将目标分类和边界框定位合二为一，实现了端到端的目标检测。YOLO的运行速度非常快，达到45帧/秒，满足实时性要求。

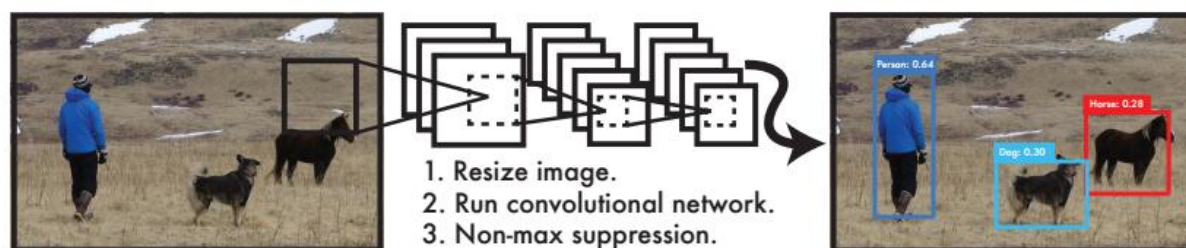


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

YOLO

- 统一检测(Unified Detection)具体实现

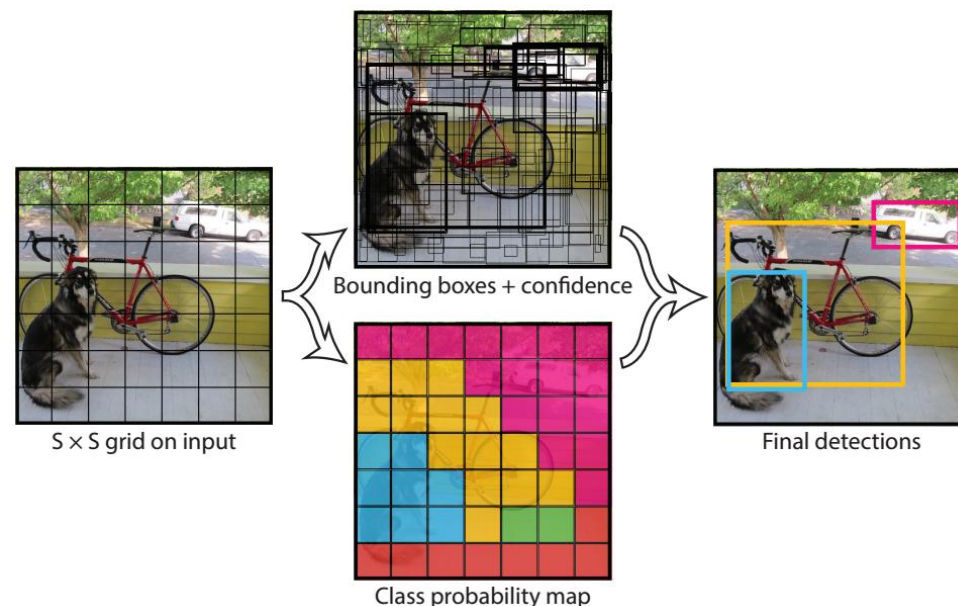
- 将输入图像分为 $S \times S$ 个格子
- 每个格子都预测 B 个Bounding box
 - 每个bbox包含五个预测值: x, y, w, h 和confidence
 - x, y, w, h 表示bbox的位置和大小, 都被归一化到 $(0, 1)$ 区间
 - confidence(置信度分数) 综合考虑当前bbox内存在目标的可能性 $\Pr(\text{Object})$ 以及预测目标位置的准确性

$\text{IoU}(\text{pred}|\text{truth})$ 定义为: $\text{confidence} = \Pr(\text{Object}) * \text{IoU}_{\text{pred}}^{\text{truth}}$, $\Pr(\text{Object}) = \begin{cases} 1, & \text{object exists in that cell} \\ 0, & \text{no object exists in that cell} \end{cases}$

- 每个格子还要预测分别属于 C 种类别的条件概率 $\Pr(\text{Class}_i|\text{Object})$, $i = 0, 1, \dots, C$, 其中 C 为数据集中物体类别的数量; 在测试时, 属于该格子的 B 个bbox共享这 C 个类别的条件概率, 则某个bbox的类别置信度为:

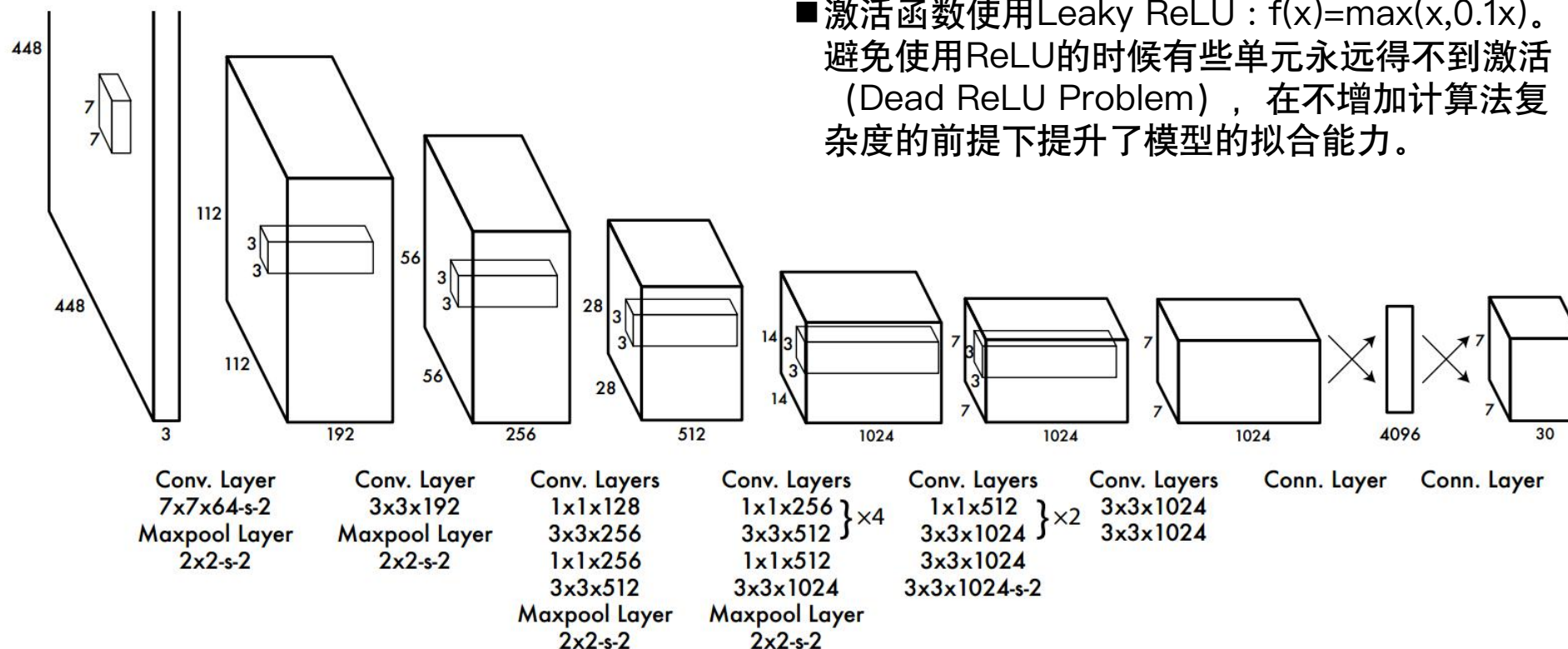
$$\Pr(\text{Class}_i|\text{Object}) * \Pr(\text{Object}) * \text{IoU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IoU}_{\text{pred}}^{\text{truth}}$$

- 最终输出tensor的维度为: $S \times S \times (B \times 5 + C)$



YOLO

● 网络结构



- 网络结构基于GoogleNet;
- 对于PASCAL VOC数据集, 采用S=7, B=2, C=20, 最终输出tensor维度为 $7 \times 7 \times 30$ (其中 $30=B*5+C$).

YOLO

YOLO(v1)的优点

- 1、检测速度快。 YOLO将目标检测重建为单一回归问题，对输入图像直接处理，同时输出边界框坐标和分类概率，而且每张图像只预测98个bbox，检测速度非常快，在Titan X 的 GPU 上能达到45 FPS，Fast YOLO检测速度可以达到155 FPS。
- 2、背景误判少。以往基于滑窗或候选区域提取的目标检测算法，只能看到图像的局部信息，会出现把背景当前景的问题。而YOLO在训练和测试时每个cell都使用全局信息做预测，因此不容易把背景误认为目标。
- 3、泛化性更好。 YOLO能够学习到目标的泛化表示，能够迁移到其它领域。例如，当 YOLO在自然图像上做训练，在艺术品上做测试时，YOLO的性能远优于DPM、R-CNN等。

YOLO

YOLO(v1) 的缺点

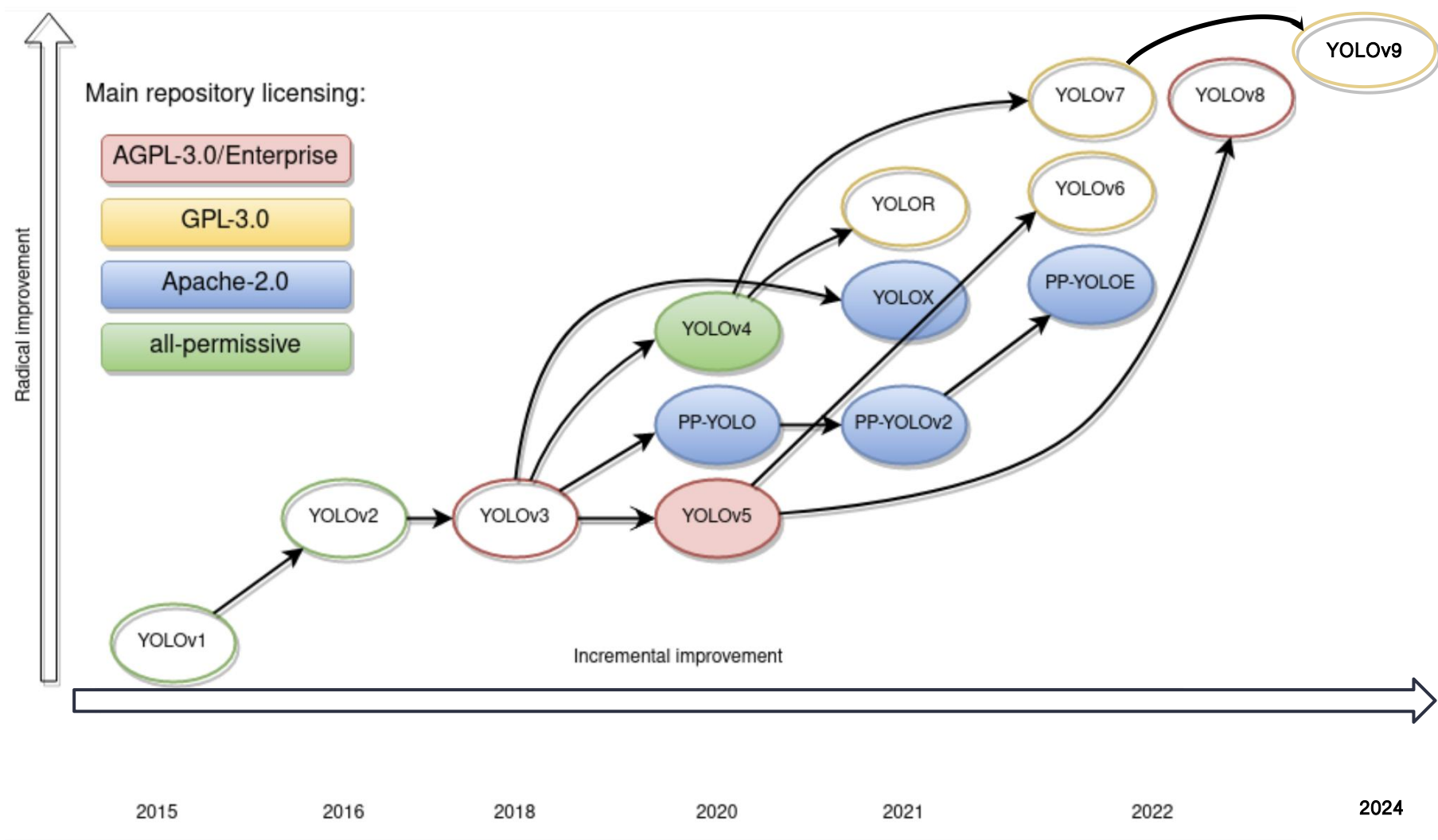
1、邻近物体检测精度低。YOLO对每个cell只预测两个bbox和一个分类，如果多个物体的中心都在同一cell内，检测精度低。

2、损失函数的设计过于简单。用坐标和分类的MSE作为损失函数不合理

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

$\mathbb{1}_i^{\text{obj}}$ 表示目标出现在cell i中 $\mathbb{1}_{ij}^{\text{obj}}$ 表示cell i中第j个边框预测目标在该cell中

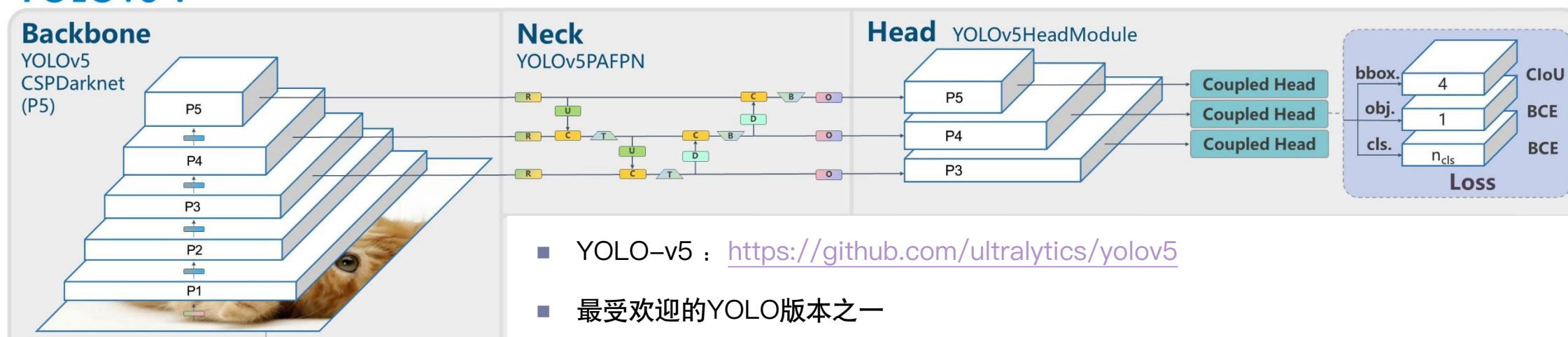
3、训练不易收敛。直接预测的bbox位置，相较于预测物体的偏移量，模型收敛不稳定。



From <https://images.app.goo.gl/HJ6A5in33HQaoVVt5>

YOLOv5

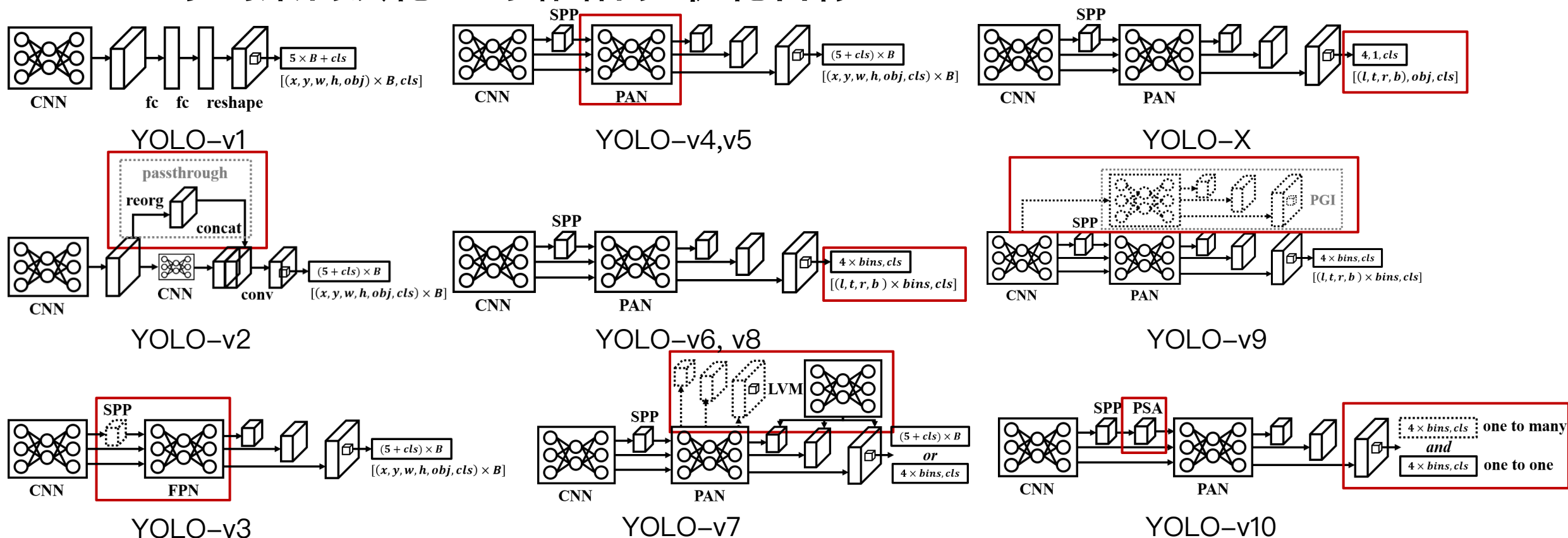
YOLOv5-I



- **Backbone:** 类似ResNet, 由1个Stem Layer和4个Stage Layer组成。
- **Neck:** FPN+PAN
 - FPN结构, 自顶向下, 将高层的语义信息通过上采样的方式传递给底层
 - PAN结构, 自下向上, 将底层的位置信息通过下采样的方式传递给高层
 - FPN层自顶向下传递强语义特征, PAN层自底向上传递强定位特征
- **Head:** 耦合检测头. 使用共同的MLP层预测类别和回归包围框
- **CIoU损失:** $\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v$
 - 对于预测框和GT框: 1) 最大化IoU 2) 最小化中心距离 3) 最小化长宽比差异

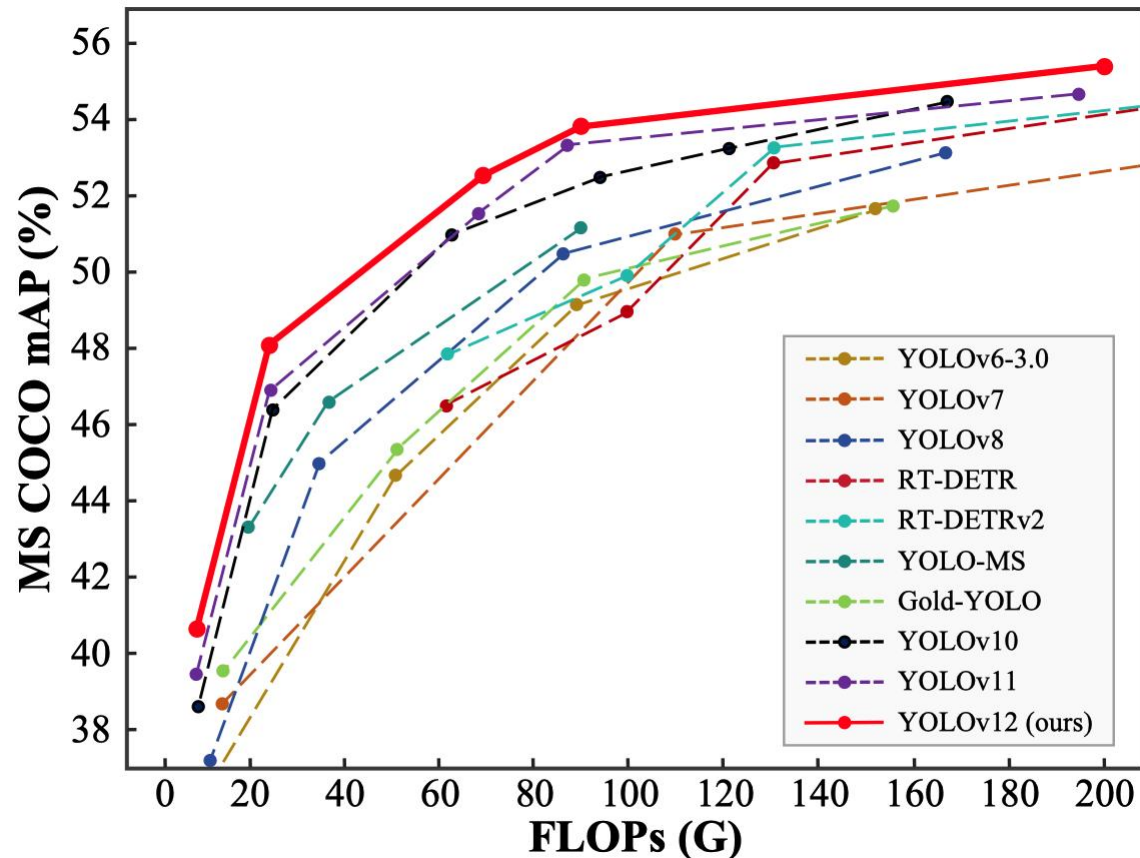
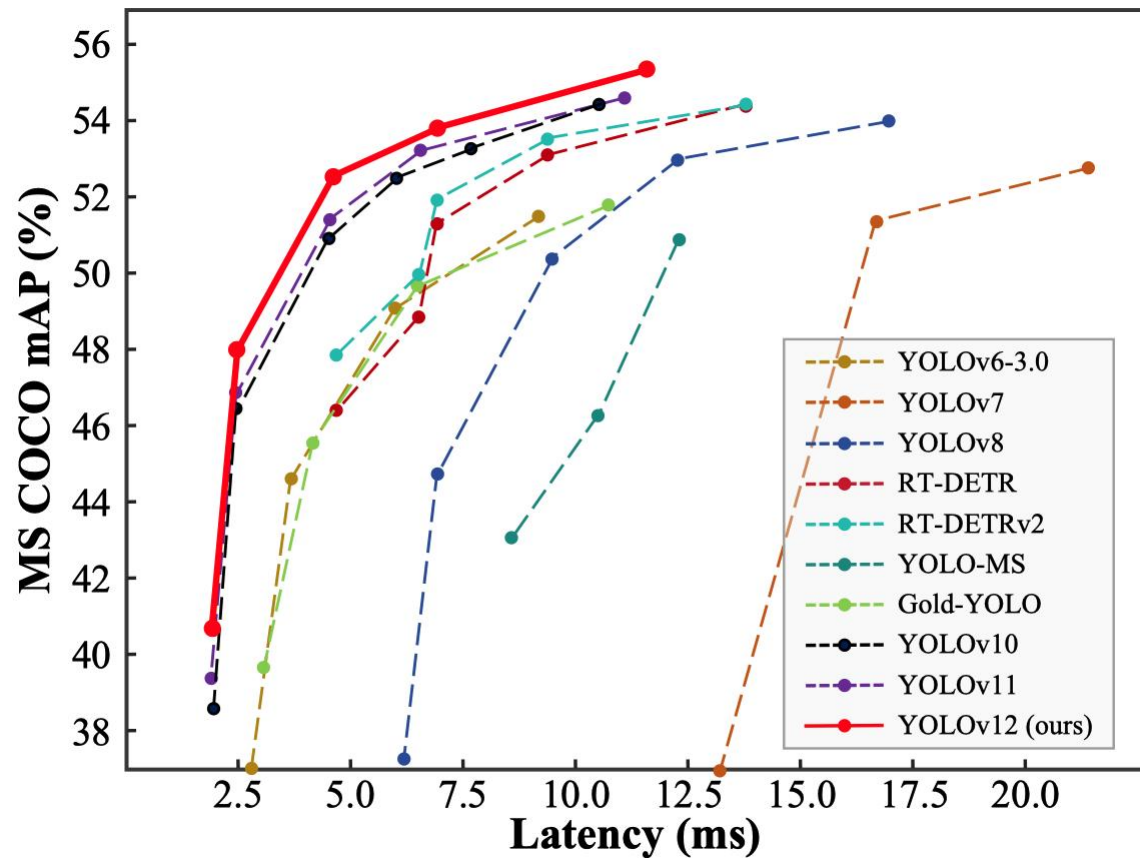
YOLO

▶ YOLO系列架构演化：网络结构+优化目标



Wang C Y, Liao H Y M. YOLOv1 to YOLOv10: The fastest and most accurate real-time object detection systems[J]. APSIPA Transactions on Signal and Information Processing, 2024, 13(1).

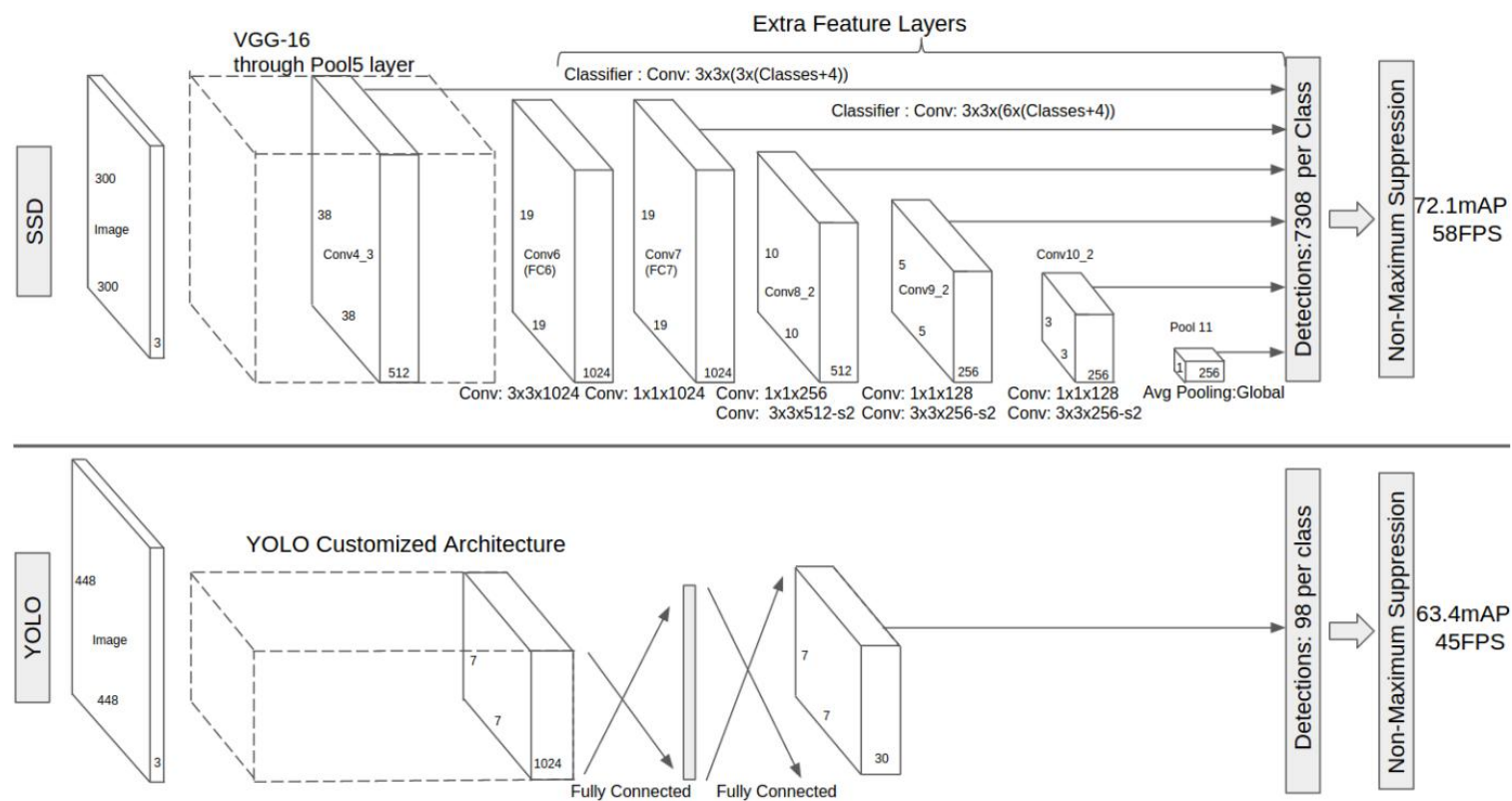
YOLO系列



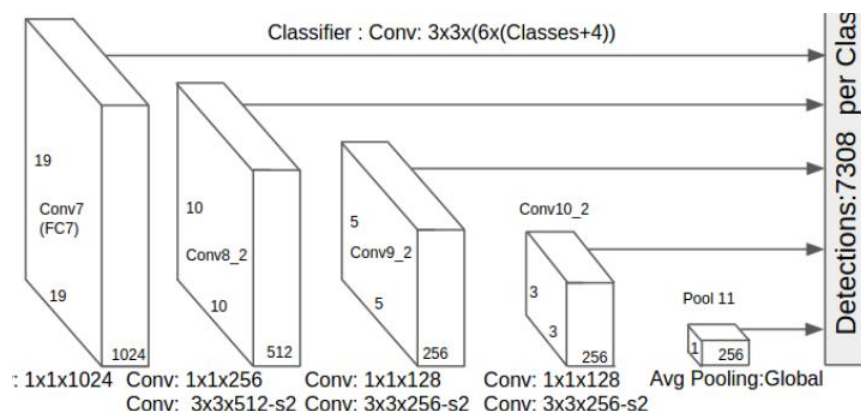
Yunjie Tian, Qixiang Ye, David Doermann, YOLOv12: Attention-Centric Real-Time Object Detectors, arXiv:2502.12524v1, 2025.

SSD

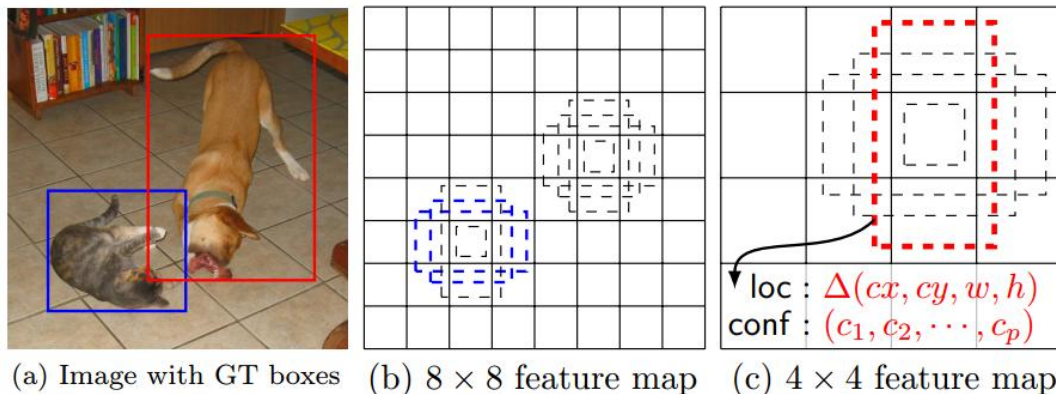
- Liu W, Anguelov D, Erhan D, et al. SSD: Single Shot MultiBox Detector[C]// ECCV. 2016.
- **主要思想：**基于YOLO直接回归 bbox和分类概率的one-stage检测方法，结合Faster R-CNN中的anchor-box思想产生先验框，并且采用特征金字塔进行多尺度预测，在满足检测速度快的同时，大大提高了检测准确度。



● 多尺度特征图检测



- CNN网络一般前面的特征图比较大，后面会逐渐采用stride=2的卷积或者pool来降低特征图大小，在大的和小的特征图都提取anchor box用来做检测，可找到最合适的anchor box尺寸，提高检测准确度。



- 比较大的特征图可以用来检测相对较小的目标，而小的特征图负责检测大目标，例如左图中8x8的特征图可以划分成更多单元，其每个单元的先验框尺度较小，适合用于检测较小的目标。

SSD

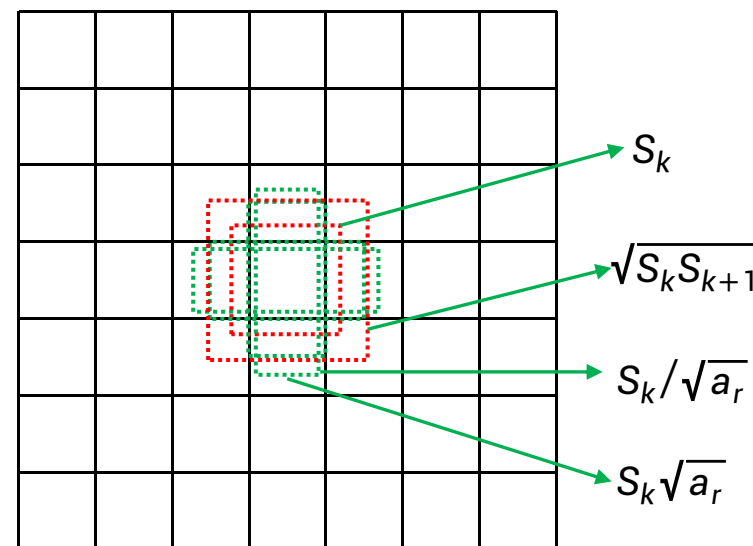
- Anchor box (论文中为default box)

- 个数: 每个点对应6个anchor boxes, 2个正方形和4个长方形(宽高比 a_r 取2、3、1/2、1/3);
- 第k层scale: $s_k = s_{\min} + \frac{s_{\max} - s_{\min}}{m - 1}(k - 1), \quad k \in [1, m]$

其中: $s_{\min} = 0.2, s_{\max} = 0.9$, m 为网络用于生成anchor box进行检测的卷积层总层数;

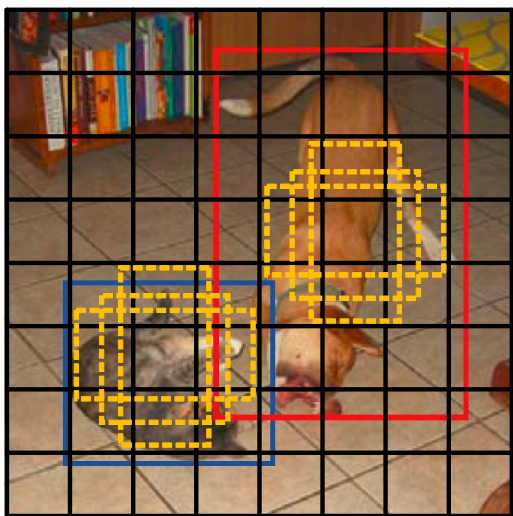
宽高比为1的情况, 增加一个默认框, 其缩放为 $s'_k = \sqrt{s_k s_{k+1}}$

- 第k层default box的宽: $w_k^a = s_k \sqrt{a_r}$
高: $h_k^a = s_k / \sqrt{a_r}$

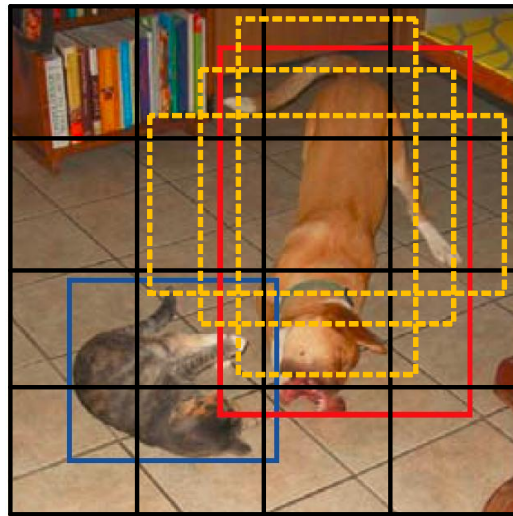


SSD

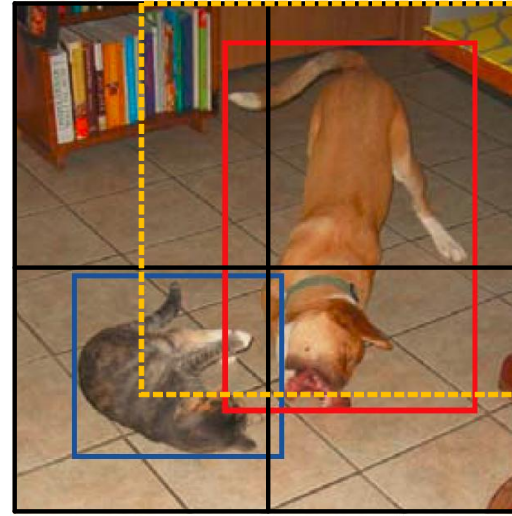
- 同时对多层特征图上的默认框计算IoU，可以找到与真实框大小和位置最接近（即IoU最大）的框，在训练时能达到最好的精度。



较低层级的特征图

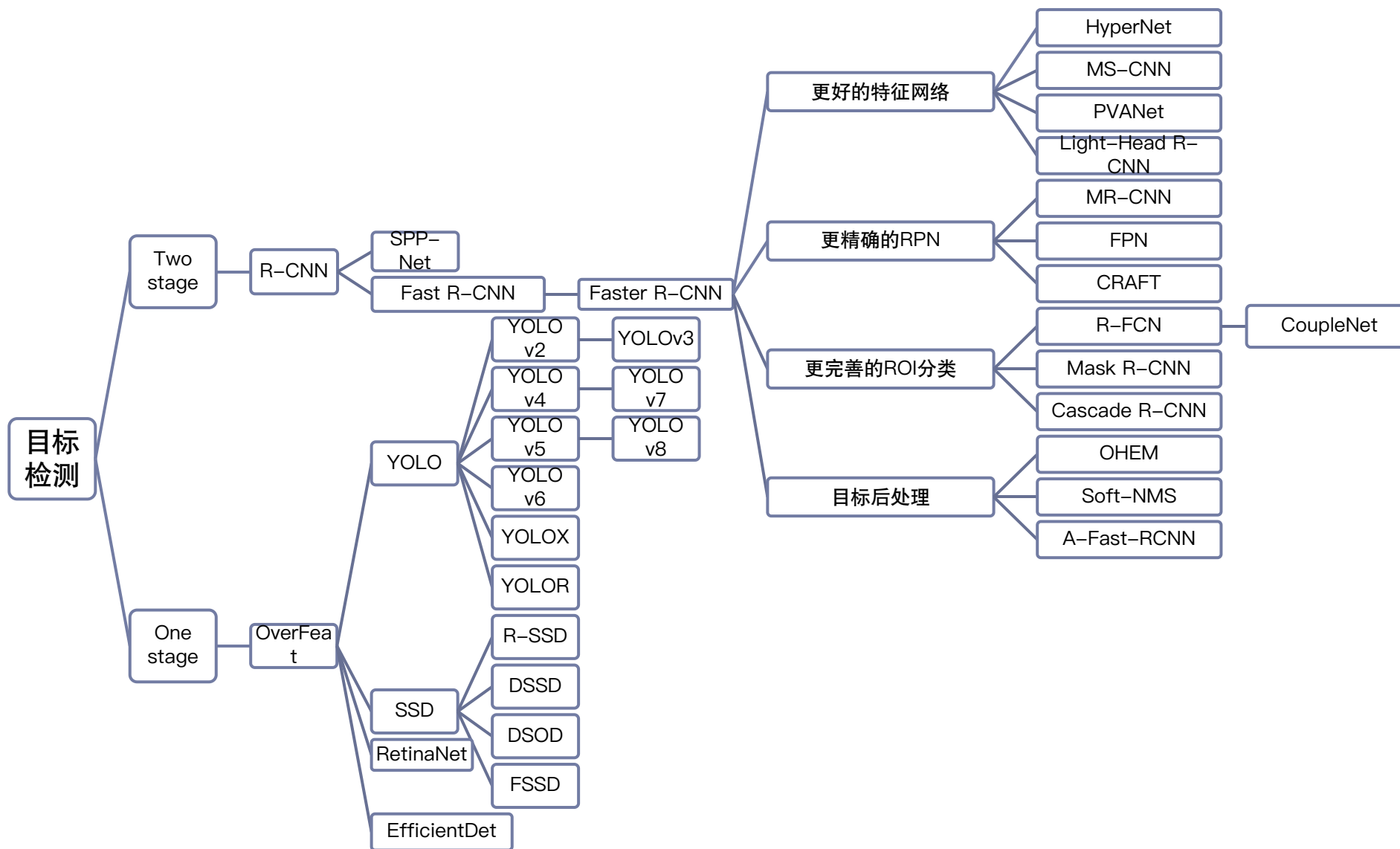


中间层级的特征图



较高层级的特征图

目标检测算法



Driving Example



像艺术家一样



Image style transfer

- ▶ Gatys et al. Image Style Transfer Using Convolutional Neural Networks. CVPR2016



Image style transfer



- ▶ 给定一张风格图像 a 和一张内容图像 p
- ▶ 风格图像经过 CNN 生成的 feature maps 组成风格特征集 A ；内容图像 p 通过 CNN 生成的 feature maps 组成内容特征集 P
- ▶ 输入一张随机噪声图像 x ，随机噪声图像 x 通过 CNN 生成的 feature maps 构成内容特征和风格特征集合 F 和 G ，目标损失函数由 A, P, F, G 计算得到
- ▶ 损失函数是希望调整图像 x ，使其最后看起来既保持内容图像 p 的内容，又有风格图像 a 的风格

Image style transfer

- ▶ 论文中使用的CNN网络为在imageNet上训练好的VGG19，去除了最后的全连接层和softmax。
- ▶ 内容损失函数
 - ▶ 只取conv4单层特征来计算内容损失
 - ▶ 计算内容图片特征和噪声图片特征之间的欧式距离

$$L_{content}(p, x, l) = \frac{1}{2} \sum_{i,j} (F_{ij}^l - P_{ij}^l)^2$$

- l : 用于计算内容特征的层数.
- F_{ij}^l : 生成图片在第 l 层第 i 个特征图上位置 j 处的特征值
- P_{ij}^l : 内容图片在第 l 层第 i 个特征图上位置 j 处的特征值
- p : 内容图片
- x : 生成图片

Gatys et al. Image Style Transfer Using Convolutional Neural Networks. CVPR2016.

Image style transfer

► 风格损失函数

- 取 conv1~conv5共5层的特征来计算风格损失
- 使用相关矩阵来表示图像的风格

$$L_{style}(a, x) = \sum_l w_l E_l$$

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^l - A_{ij}^l)^2$$

$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l \quad (\text{gram矩阵})$$

- L : 用于计算风格特征的层数
- w_l : 第 l 层的 E_l 用于计算风格损失的权重, 文中都取0.2
- a : 初始风格图片
- x : 生成图片
- A_{ij}^l : 风格图片在 l 层第 i 个特征图和第 j 个特征图的内积
- G_{ij}^l : 生成图片在 l 层第 i 个特征图和第 j 个特征图的内积
- M_l : 第 l 层的输出特征图的大小
- N_l : 第 l 层的输出特征图的数目
- F_{ik}^l : 第 l 层第 i 个特征图上位置 k 处的特征值

Image style transfer

$$L_{total}(\mathbf{p}, \mathbf{a}, \mathbf{x}) = \alpha L_{content}(\mathbf{p}, \mathbf{x}) + \beta L_{style}(\mathbf{a}, \mathbf{x})$$

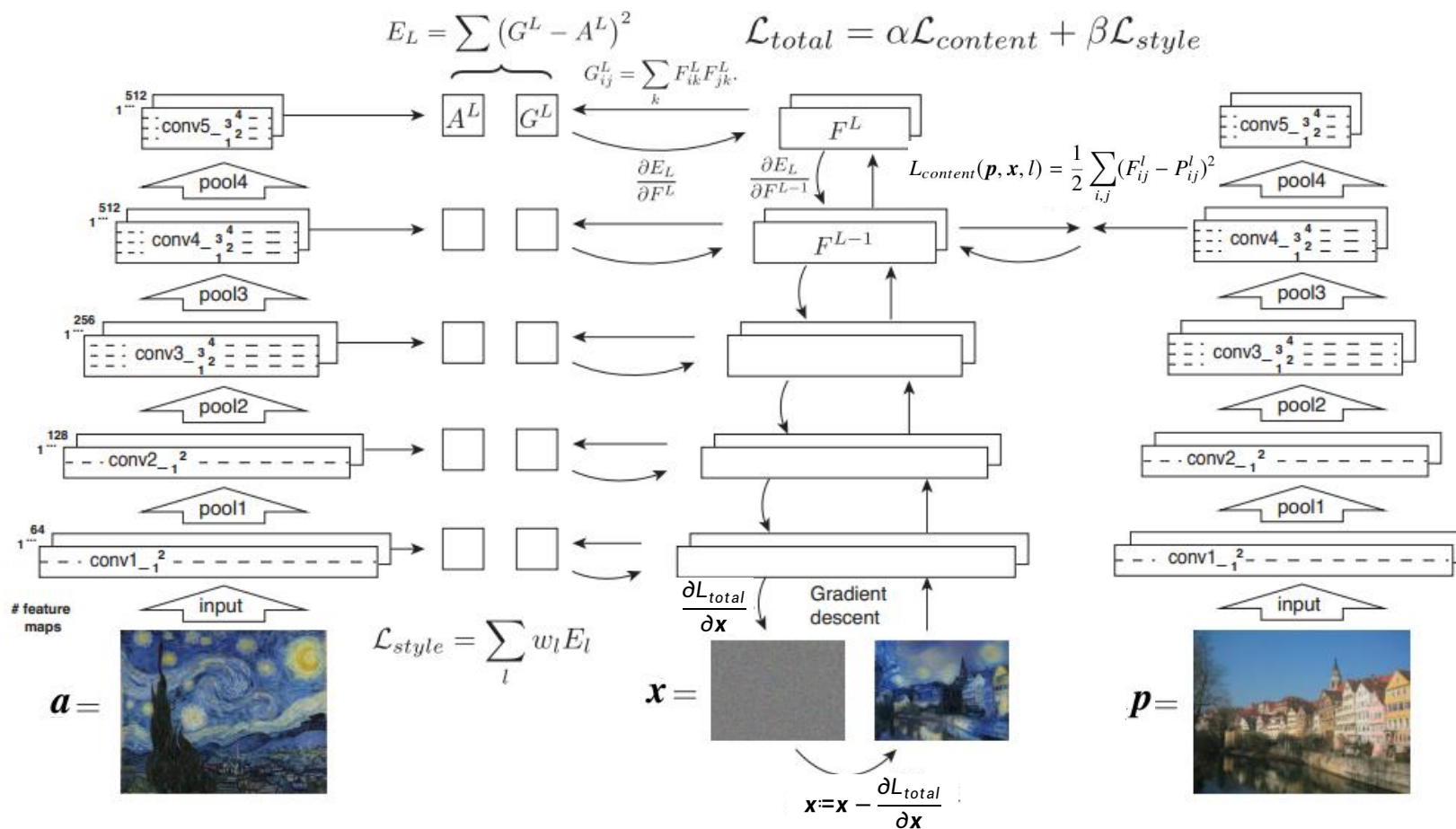


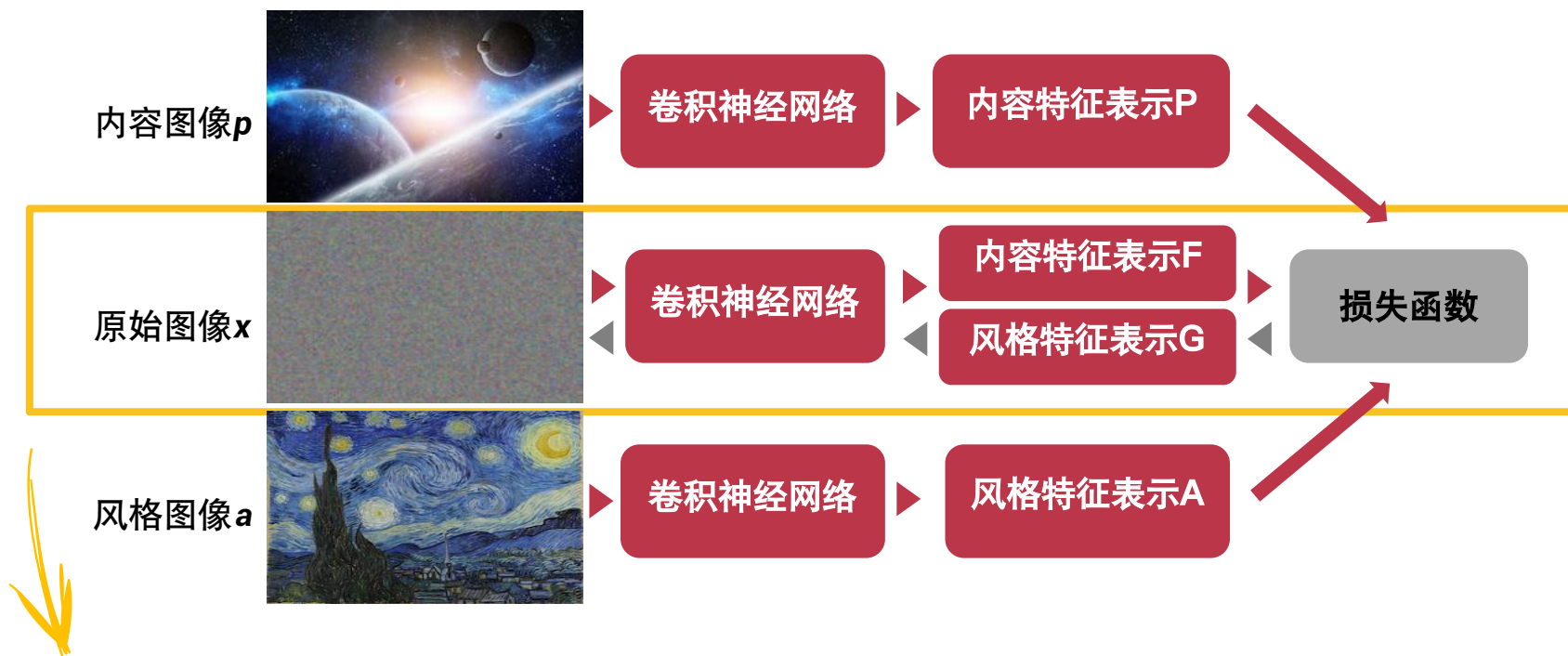
Image style transfer

$$L_{total}(p, a, x) = \alpha L_{content}(p, x) + \beta L_{style}(a, x)$$

α/β 越大
内容越具象



Real-Time Image style transfer



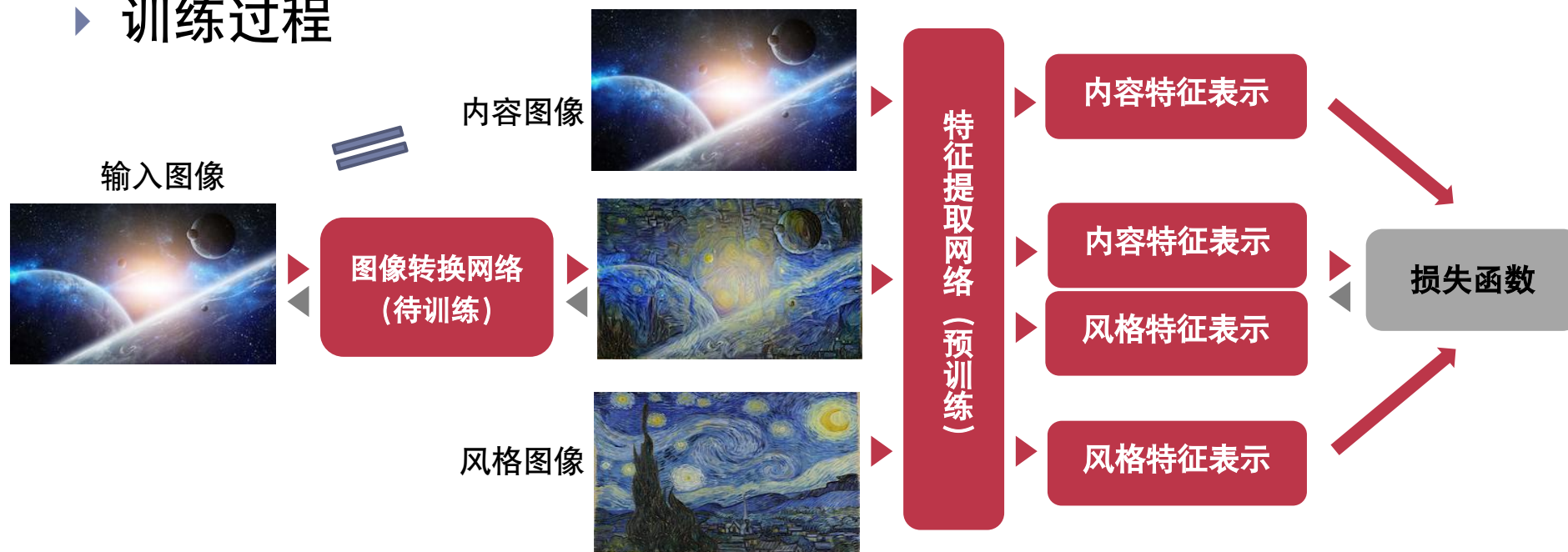
对每一张待转换图片，都要进行前馈和反馈调优过程，无法做到实时转换；

能够把图像生成网络提前训练好，
使风格转换过程不需要进行反向训练？



Real-Time Image style transfer

▶ 训练过程

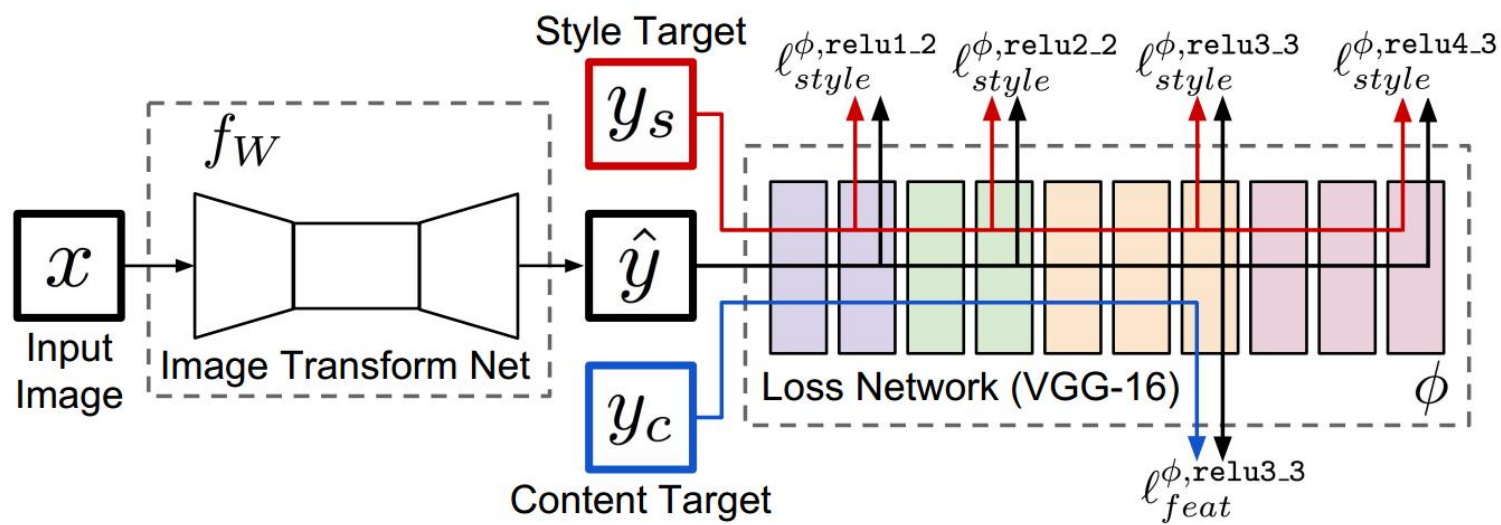


▶ 实时转换过程



Real-Time Image style transfer

- ▶ J. Johnson J, A. Alahi, Fei-Fei Li, et al. Perceptual Losses for Real-Time Style Transfer and Super-Resolution. ECCV2016.
- ▶ 项目实施: <https://github.com/jcjohnson/fast-neural-style>



▶ Image Transform Net

- ▶ 深度卷积网络，参考DCGAN的设计思想：用步长卷积和小数步长卷积替代pooling、每个卷积层后接BatchNorm和ReLU
- ▶ 增加残差结构，使网络更易训练

Real-Time Image style transfer



Content Gatys et al. Our
s

Image Size	Gatys <i>et al.</i> [11]			Ours	Speedup		
	100	300	500		100	300	500
256 × 256	3.17	9.52s	15.86s	0.015s	212x	636x	1060x
512 × 512	10.97	32.91s	54.85s	0.05s	205x	615x	1026x
1024 × 1024	42.89	128.66s	214.44s	0.21s	208x	625x	1042x

提纲

- ▶ 适合图像处理的卷积神经网络
- ▶ 适合语音/文本处理的循环神经网络
- ▶ 从深度学习到大模型
- ▶ 神经网络量化
- ▶ Driving Example
- ▶ 本章小结
- ▶ 本章实验

循环神经网络



图像信息

任务：理解图像内容

方法：卷积神经网络

序列信息

任务：理解语音/文字/视频

方法：循环神经网络

任务特点：按时序输入的数据之间不是相互独立的，前后数据之间有相关性，所以网络要有“存储”信息的能力。

循环神经网络

- ▶ 主要应用场景：机器翻译、图片描述、视频标注、视觉问答等

美国总统布什昨天在白宫与以色列总理沙龙就中东局势举行了一个小时的会谈。

US President George W. Bush held an hour-long meeting with Israeli Prime Minister Ariel Sharon on the situation in the Middle East yesterday at the White House.

机器翻译



"girl in pink dress is jumping in air."



"black and white dog jumps over bar."



"young girl in pink shirt is swinging on swing."



"man in blue wetsuit is surfing on wave."

图片描述

主要应用场景

Applications

- Natural Language Processing
 - Language Modeling
 - Speech Recognition
 - Machine Translation
 - Conversation Modeling
 - Question Answering
- Computer Vision
 - Object Recognition
 - Image Generation
 - Video Analysis
- Multimodal (CV+NLP)
 - Image Captioning
 - Video Captioning
 - Visual Question Answering
- Turing Machines
- Robotics
- Other

- GitHub Project: **Awesome Recurrent Neural Networks**
- A curated list of resources dedicated to recurrent neural networks (closely related to deep learning).
- <https://github.com/kjw0612/awesome-rnn>

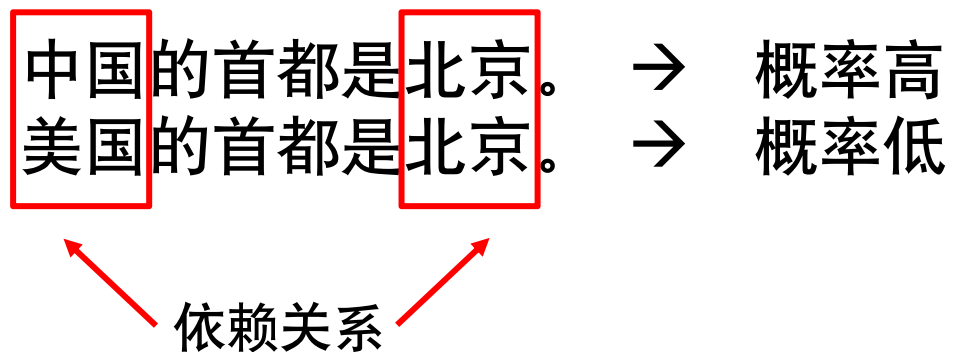
语言模型

- ▶ 序列数据：有先后顺序的一组数据，数据间有相互依赖关系
 - ▶ 将文本转化为一组由词元（token）组成的序列数据
- ▶ 语言模型：估计序列发生的概率分布

$$P(x_1, x_2, \dots, x_t) = \prod_{i=1}^t P(x_i | x_1, x_2, \dots, x_{i-1})$$

中国的首都是北京。 → 概率高
美国的首都是北京。 → 概率低

依赖关系



模型处理

- ▶ 分词 (tokenization)
- ▶ 嵌入 (embedding)
- ▶ 网络模型

https://www.bilibili.com/video/BV1dfmoY2ESA/?spm_id_from=333.1387.upload.video_card.click&vd_source=17457963d33d31888494e6740726da1a

tokenization

- ▶ token（词元）：文本的基本单位（经过分词后获得）
- ▶ tokenization（词元化/分词）：将清洗后的文本字符串划分为有意义的基本单元（词元）组成的序列
- ▶ 词表：由所有词元组成的列表

tokenization

▶ token的不同粒度

- ▶ character-level: token为字符
- ▶ word-level: token为单词
- ▶ subword-level: token为subword

“A hippopotamus ate my homework.”

Vocab Type	Example	Ex. length
character-level	<i>['A', ' ', 'h', 'i', 'p', 'p', 'o', 'p', 'o', 't', 'a', 'm', 'u', 's', ' ', ' ', 'a', 't', 'e', ' ', ' ', 'm', 'y', ' ', ' ', 'h', 'o', 'm', 'e', 'w', 'o', 'r', 'k', '.']</i>	31
subword-level	<i>['A', 'hip', '##pop', '##ota', '##mus', 'ate', 'my', 'homework', '.']</i>	9
word-level	<i>['A', 'hippopotamus', 'ate', 'my', 'homework', '.']</i>	5

tokenization

- ▶ Subword Tokenization: Byte Pair Encoding (BPE, 字节对编码)

- ▶ 核心思路: 不断将最频繁出现的一对词元合并成一个词元

- ▶ 步骤

- ▶ 预分词 ("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

- ▶ 基础词表 ["b", "g", "h", "n", "p", "s", "u"]

- ▶ 合并 ("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)
("h" "u" "g", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)
("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4),
("hug" "s", 5)

- ▶ 目前LLM中常用的分词方法

- ▶ GPT词表 40478: 478 base characters + 40000次合并

Neural Machine Translation of Rare Words with Subword Units, Sennrich et al, ACL2016.

embedding

- ▶ embedding（词表示/词嵌入）：
 - ▶ 将离散的token转化为固定维度的连续的数值向量表示，捕捉token的语义信息
 - ▶ 最简单粗暴的embedding: one-hot
 - ▶ 高效的embedding: word2vec
 - ▶ Transformer中使用的embedding: 学习embedding矩阵

苹果 $\longrightarrow$ [0.1, 0.3, 0.4]

红苹果 ?= 红 + 苹果

$$\begin{bmatrix} 0.3 \\ 0.9 \\ 0.9 \end{bmatrix} = f \left\{ \begin{bmatrix} 0.1 \\ 0.2 \\ 0.1 \end{bmatrix}, \begin{bmatrix} 0.2 \\ 0.7 \\ 0.8 \end{bmatrix} \right\}$$

embedding

- ▶ 最简单粗暴的embedding: one-hot

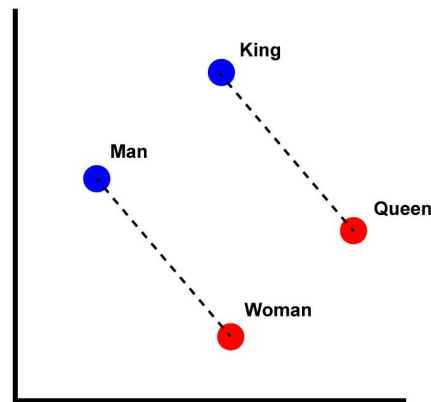
- ▶ 对于大小为N的词表，用长度为N的向量表示token，向量中token索引对应的值为1，其余值为0
- ▶ 缺陷：无法准确表达不同词之间的相似度（任意两个token的余弦相似度为0）

维度较高，稀疏表示，计算不友好

苹果 $\xrightarrow{\text{one-hot}}$ $[0, \dots, 0, 1, 0, \dots, 0]$

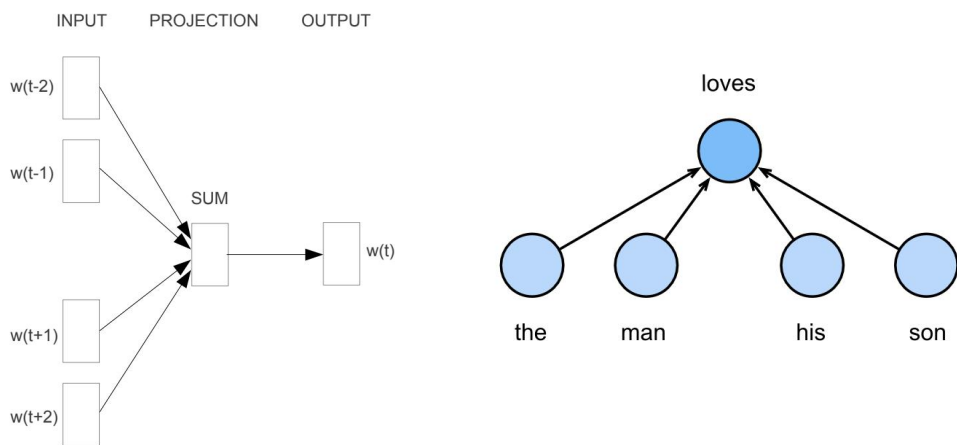
embedding

- ▶ 高效的embedding: word2vec
 - ▶ 较为准确的表达不同词之间的相似性和关系
 - ▶ 稠密表示, 维度低, 计算友好
 - ▶ 预训练: 利用大量语料, 无监督训练

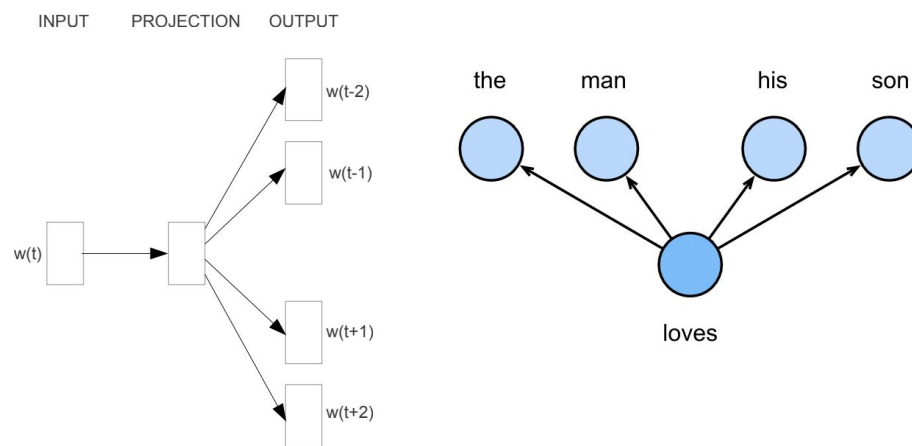


Man is to Woman as King is to Queen

$$\begin{aligned}\text{King} - \text{Man} &= \text{Queen} - \text{Woman} \\ \text{Queen} &= \text{King} - \text{Man} + \text{Woman}\end{aligned}$$



①Continuous Bag-of-Words (CBOW, 连续词袋模型)

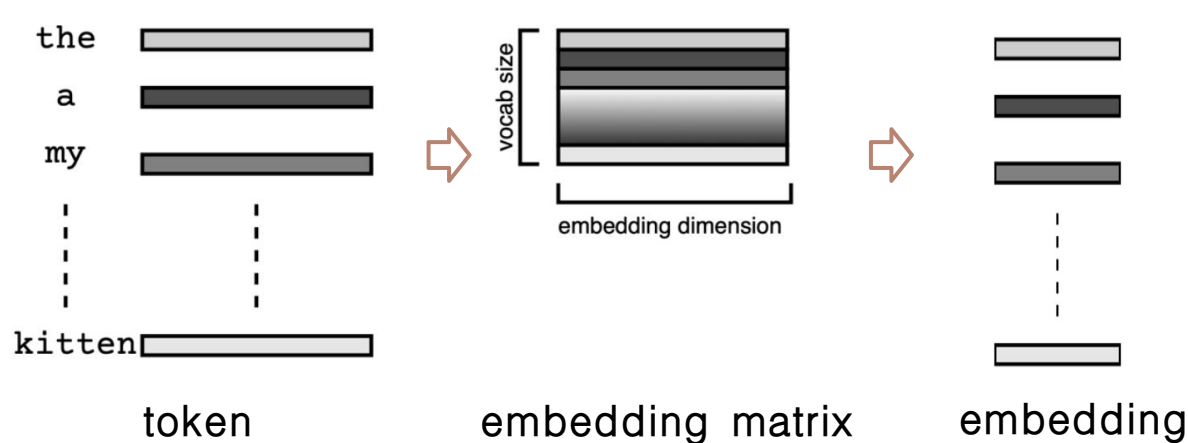


②Skip-gram (跳元模型)

Efficient Estimation of Word Representations in Vector Space, Mikolov et al, ICLR workshop 2013.

embedding

- ▶ Transformer中使用的embedding: 学习embedding矩阵
 - ▶ 较为准确的表达不同词之间的相似性和关系。稠密表示, 维度低, 计算友好
 - ▶ 输入的token用one-hot编码表示, 矩阵乘法可以简化为查表操作
 - ▶ 在训练过程中学习embedding矩阵权重



$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 8 & 2 & 1 & 9 \\ 6 & 5 & 4 & 0 \\ 7 & 1 & 6 & 2 \\ 1 & 3 & 5 & 8 \\ 0 & 4 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 5 & 8 \end{bmatrix}$$

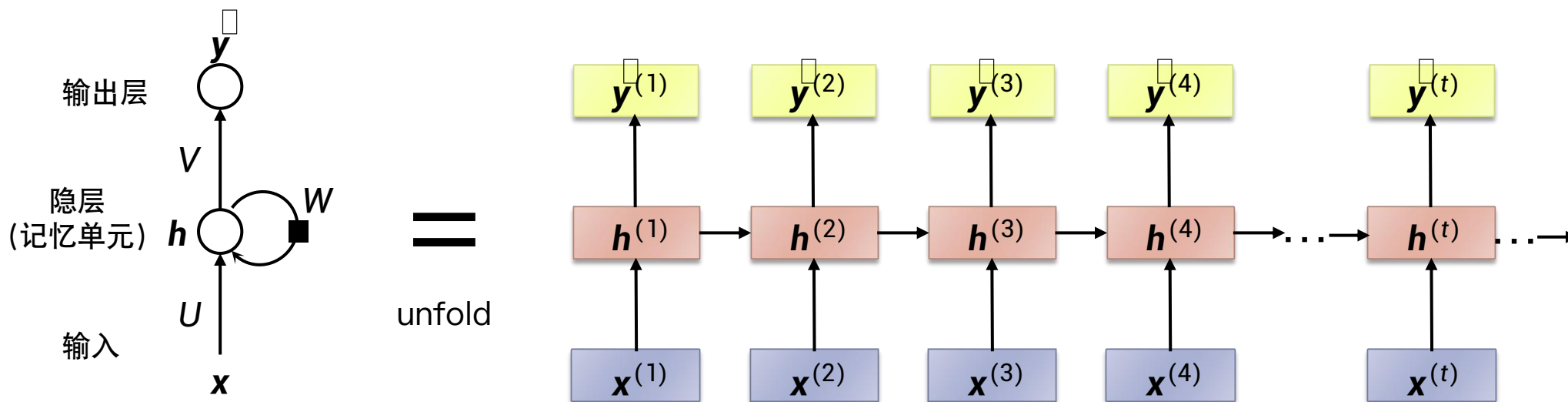
One-hot vector

Embedding Weight Matrix

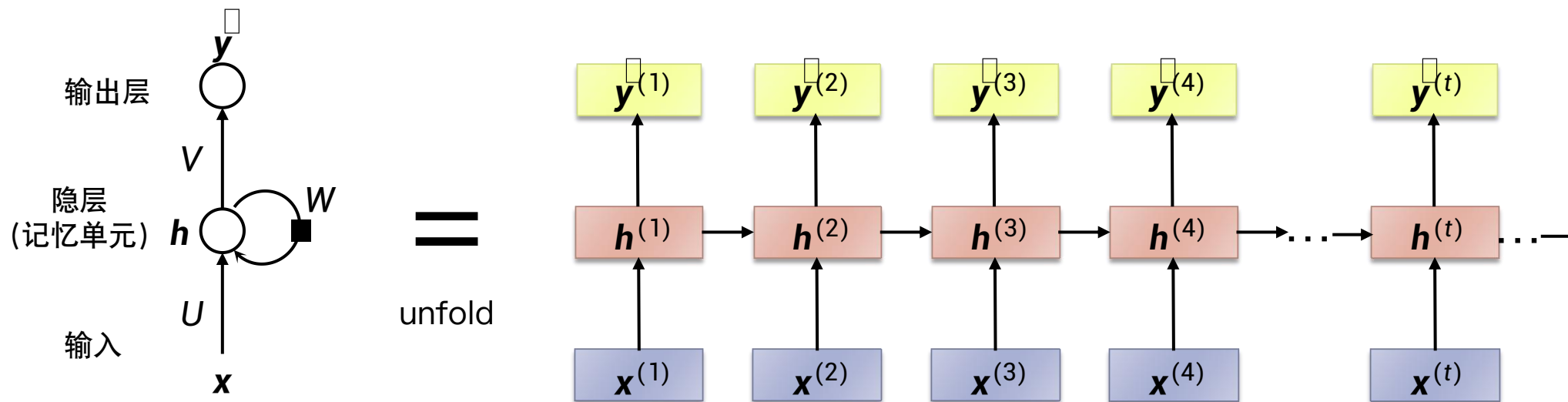
Hidden layer output

循环神经网络结构

- ▶ 循环神经网络 (RNN) 通过使用带自反馈的神经元，能够处理任意长度的序列
- ▶ 时刻 t 的输入为 $\mathbf{x}^{(t)}$ ，隐藏状态为 $\mathbf{h}^{(t)}$ 。 $\mathbf{h}^{(t)}$ 既和当前时刻输入 $\mathbf{x}^{(t)}$ 相关，也和上一时刻隐藏状态 $\mathbf{h}^{(t-1)}$ 相关
- ▶ $\mathbf{h}^{(t)} = f(W\mathbf{h}^{(t-1)} + U\mathbf{x}^{(t)} + \mathbf{b})$ ， f 是非线性激活函数，常用tanh或ReLU



循环神经网络结构



- **时序sequence:** RNN能建模序列数据，序列指的是前、后输入数据($x^{(t)}$, $x^{(t+1)}$)不独立，相互影响
- **循环recurrent:** 对每个输入的操作都是一样的，循环往复地重复这些相同操作，每时刻有相同参数 W 和 U (参数共享)
- **记忆memory:** 隐藏层 $h^{(t)}$ 中捕捉了所有时刻 t 之前的信息，理论上 $h^{(t)}$ 记忆的内容可以无限长，然而实际上记忆还是有限的

循环神经网络结构

▶ 正向计算过程

初始时刻: h_0 为输入的初始零向量

$$\mathbf{h}^{(1)} = f(W\mathbf{h}^{(0)} + U\mathbf{x}^{(1)} + \mathbf{b})$$

$$\mathbf{o}^{(1)} = V\mathbf{h}^{(1)} + \mathbf{c}$$

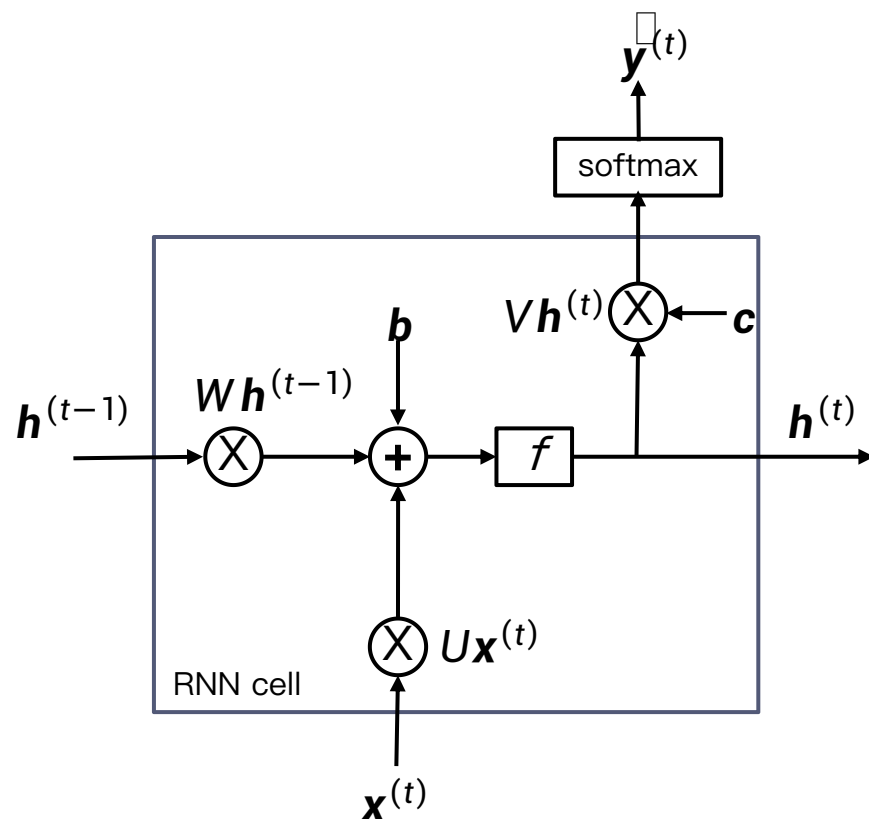
$$\hat{\mathbf{y}}^{(1)} = \text{softmax}(\mathbf{o}^{(1)})$$

一般时刻:

$$\mathbf{h}^{(t)} = f(W\mathbf{h}^{(t-1)} + U\mathbf{x}^{(t)} + \mathbf{b})$$

$$\mathbf{o}^{(t)} = V\mathbf{h}^{(t)} + \mathbf{c}$$

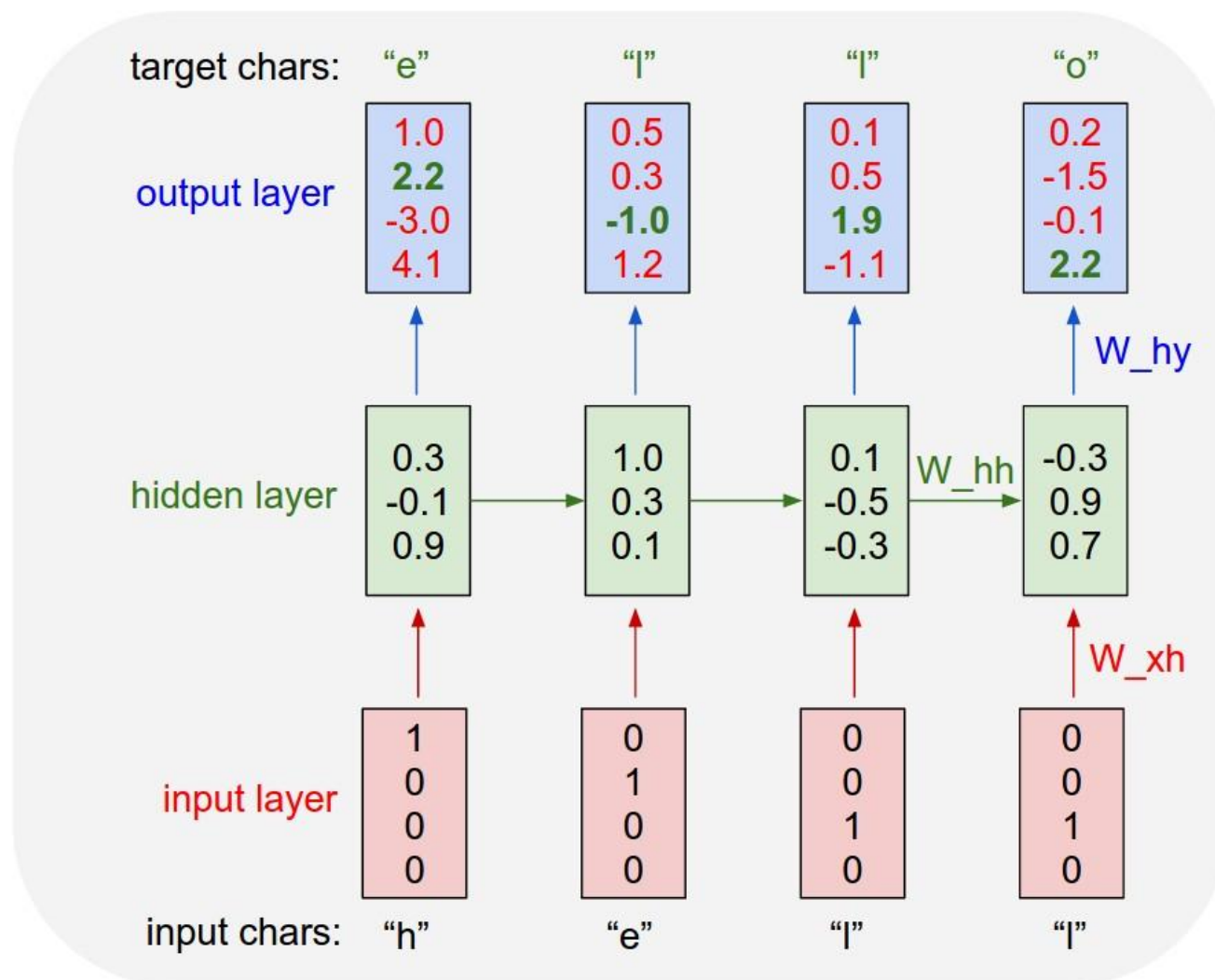
$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{o}^{(t)})$$



- f 常用tanh或ReLU激活函数
- W 、 U 、 V 、 b 在时间序列上共享参数

循环神经网络

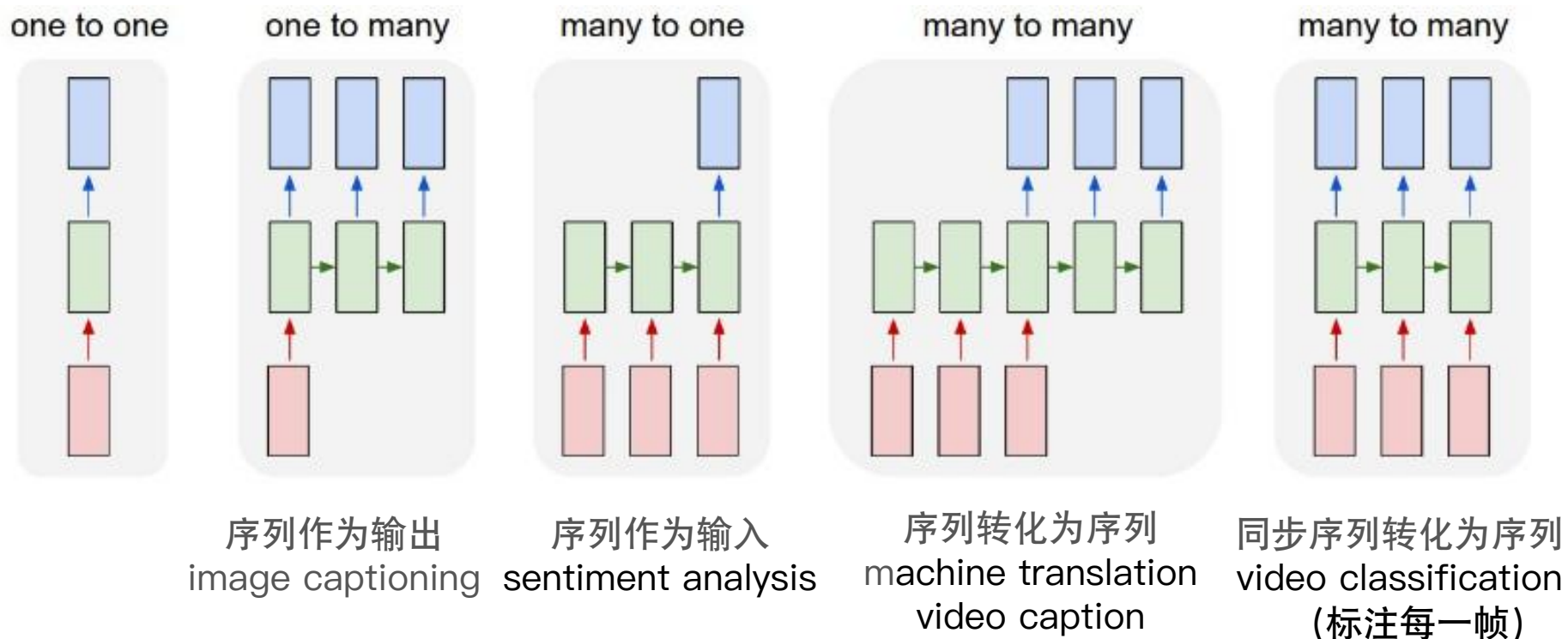
► 示例



<https://karpathy.github.io/2015/05/21/rnn-effectiveness/>

循环神经网络结构

► RNN的多种输入-输出结构



对序列的长度（绿色块块多少个）无要求，不用预先定义

循环神经网络结构

反向传播BPTT (back-propagation through time)

某个时刻的损失函数为:

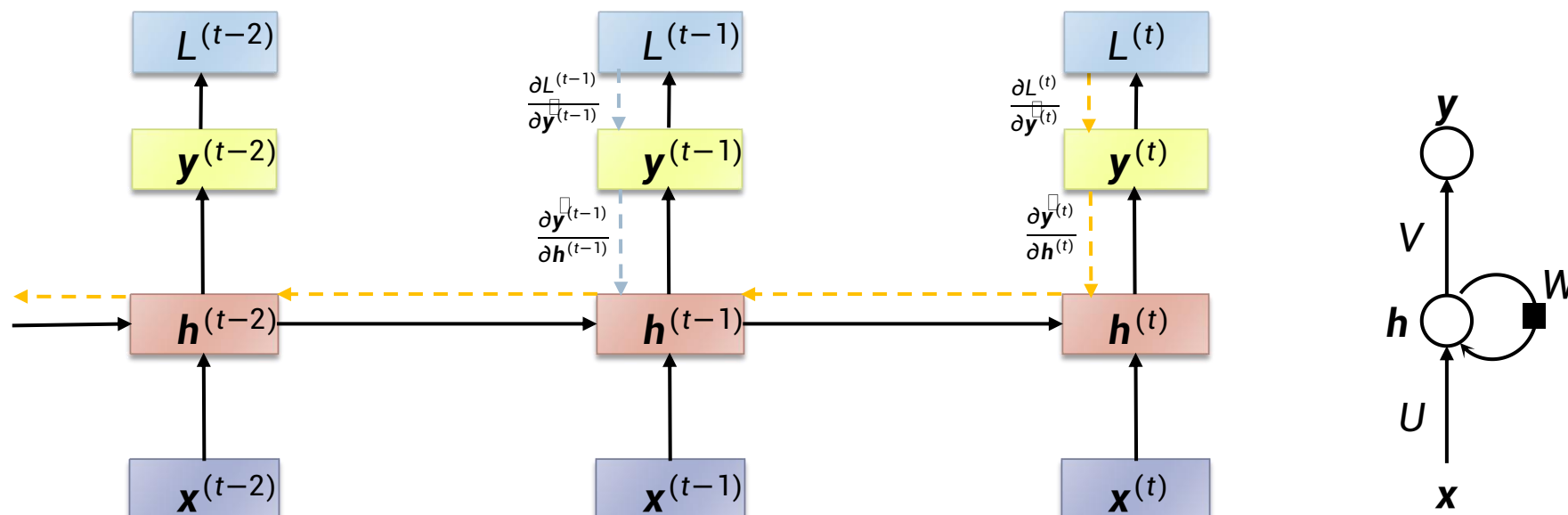
$$L^{(t)} = -\mathbf{y}^{(t)} \ln \hat{\mathbf{y}}^{(t)}$$

整个序列的损失函数为:

$$L = \sum_{t=1}^T L^{(t)} = - \sum_{t=1}^T \mathbf{y}^{(t)} \ln \hat{\mathbf{y}}^{(t)}$$

损失函数对W的偏导为:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L^{(t)}}{\partial W} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L^{(t)}}{\partial \hat{\mathbf{y}}^{(t)}} \frac{\partial \hat{\mathbf{y}}^{(t)}}{\partial \mathbf{h}^{(t)}} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}} \frac{\partial \mathbf{h}^{(k)}}{\partial W}$$



RNN的梯度消失与梯度爆炸

► 梯度爆炸/梯度消失

根据推导可知序列损失函数对U和W的梯度为：

$$\frac{\partial L}{\partial W} = \sum_t \sum_{k=1}^{k=t} \frac{\partial L^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial \mathbf{o}^{(t)}} \frac{\partial \mathbf{o}^{(t)}}{\partial \mathbf{h}^{(t)}} \left(\prod_{i=k+1}^t W^\top \text{diag} \left(1 - (\mathbf{h}^{(i)})^2 \right) \right) \frac{\partial \mathbf{h}^{(k)}}{\partial W}$$

$$\frac{\partial L}{\partial U} = \sum_t \sum_{k=1}^{k=t} \frac{\partial L^{(t)}}{\partial \hat{y}^{(t)}} \frac{\partial \hat{y}^{(t)}}{\partial \mathbf{o}^{(t)}} \frac{\partial \mathbf{o}^{(t)}}{\partial \mathbf{h}^{(t)}} \left(\prod_{i=k+1}^t W^\top \text{diag} \left(1 - (\mathbf{h}^{(i)})^2 \right) \right) \frac{\partial \mathbf{h}^{(k)}}{\partial U}$$

$$\text{令 } \gamma = \left\| \prod_{i=k+1}^t W^\top \text{diag} \left(1 - (\mathbf{h}^{(i)})^2 \right) \right\|_2$$

$$\text{当 } t \gg k \text{ 时, } \gamma \begin{cases} \rightarrow \infty, & \frac{\partial L}{\partial U} \rightarrow \infty, & \frac{\partial L}{\partial W} \rightarrow \infty & \text{梯度爆炸} \\ \rightarrow 0, & \frac{\partial L}{\partial U} \rightarrow 0, & \frac{\partial L}{\partial W} \rightarrow 0 & \text{梯度消失} \end{cases}$$

由于梯度爆炸或消失的存在，循环神经网络实际上只能学习到短期的依赖关系。

RNN的梯度消失与梯度爆炸

- ▶ 循环神经网络的递归结构导致梯度消失/梯度爆炸现象更明显

- ▶ 如何改进

- ▶ 梯度爆炸问题

- ▶ 梯度截断

Algorithm 1 Pseudo-code for norm clipping the gradients whenever they explode

```
 $\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$   
if  $\|\hat{\mathbf{g}}\| \geq threshold$  then  
     $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$   
end if
```

Pascanu, R.; Mikolov, T. & Bengio, Y. Dasgupta, S. & McAllester, D. (Eds.) On the difficulty of training recurrent neural networks, ICML2013.

- ▶ 梯度消失问题

- ▶ 模型上的改进, 例如LSTM、GRU算法

RNN的梯度消失与梯度爆炸

- ▶ 由于梯度消失，循环神经网络无法处理长期依赖关系；
- ▶ 比如，考虑一个语言模型，试图根据之前单词预测下一个。
- ▶ 如果要预测 “The clouds are in the sky” 中最后一个单词，不需要更多的上下文即可知道下一个单词会是 “sky”。在这种情况下，相关信息与预测位置的间隔比较小，RNNs可以学会使用之前的信息。
- ▶ 考虑试图预测 “I grew up in **Italy**... I speak fluent Italian.” 中最后一个，则需要用到包含 “Italy” 的上下文，从前面的信息推断后面的单词。相关信息与预测位置的间隔可能会很大。随着这种间隔的拉长，RNNs就会无法学习连接信息。

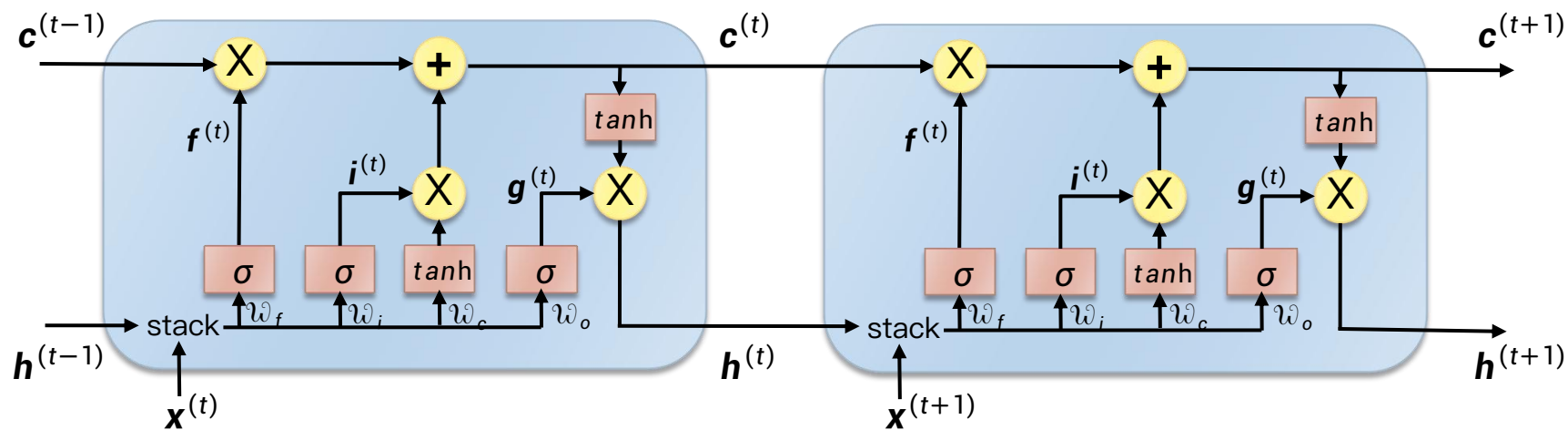
如何记住长期依赖？



长短期记忆模型

► LSTM (Long Short-Term Memory networks)

- Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural computation, 1997, 9(8): 1735–1780.



- 隐藏状态：作为神经网络的记忆，保存着网络先前观察到的数据信息。
- 单元状态：类似信息传送带，它贯穿整个链条，只有一些小的线性相互作用；这很容易让信息以不变的方式向下流动；LSTM有能力向单元状态中移除或添加信息，这种管理结构称为门限。

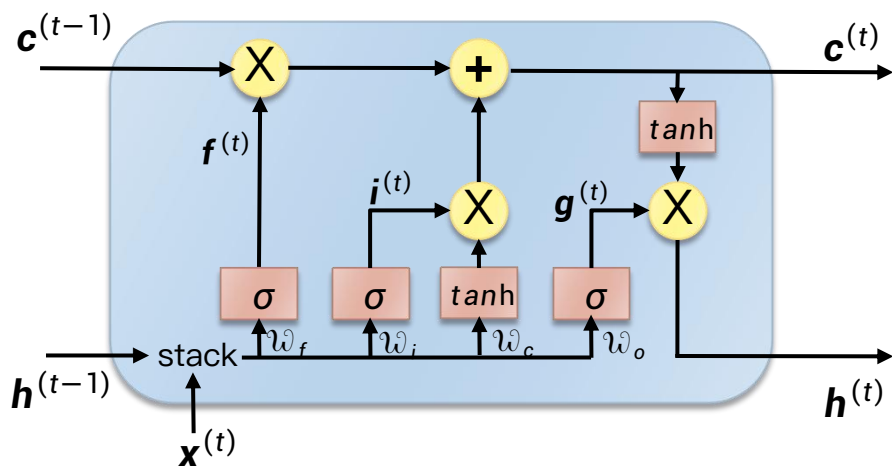
长短期记忆模型

► LSTM (Long Short-Term Memory networks)

■ Forget gate (遗忘门) : 记住前一时刻单元状态的多少内容 $f^{(t)} = \sigma(U^f x^{(t)} + W^f h^{(t-1)} + b^f) = \sigma\left(\mathcal{W}_f \begin{pmatrix} h^{(t-1)} \\ x^{(t)} \end{pmatrix} + b^f\right)$

■ Input gate (输入门) : 写入多少输入到当前单元 $i^{(t)} = \sigma(U^i x^{(t)} + W^i h^{(t-1)} + b^i) = \sigma\left(\mathcal{W}_i \begin{pmatrix} h^{(t-1)} \\ x^{(t)} \end{pmatrix} + b^i\right)$

■ Output gate (输出门) : 输出多少当前单元状态 $g^{(t)} = \sigma(U^o x^{(t)} + W^o h^{(t-1)} + b^o) = \sigma\left(\mathcal{W}_o \begin{pmatrix} h^{(t-1)} \\ x^{(t)} \end{pmatrix} + b^o\right)$



内部状态更新: $c^{(t)} = f^{(t)} c^{(t-1)} + i^{(t)} \tilde{c}^{(t)}$

$$\tilde{c}^{(t)} = \tanh(Ux^{(t)} + Wh^{(t-1)} + b) = \sigma\left(\mathcal{W}_c \begin{pmatrix} h^{(t-1)} \\ x^{(t)} \end{pmatrix} + b\right)$$

LSTM cell输出: $h^{(t)} = g^{(t)} \tanh(c^{(t)})$

长短期记忆模型

► LSTM变体

- 窥视孔连接(peephole connection): 门值不仅取决于 h_{t-1} 和 x_t , 也取决于上一个单元状态的值 c_{t-1}

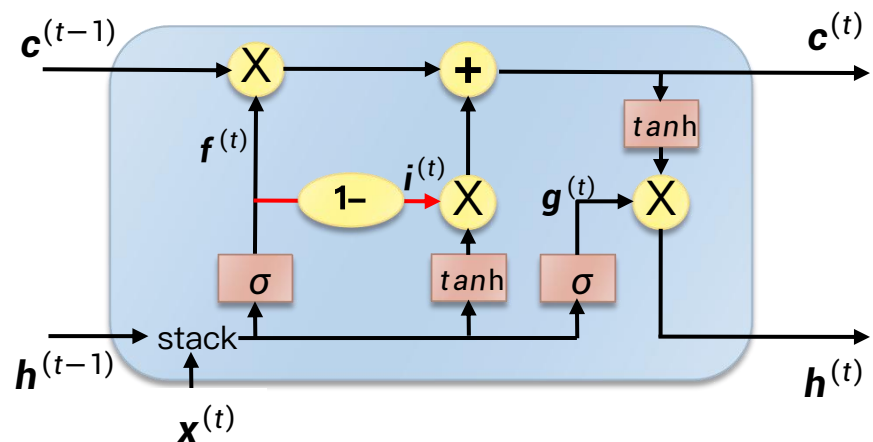
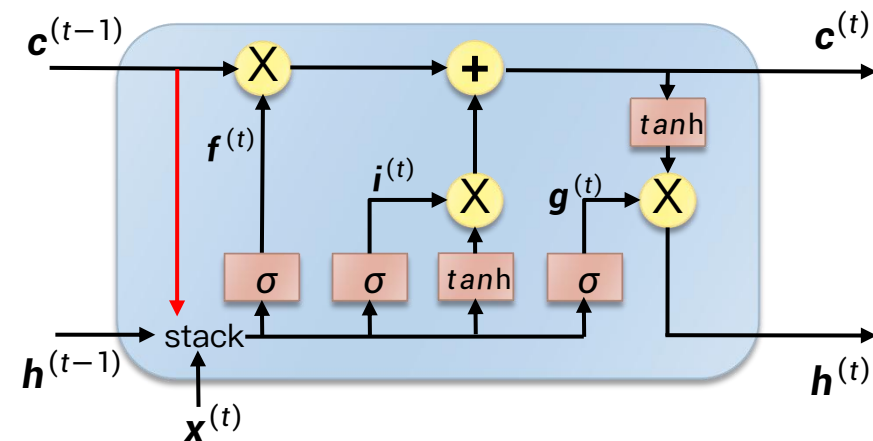
$$f^{(t)} = \sigma \left(U^f x^{(t)} + W^f h^{(t-1)} + M^f c^{(t-1)} + b^f \right)$$

$$i^{(t)} = \sigma \left(U^i x^{(t)} + W^i h^{(t-1)} + M^i c^{(t-1)} + b^i \right)$$

$$g^{(t)} = \sigma \left(U^o x^{(t)} + W^o h^{(t-1)} + M^o c^{(t-1)} + b^o \right)$$

- 耦合输入门和遗忘门: 不单独决定遗忘和新增信息, 在输入时一起做决定

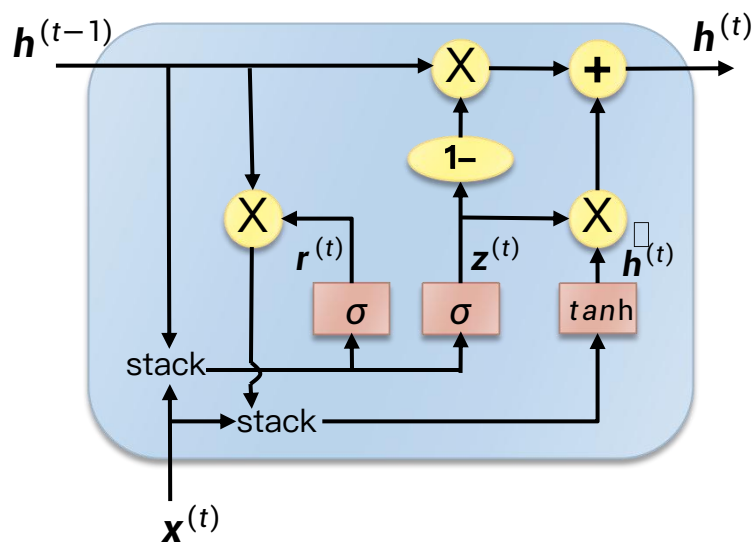
$$c^{(t)} = f^{(t)} c^{(t-1)} + (1 - f^{(t)}) \tilde{c}^{(t)}$$



长短期记忆模型

► GRU (Gated Recurrent Unit)

- Chung J, Gulcehre C, Cho K H, et al. Empirical evaluation of gated recurrent neural networks on sequence modeling[J]. arXiv preprint arXiv:1412.3555, 2014.
- 在LSTM的基础上，将单元状态和隐藏状态合并，将遗忘门和输入门合并为更新门，无输出门。更新门决定历史信息和当前信息如何相加；重置门决定保留多少历史信息。



更新门 (update gate) : $z^{(t)} = \sigma \left(U^z x^{(t)} + W^z h^{(t-1)} + b^z \right)$

重置门 (reset gate) : $r^{(t)} = \sigma \left(U^r x^{(t)} + W^r h^{(t-1)} + b^r \right)$

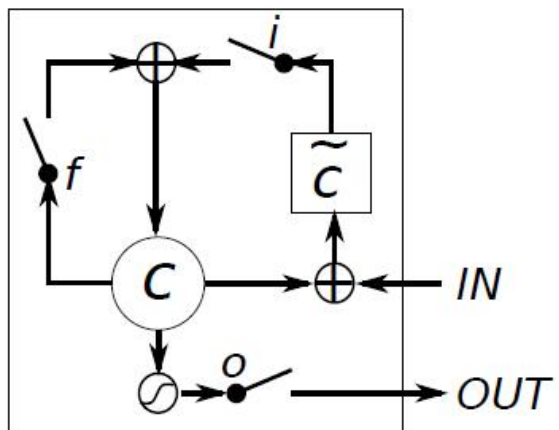
隐藏状态更新:

$$h^{(t)} = (1 - z^{(t)})h^{(t-1)} + z^{(t)}\tilde{h}^{(t)}$$

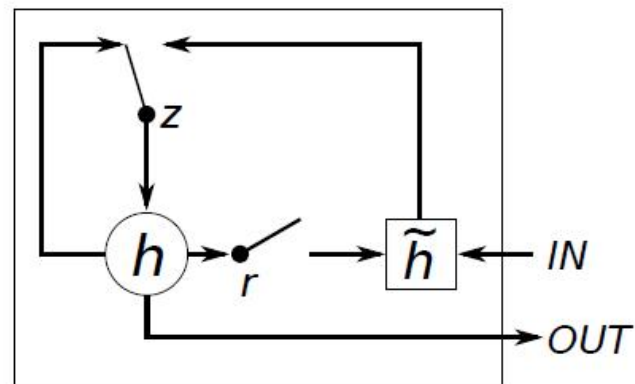
$$\tilde{h}^{(t)} = \tanh \left(Ux^{(t)} + W(r^{(t)}h^{(t-1)}) + b \right)$$

长短期记忆模型

► LSTM与GRU



(a) Long Short-Term Memory



(b) Gated Recurrent Unit

- 哪个模型更好无定论。GRU参数量更少，训练速度快；在训练数据足够的情况下，LSTM的表征能力更强；可自行选择；

提纲

- ▶ 适合图像处理的卷积神经网络
- ▶ 适合语音/文本处理的循环神经网络
- ▶ 从深度学习到大模型
- ▶ 神经网络量化
- ▶ 本章小结

Seq2Seq

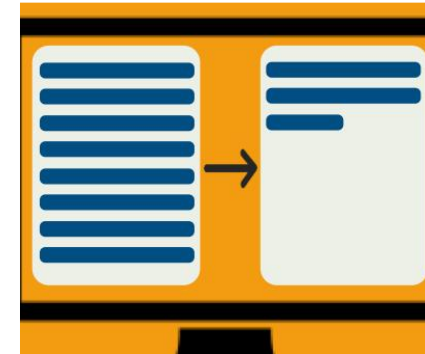
- ▶ Seq2Seq: Sequence-to-Sequence, 序列到序列模型
 - ▶ 一种特殊的语言模型, 是自然语言处理领域的重要模型
 - ▶ 应用: 将一种时序数据转换为另一种时序数据



机器翻译



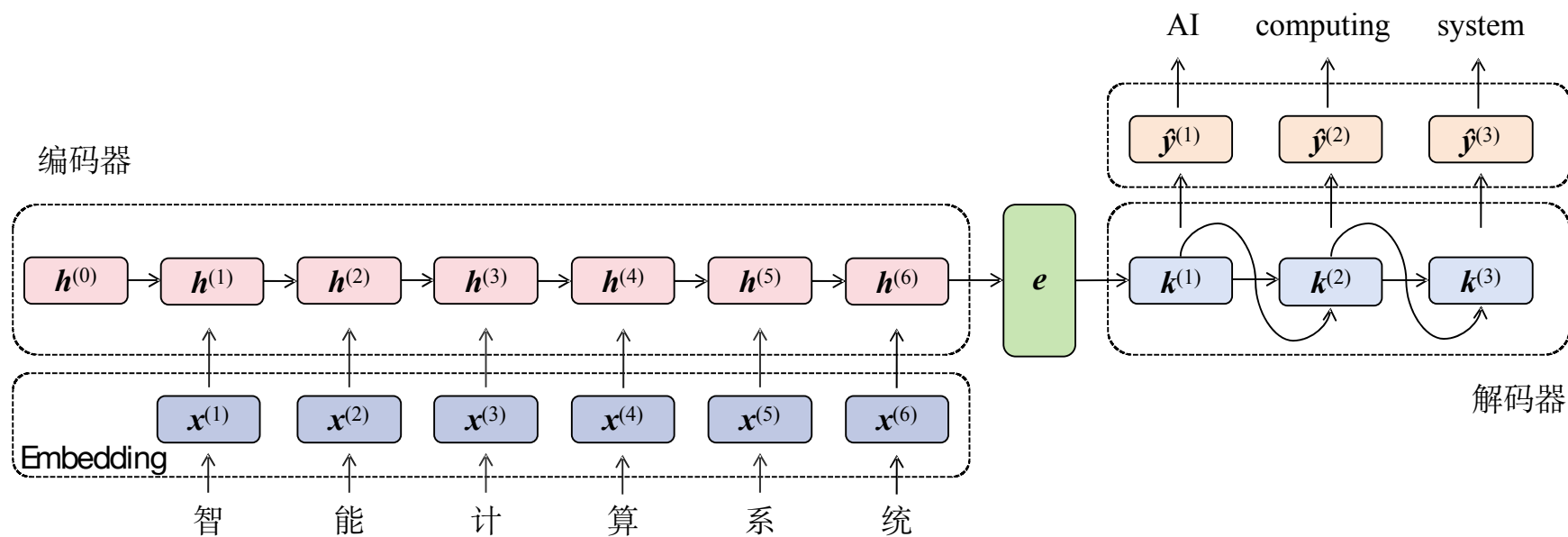
对话系统



自动文摘

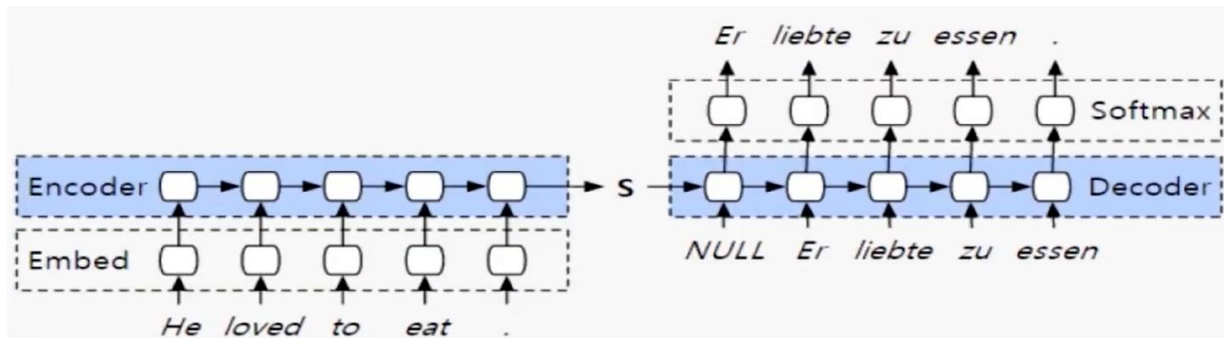
Seq2Seq

- ▶ 由编码器 (encoder)和解码器 (decoder)两部分组成
- ▶ 编码器根据输入数据生成语义编码
- ▶ 解码器根据该语义编码输出处理结果
- ▶ Seq2Seq模型适用于输入序列和输出序列不等长的情况

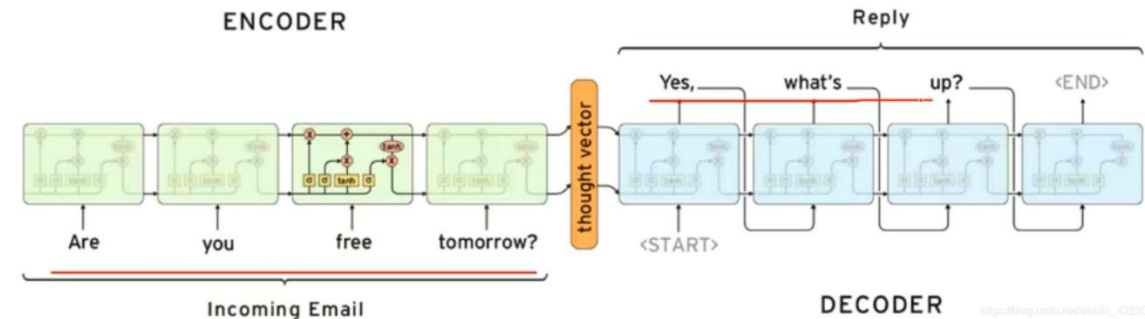


Seq2Seq

- ▶ 传统Seq2Seq模型通常使用RNN或LSTM作为编码器和解码器



基于RNN的Seq2Seq

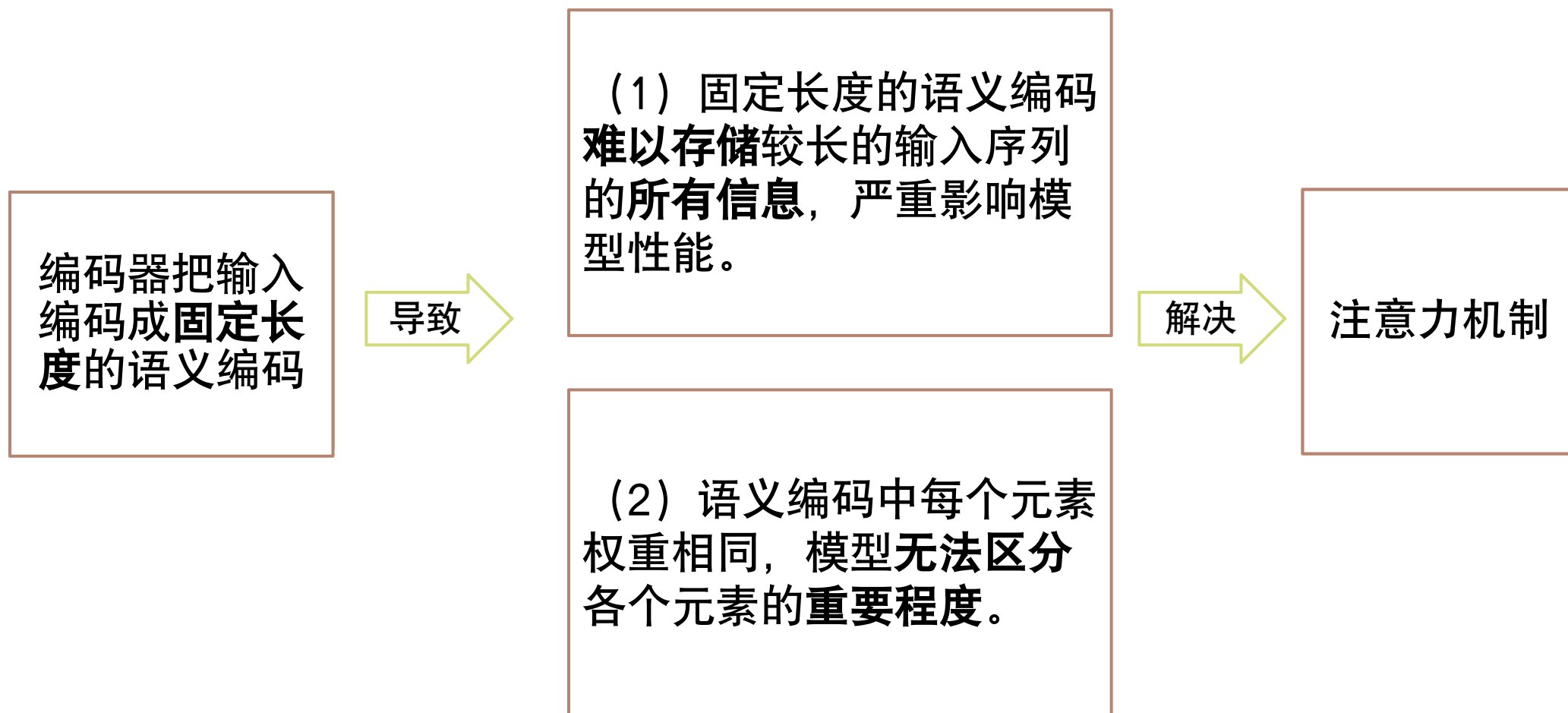


基于LSTM的
Seq2Seq

[1] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, Yoshua Bengio, Learning Phrase Representation using RNN Encoder-Decoder for Statistical Machine Translation, EMNLP2014.

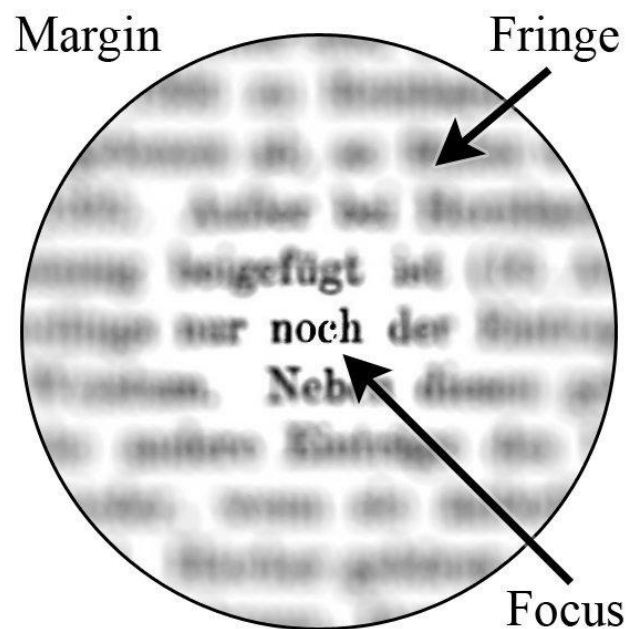
[2] Ilya Sutskever, Oriol Vinyals, Quoc V. Le, Sequence to Sequence Learning with Neural Networks, NIPS2014.

Seq2Seq



注意力机制

- ▶ 注意力机制可以抽取少量**重要信息**，并聚焦于这些重要信息，忽略大多数不重要的信息
- ▶ 注意力机制可以有效地提升基于RNN（LSTM）的Seq2Seq模型的信息处理能力

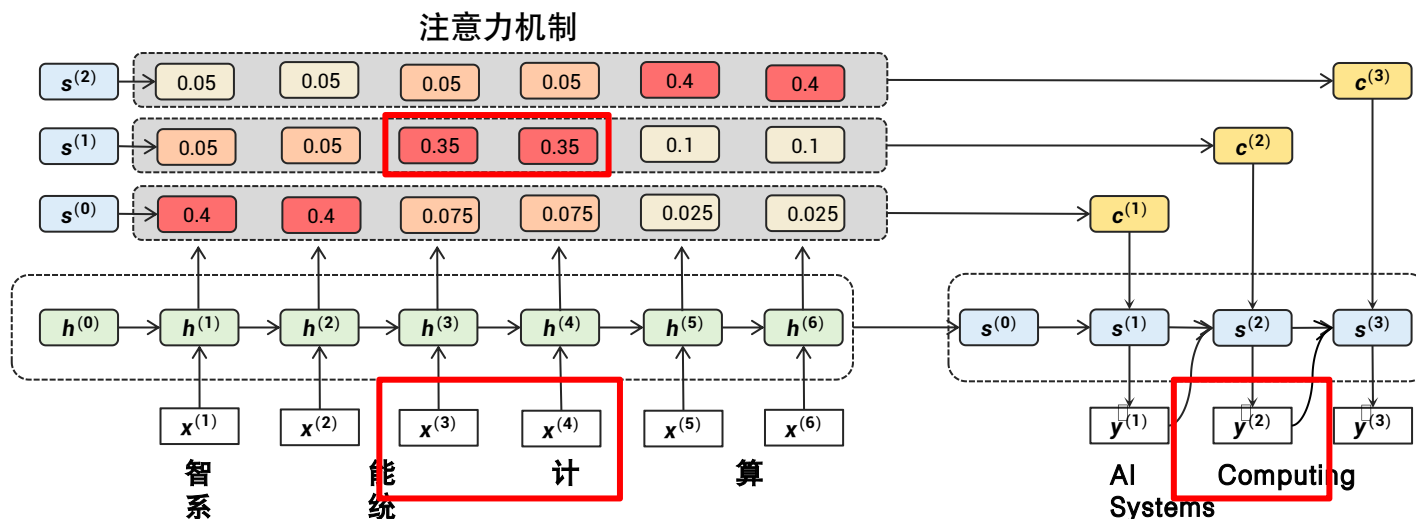


视觉神经系统的注意力机制

D Bahdanau, K Cho, Y Bengio, Neural machine translation by jointly learning to align and translate, ICLR2015.

注意力机制

- ▶ 注意力机制通过对输入部分赋予不同的权重，有效抽取关键信息



1. 计算相似度:

$$a_{ij} = \text{softmax}(a(s^{(i-1)}, h^{(j)}))$$

2. 注意力汇聚:

$$c^{(i)} = \sum_j a_{ij} h^{(j)}$$

- ▶ 注意力机制的优势

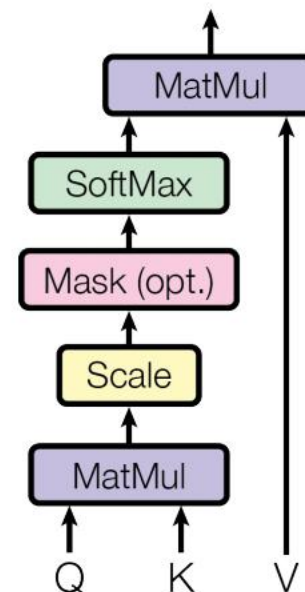
- ▶ 聚焦关键信息，使模型做出更加准确的判断
- ▶ 可以并行运算，比RNN等序列化计算的速度快

D Bahdanau, K Cho, Y Bengio, Neural machine translation by jointly learning to align and translate, ICLR2015.

注意力机制：缩放点积注意力

- ▶ 输入：查询Q、键K、值V，每个值V都与一个键K配对
- ▶ 原理：根据Q和K的相似度计算V的权重
- ▶ 计算过程：
 - ▶ 计算Q和K的向量内积，得到输入中词与词之间的相关关系
 - ▶ 经过尺度变换（scale）、掩码（Mask）和Softmax等处理，得到权重矩阵
 - ▶ 将权重矩阵与V相乘（注意力汇聚）

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \text{Mask} \right) V$$



缩放点积注意力

Scaled dot-product attention

除以 $\sqrt{d_k}$ ： q 和 k 中数值都是均值为0，方差为1， qk 内积的方差为 d_k 。当 d_k 较大时， qk 内积值可能会很大，除以 $\sqrt{d_k}$ ，使 qk 内积的方差变为1，以获得较大的softmax梯度，防止梯度消失

注意力机制：自注意力

自注意力 (self-attention)

- ▶ 查询Q、键K、值V都来源于同一组输入
- ▶ 自注意力机制擅长捕捉序列数据内部的相关性

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{Mask}\right) V$$

$$Q = IW^Q$$

$$K = IW^K$$

$$V = IW^V$$

传统注意力

- ▶ 基于输入和输出计算注意力权重

VS

自注意力

- ▶ 先对输入自身进行计算
- ▶ 再把输入端的自注意力权重加入到对输出的影响中

翻译：

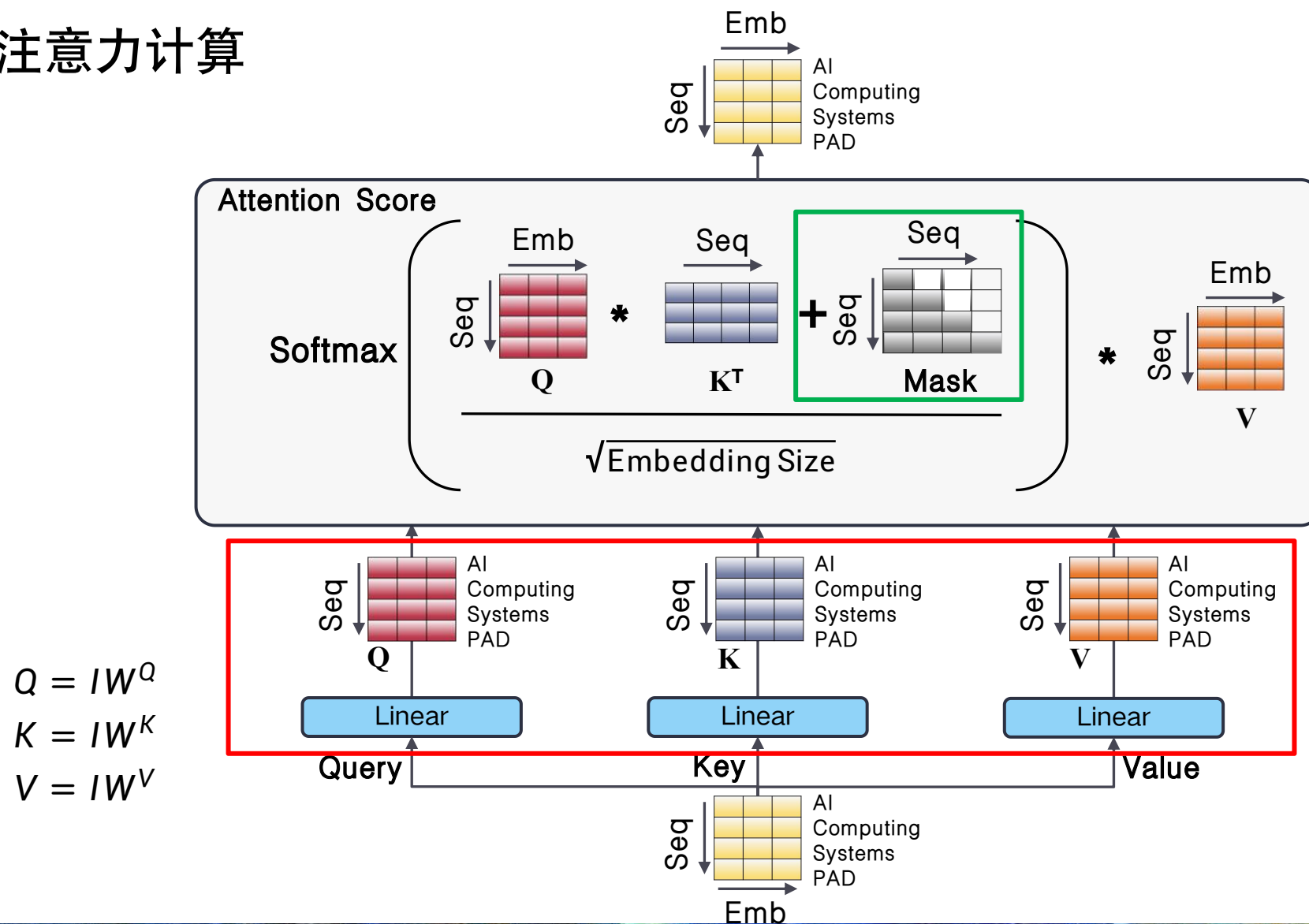
苹果是我最喜欢的水果 apple
苹果是我最喜欢的手机 Iphone

- ▶ 仅能捕捉输入和输出之间的相关关系

- ▶ 不仅可以捕捉输入和输出的相关关系，还可以捕捉输入内部的相关关系

Transformer：自注意力

► 自注意力计算



注意力机制：多头注意力

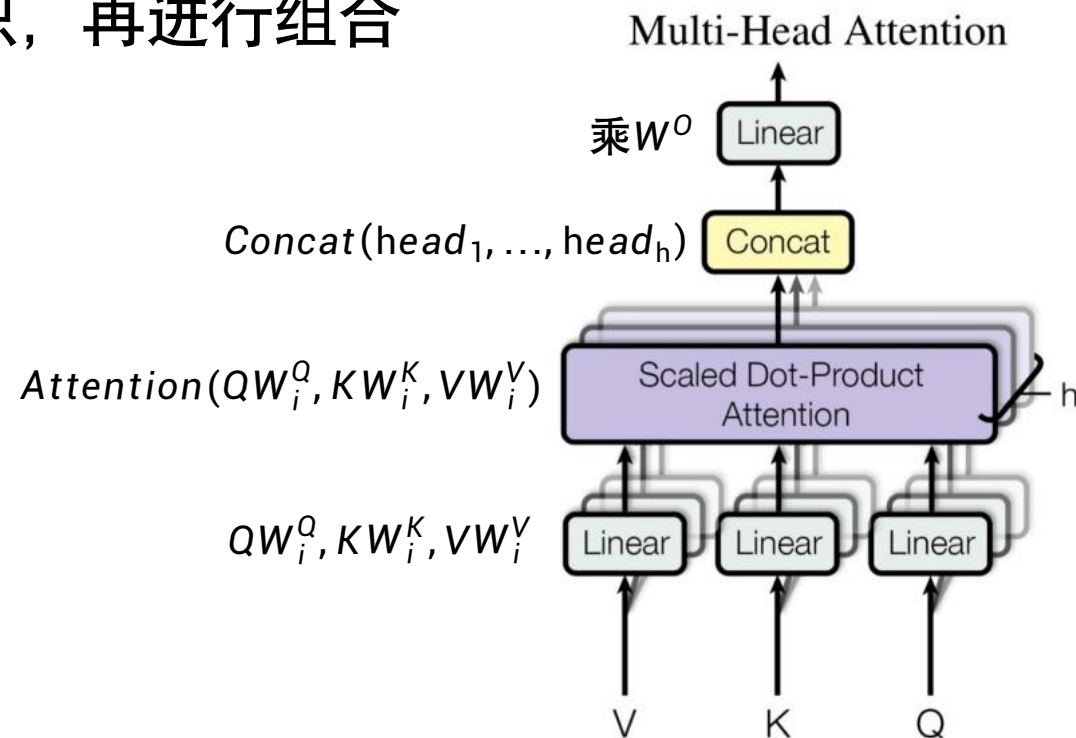
▶ 原理：基于相同的注意力机制学习不同的知识，再进行组合

▶ 计算过程

- ▶ 对QKV进行 h 组不同的线性变换
- ▶ 将 h 组不同线性变换后的QKV并行计算注意力
- ▶ 将 h 组注意力计算结果拼接在一起，并进行线性映射

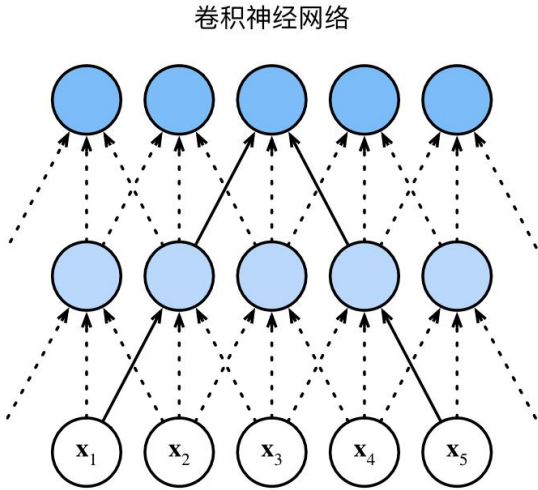
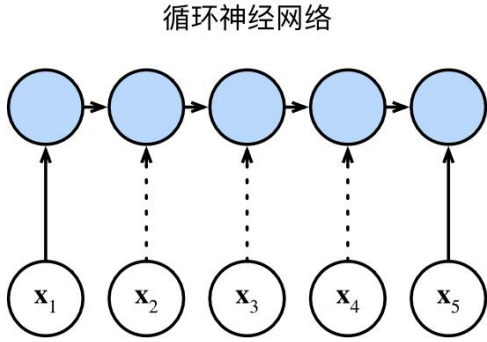
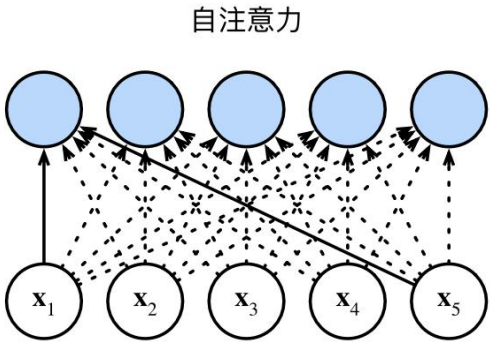
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where, } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



复杂度分析

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

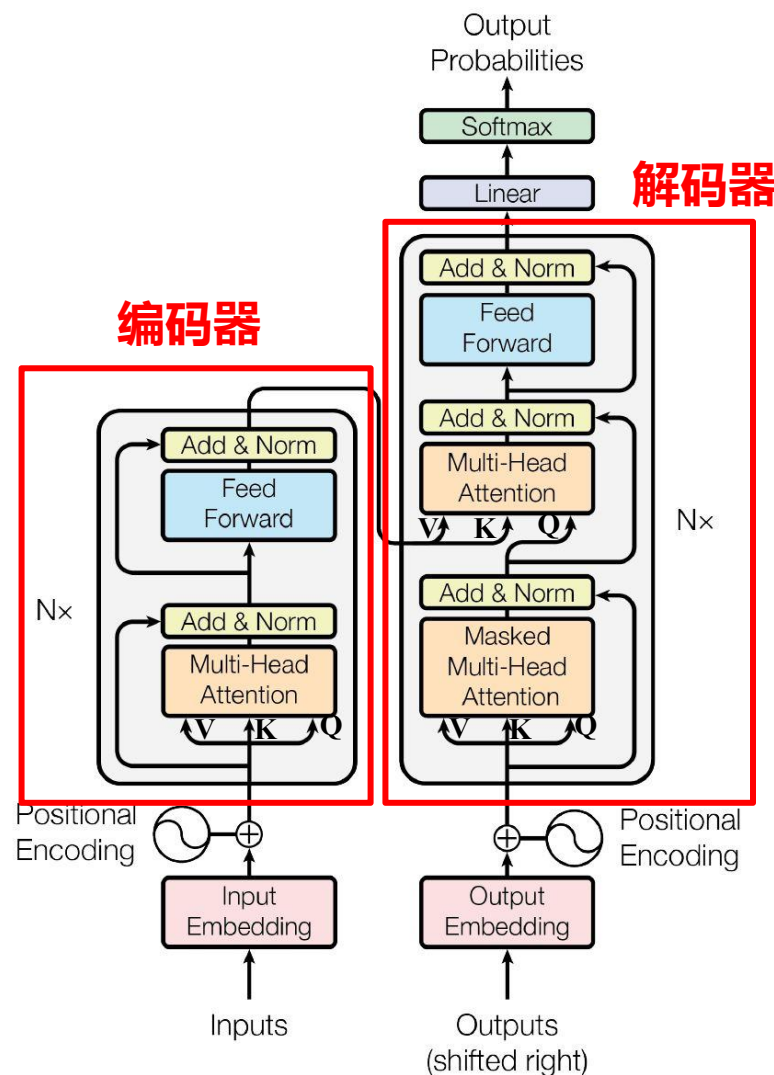
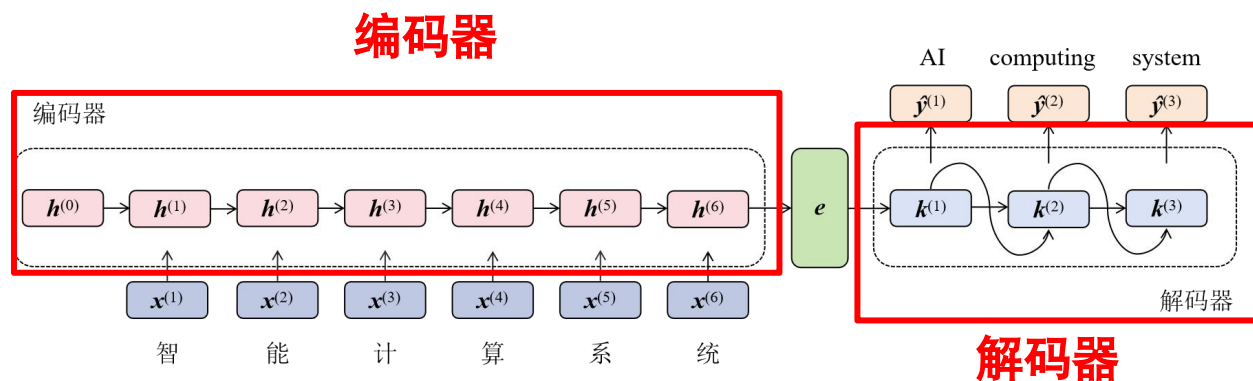


Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. NeurIPS'2017.

Transformer

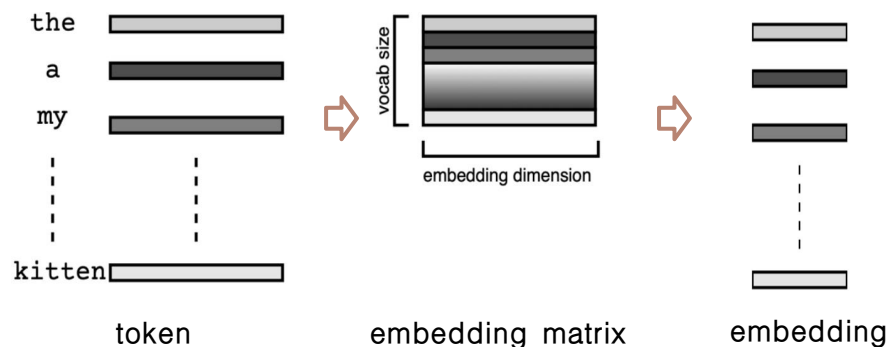
Transformer结构特点:

- 遵循seq2seq结构, 编码器 (encoder) + 解码器 (decoder)
- 去掉了循环神经网络结构 (效率低, 无法并行训练)
- 大量重复使用attention

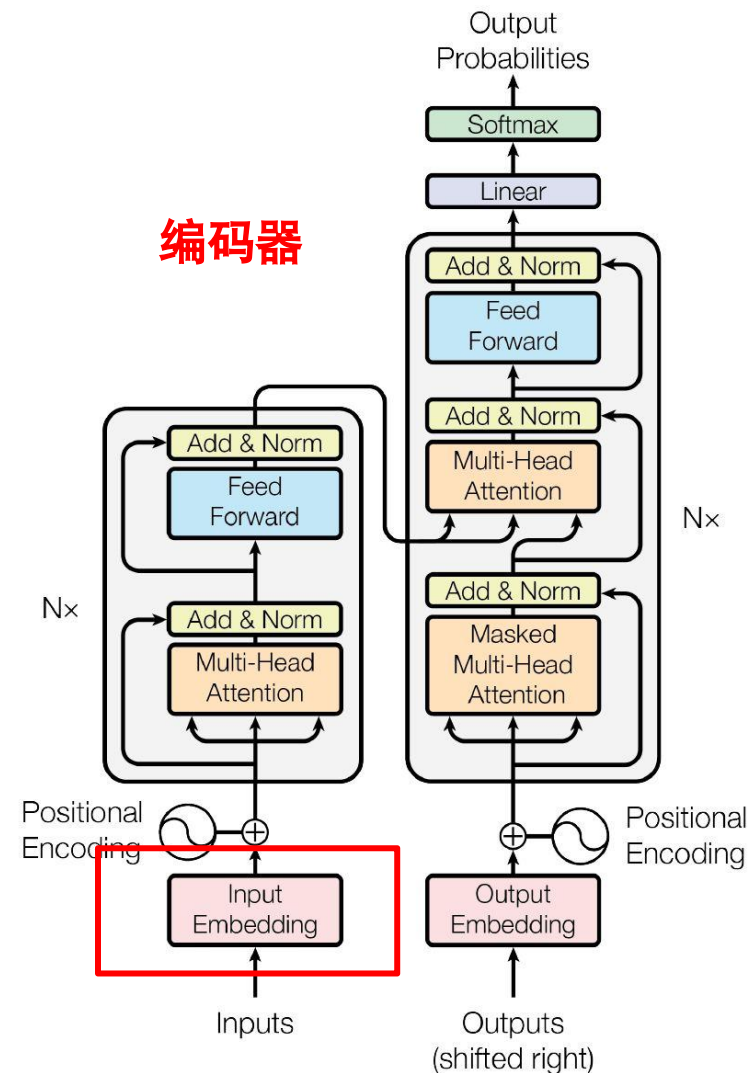


Transformer

词元嵌入 (Token embedding)



Token String	Token ID	Embedded Token Vector
'<s>' ->	0 ->	[0.1150, -0.1438, 0.0555, ...]
'<pad>' ->	1 ->	[0.1149, -0.1438, 0.0547, ...]
'</s>' ->	2 ->	[0.0010, -0.0922, 0.1025, ...]
'<unk>' ->	3 ->	[0.1149, -0.1439, 0.0548, ...]
'.'	4 ->	[-0.0651, -0.0622, -0.0002, ...]
' the'	5 ->	[-0.0340, 0.0068, -0.0844, ...]
','	6 ->	[0.0483, -0.0214, -0.0927, ...]
' to'	7 ->	[-0.0439, 0.0201, 0.0189, ...]
' and'	8 ->	[0.0523, -0.0208, -0.0254, ...]
' of'	9 ->	[-0.0732, 0.0070, -0.0286, ...]
' a'	10 ->	[-0.0194, 0.0302, -0.0838, ...]
...		



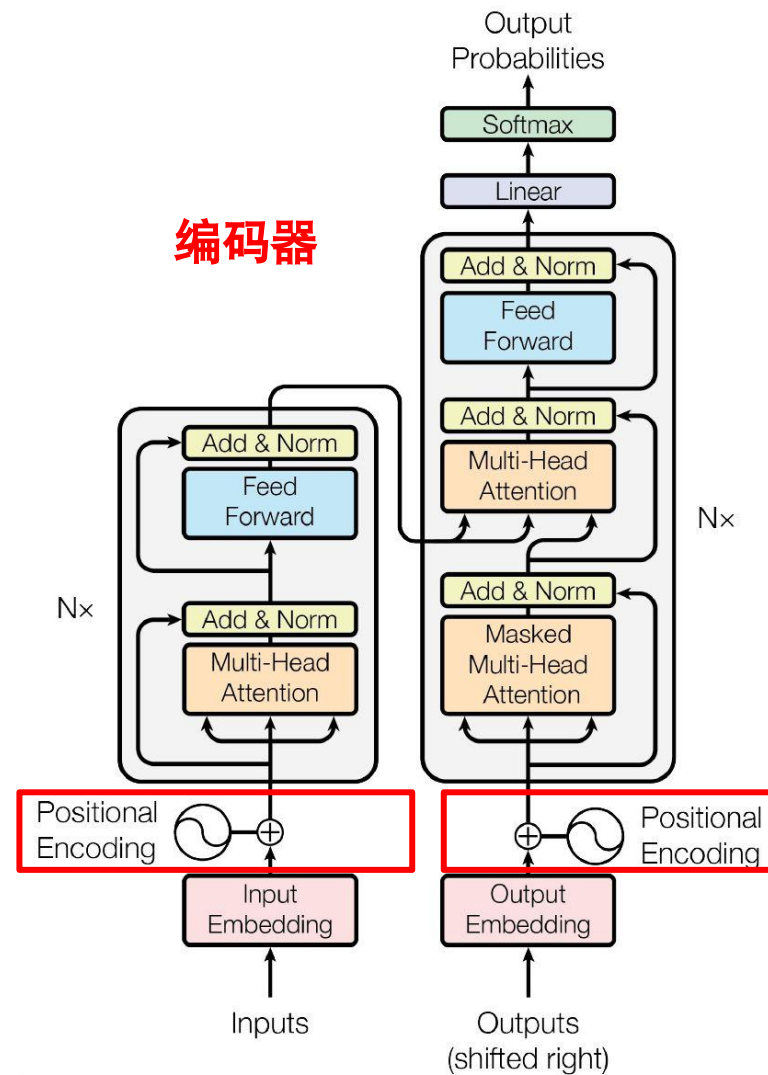
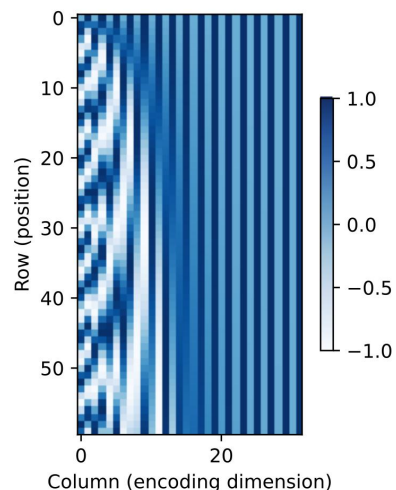
Transformer

▶ 位置编码 (Position Encoding)

- ▶ 使用了波长从 2π 到 $10000 \cdot 2\pi$ 的不同正弦和余弦函数计算, 函数的输入为当前嵌入结果在序列中的位置pos以及嵌入结果中每个元素所在的维度 $2i$ 或 $2i+1$

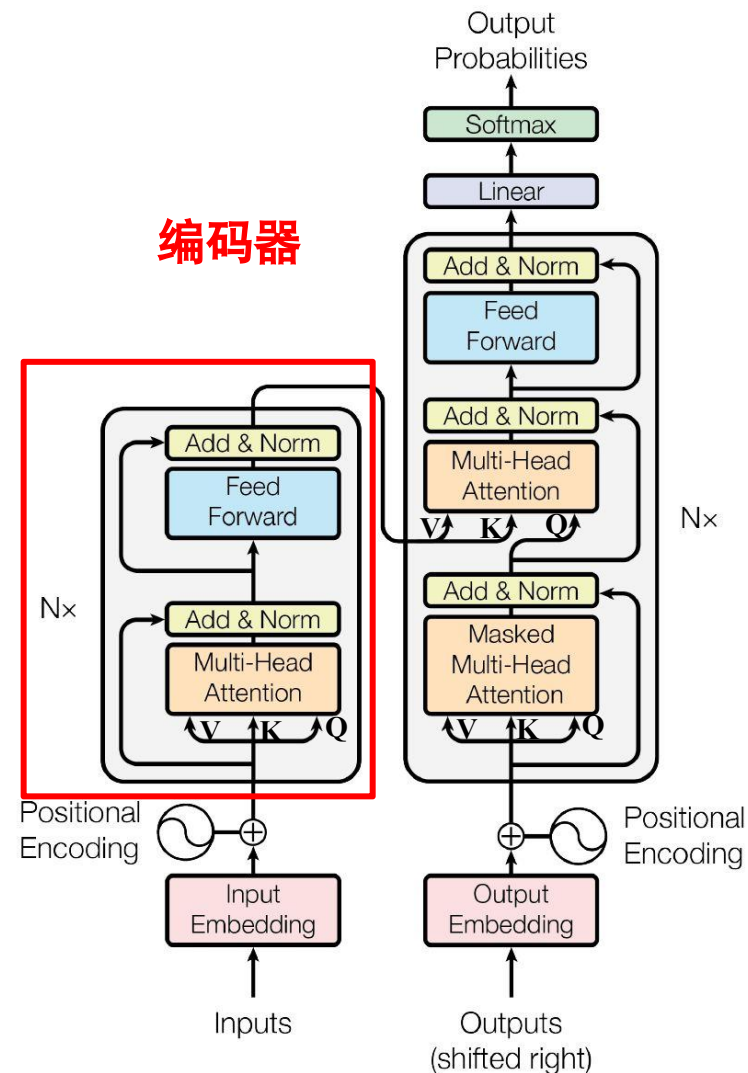
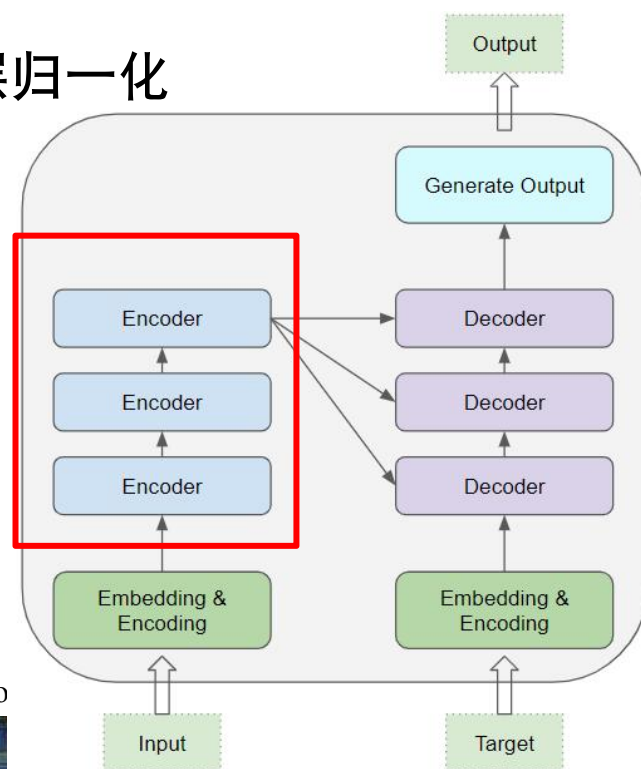
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$



Transformer

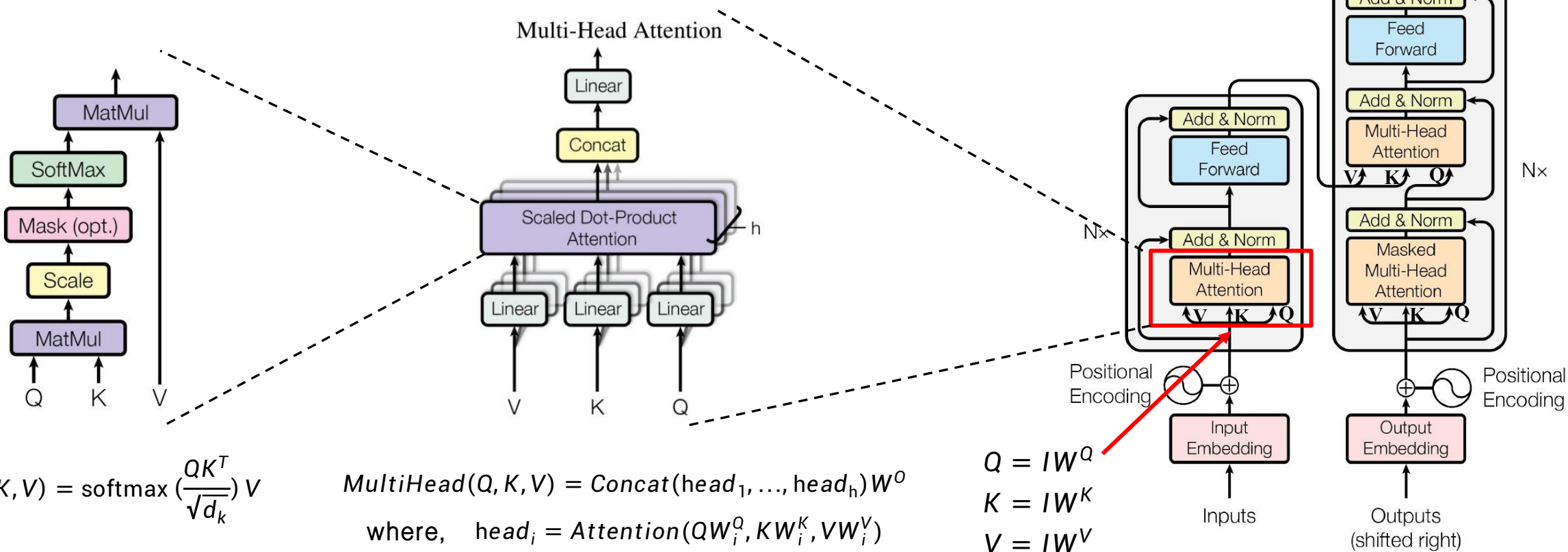
- ▶ 编码器：N个相同的层组成 (N=6)
 - ▶ 每层由两个子层(Sub-layer)组成
 - ▶ 多头注意力机制(Multi-Head Self-Attention)
 - ▶ 前馈网络(Feed-forward network)
 - ▶ 每个子层还增加了残差连接和层归一化



Transformer

► 编码器layer

- 多头自注意力: QKV是同源的, 无mask



Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. NeurIPS'2017.

Transformer

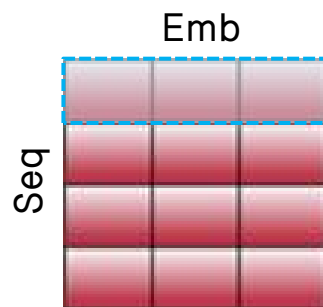
▶ 层归一化 (Layer Norm)

- ▶ 原理：将中间层的输入标准化为均值为0，方差为1的高斯分布
- ▶ 沿embedding方向，对每个嵌入向量分别进行层归一化

在每个token vector上计算

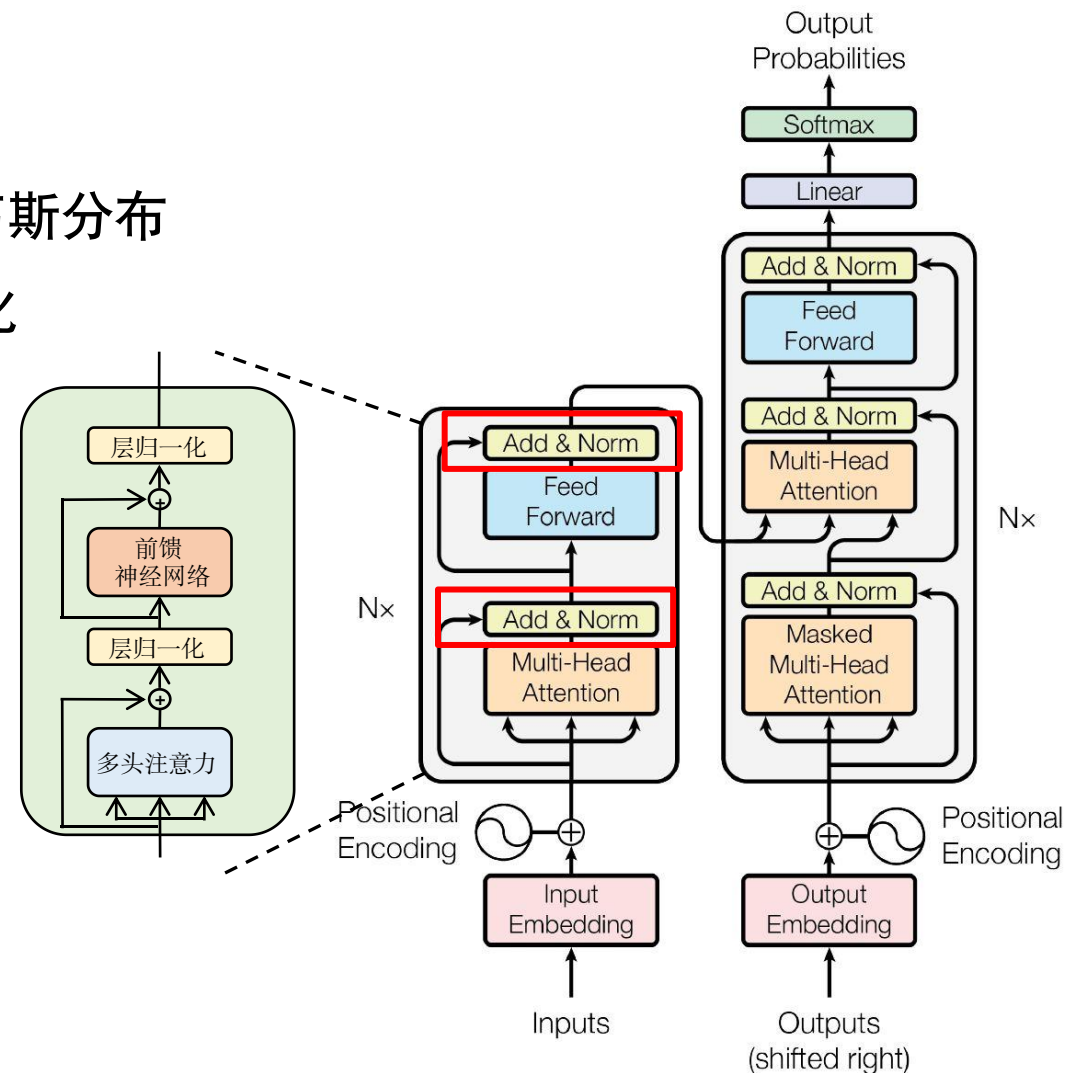
$$\text{layer-norm}(x) = \frac{x - \text{mean}(x)}{\sqrt{\text{variance}(x) + \epsilon}} \cdot \underbrace{\gamma + \beta}_{\text{Learnable parameters}}$$

Small additive constant



▶ 作用

- ▶ 缓解梯度消失问题，加快收敛速度
- ▶ 可以达到类似正则化的效果，防止过拟合
- ▶ 降低调参难度，可以使用较大的学习率

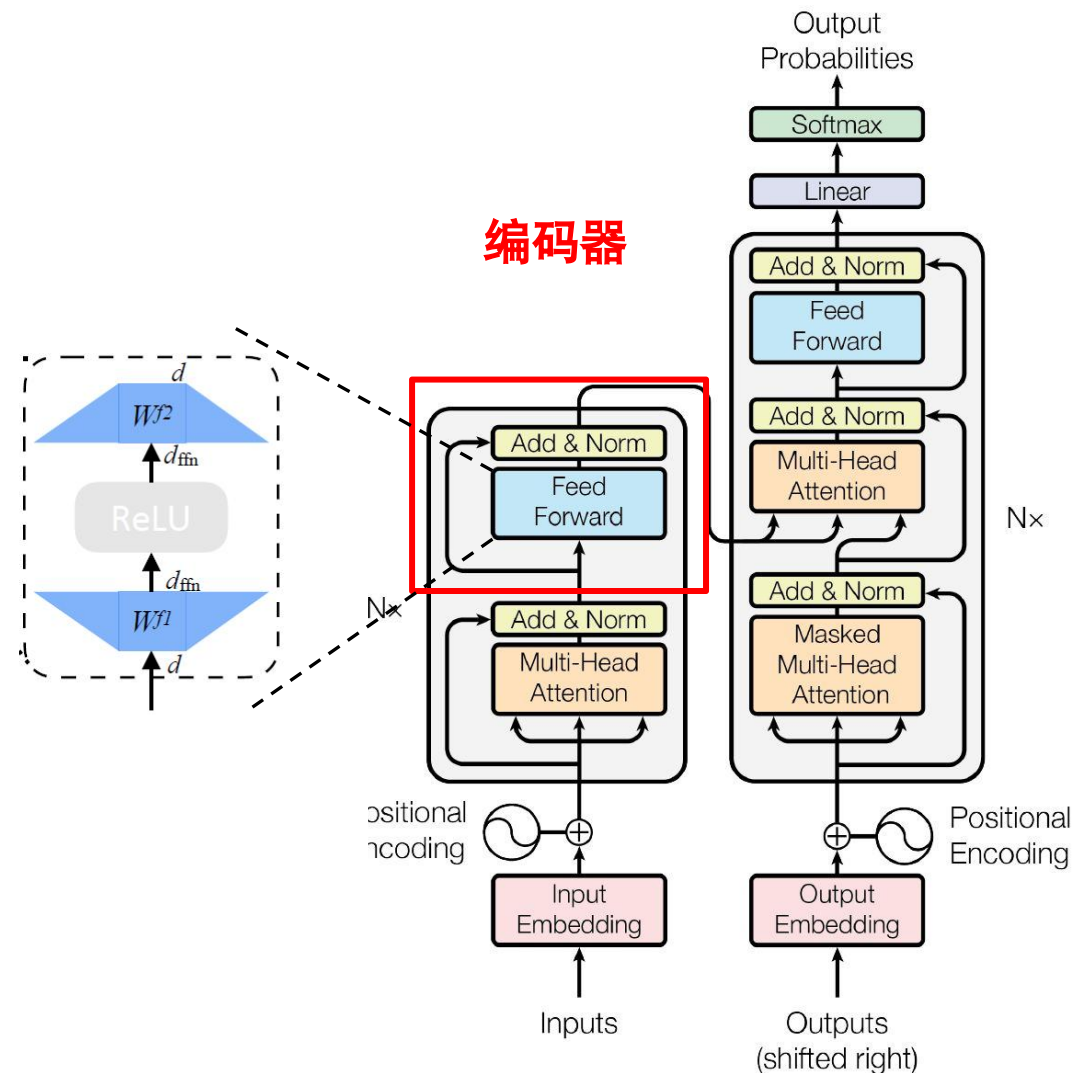
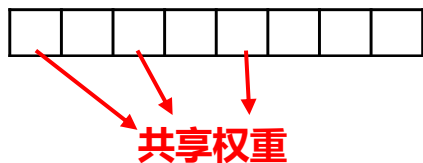


Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. "Layer Normalization." arXiv:1607.06450, 2016

Transformer

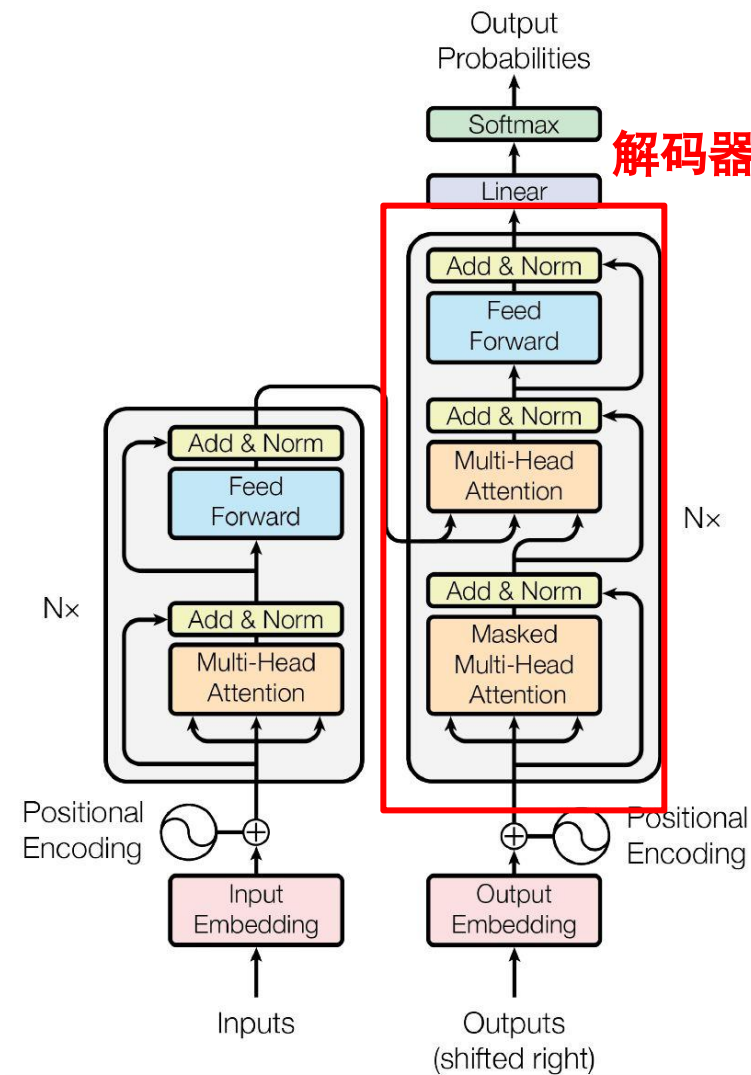
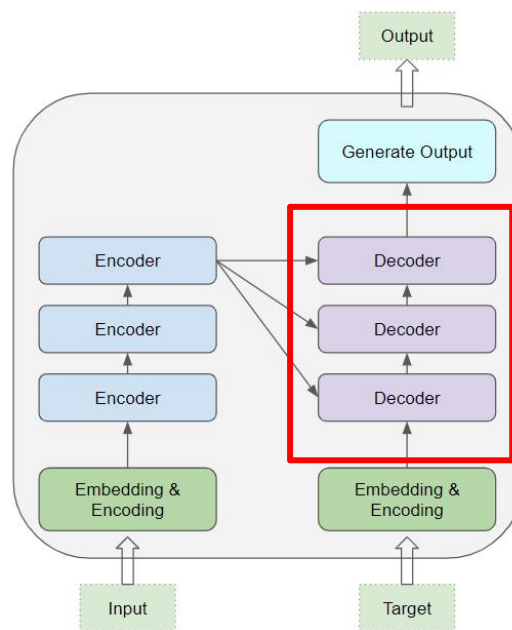
► 编码器layer

- FFN前馈子层 $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$
 - 先升维再降维: $512 \rightarrow 2048 \rightarrow 512$
 - 使用ReLU, 引入非线性变换, 增加特征表达能力
 - 序列中每个位置的计算都使用相同的线性变换权重



Transformer

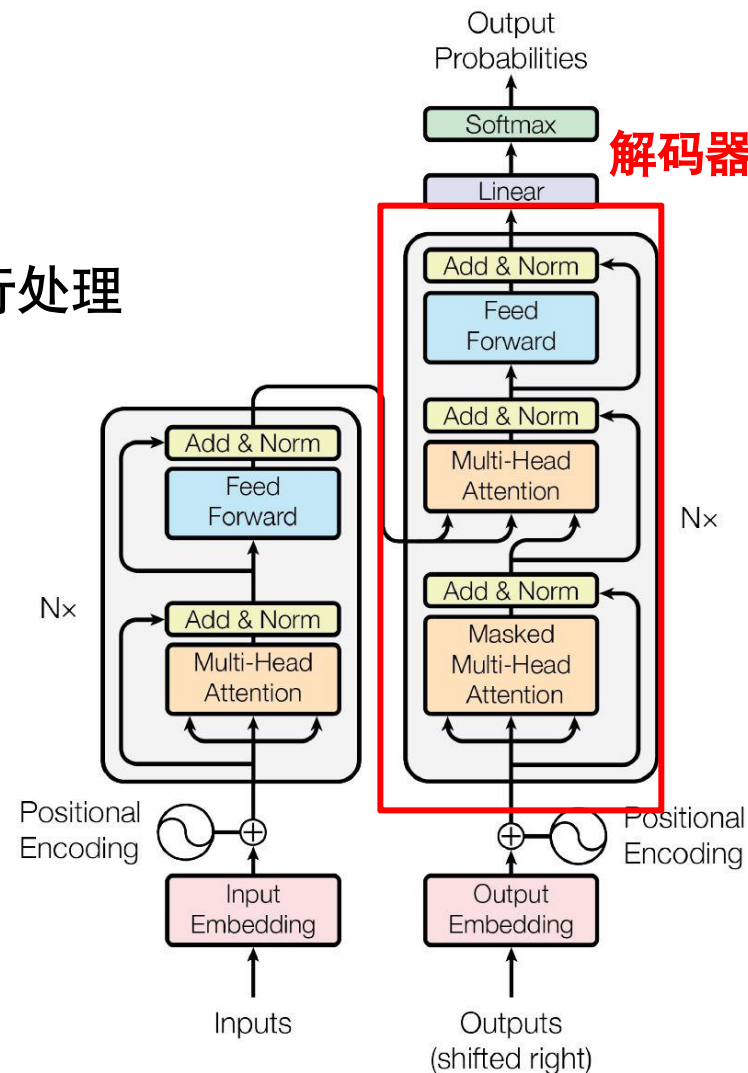
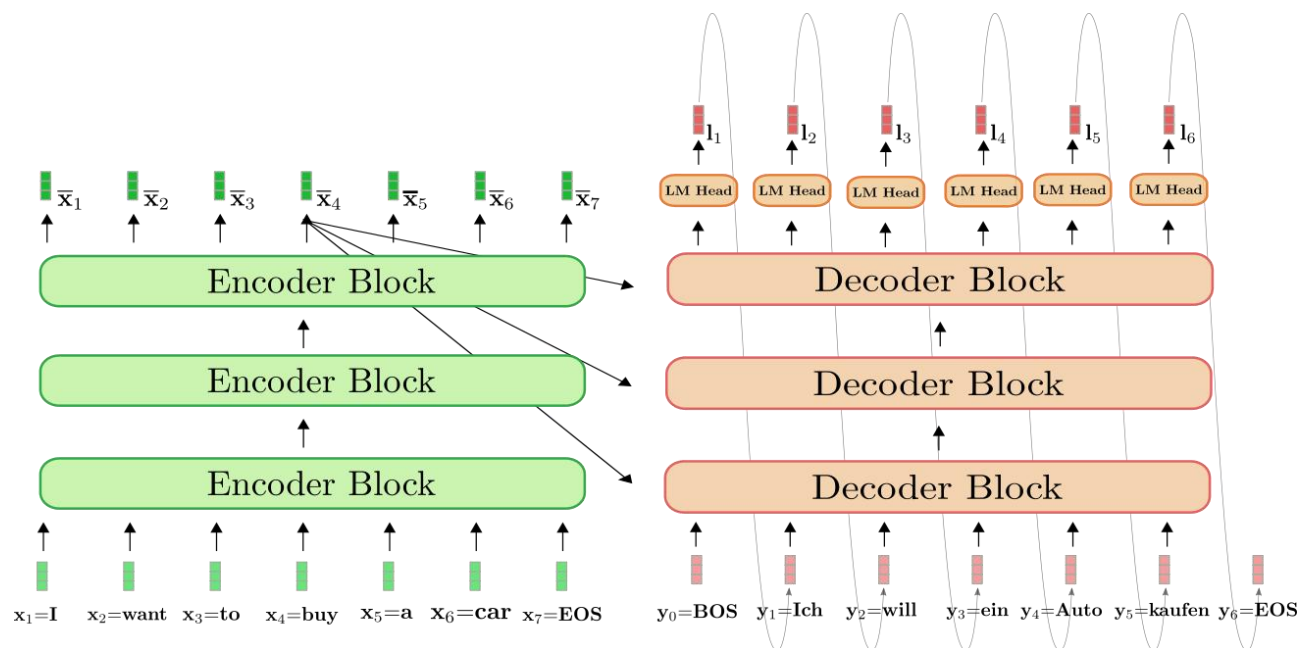
- ▶ 解码器: N个相同的层组成 (N=6)
 - ▶ 每层由三个子层(Sub-layer)组成
 - ▶ 带mask的多头自注意力机制(Multi-Head Self-Attention)
 - ▶ 交叉注意力子层 (cross-attention)
 - ▶ 前馈网络(Feed-forward network)
 - ▶ 每个子层增加了残差连接和层归一化



Transformer

▶ 解码器的输入

- ▶ 编码器的输出，以及上一个位置的解码器输出
- ▶ 训练时，有编码器输入对应的label（整体向右偏移一位），可以并行处理
- ▶ 推理时，没有label，需要逐个位置进行解码，无法并行

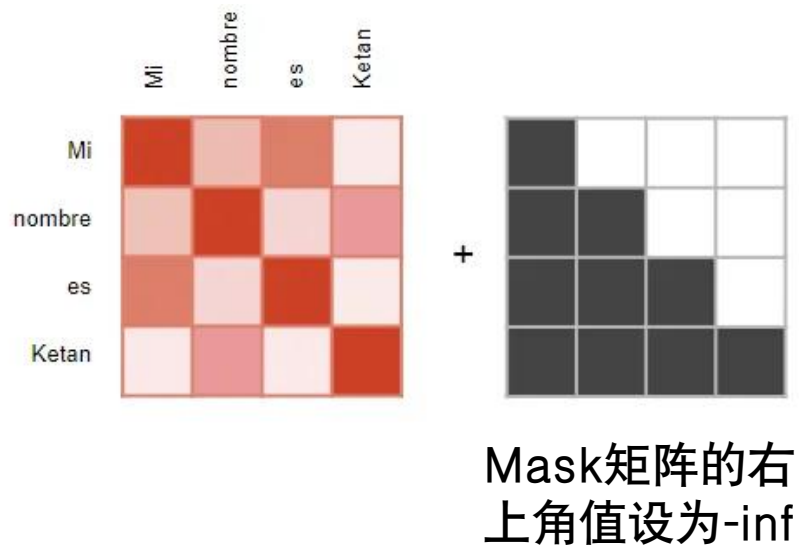


Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. NeurIPS'2017.

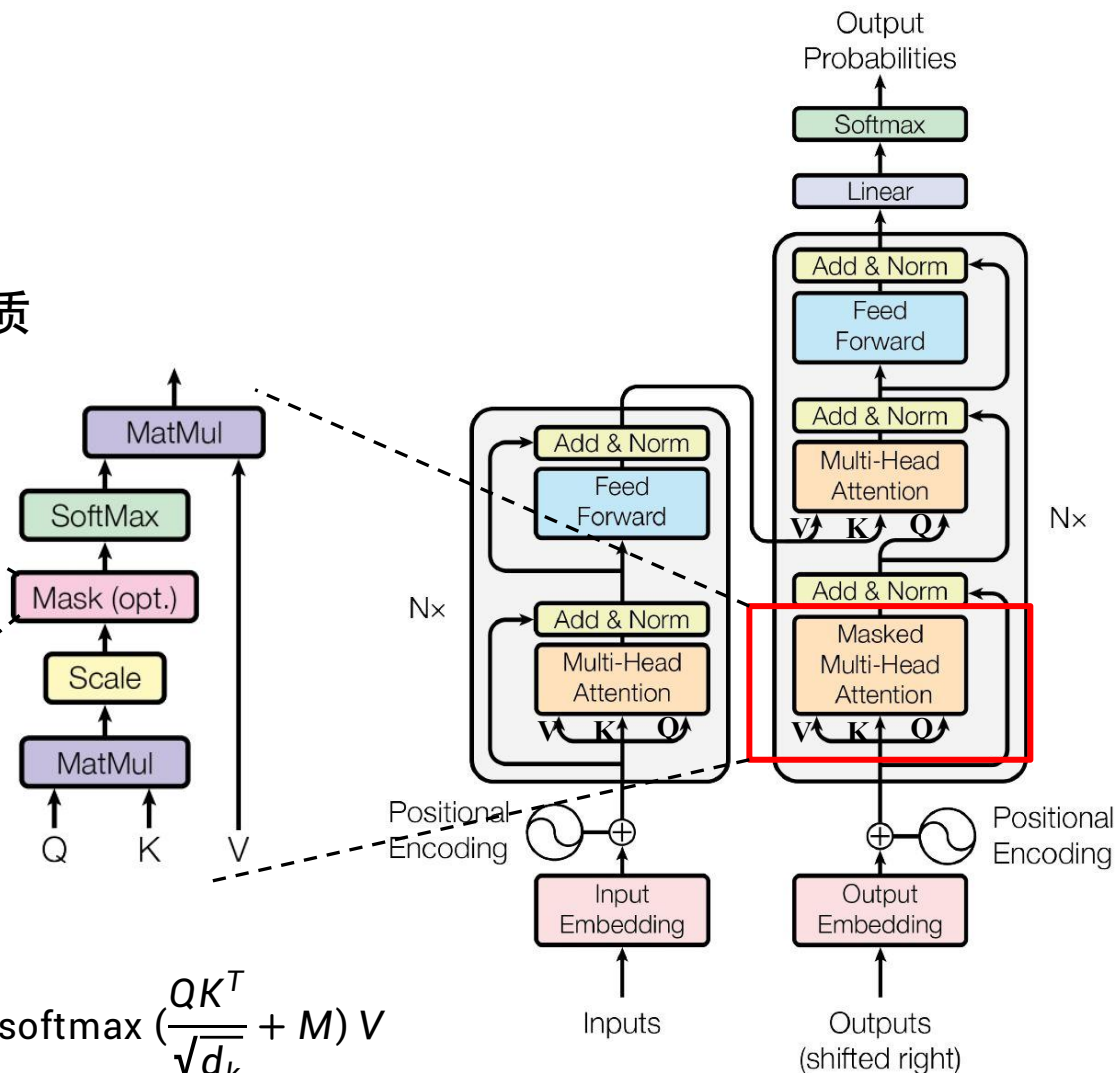
Transformer

解码器

- ▶ 解码器layer:
 - ▶ 带mask的自注意力子层
 - ▶ 捕捉输出序列内部的相关关系
 - ▶ mask去掉当前位置后面的token的影响, 满足自回归性质



$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$



Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. NeurIPS'2017.

Transformer

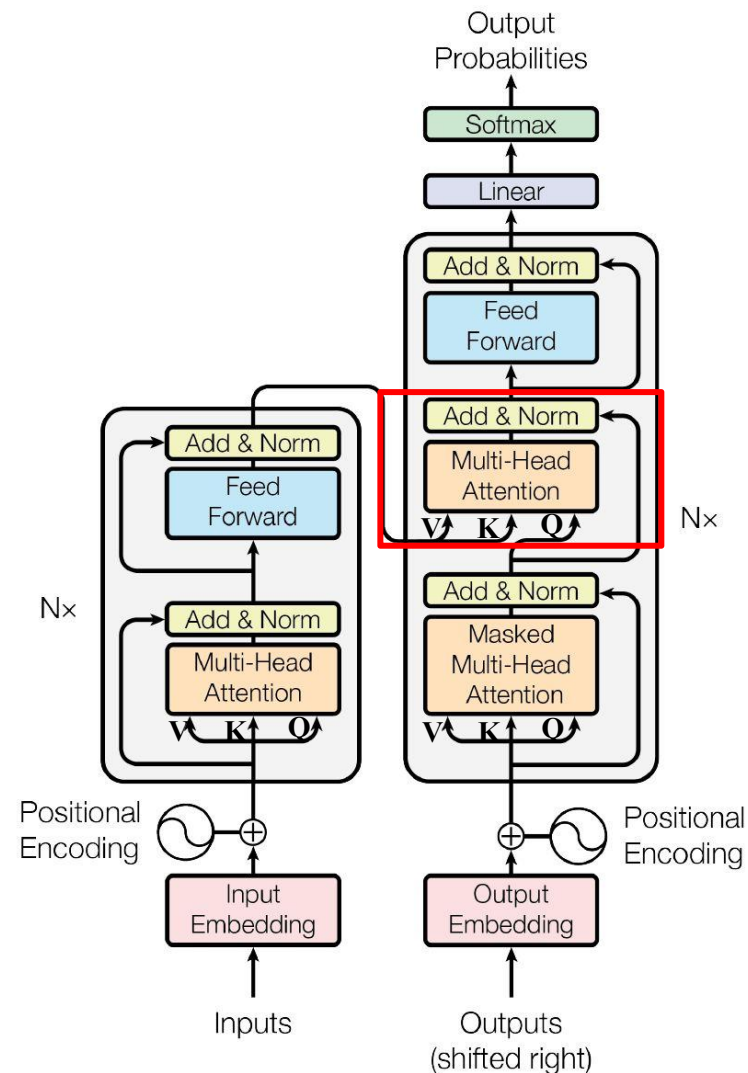
解码器

▶ 解码器layer:

▶ 交叉注意力子层

- ▶ 捕捉输入序列和输出序列之间的相关关系
- ▶ 残差连接来源于Q（解码器的自注意力子层的输出）

	N	d	d_{ff}	h	$dk=dv$	参数量
base	6	512	2048	8	64	65M
large	6	1024	4096	16	64	213M

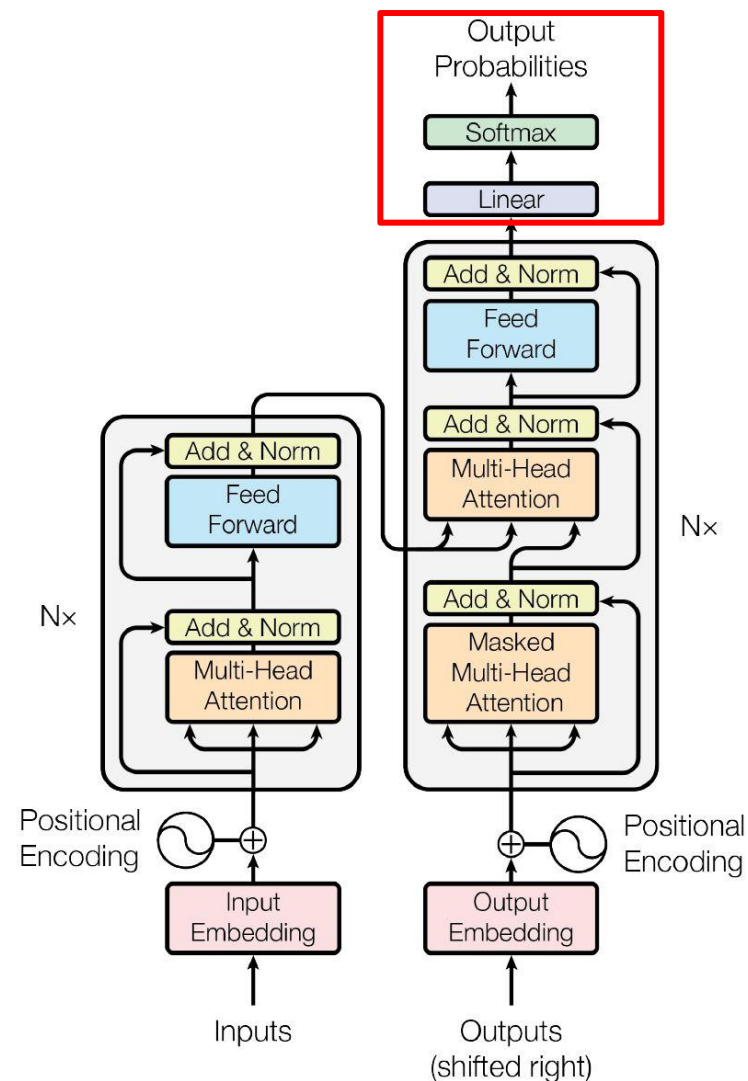
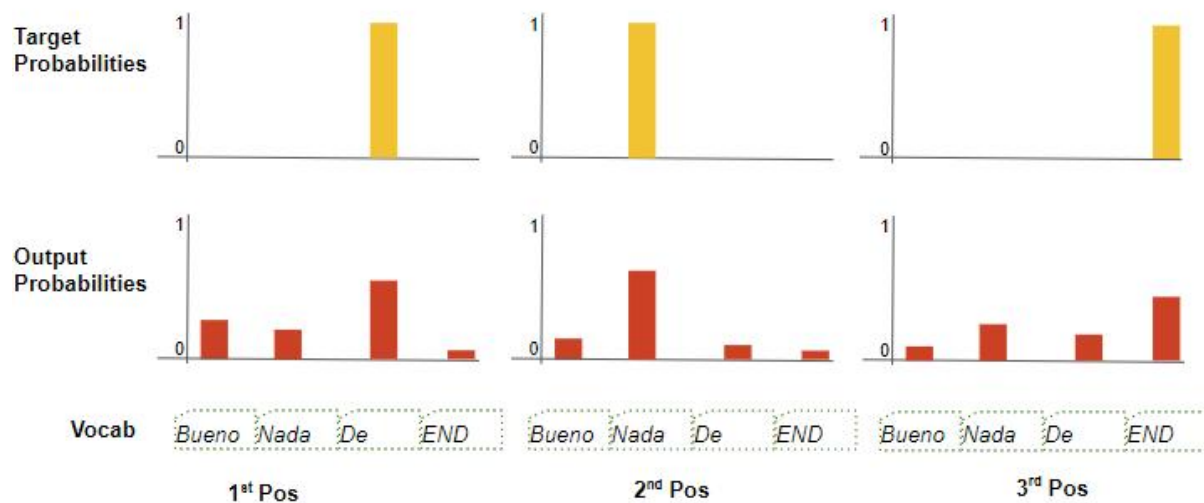


Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. NeurIPS'2017.

Transformer

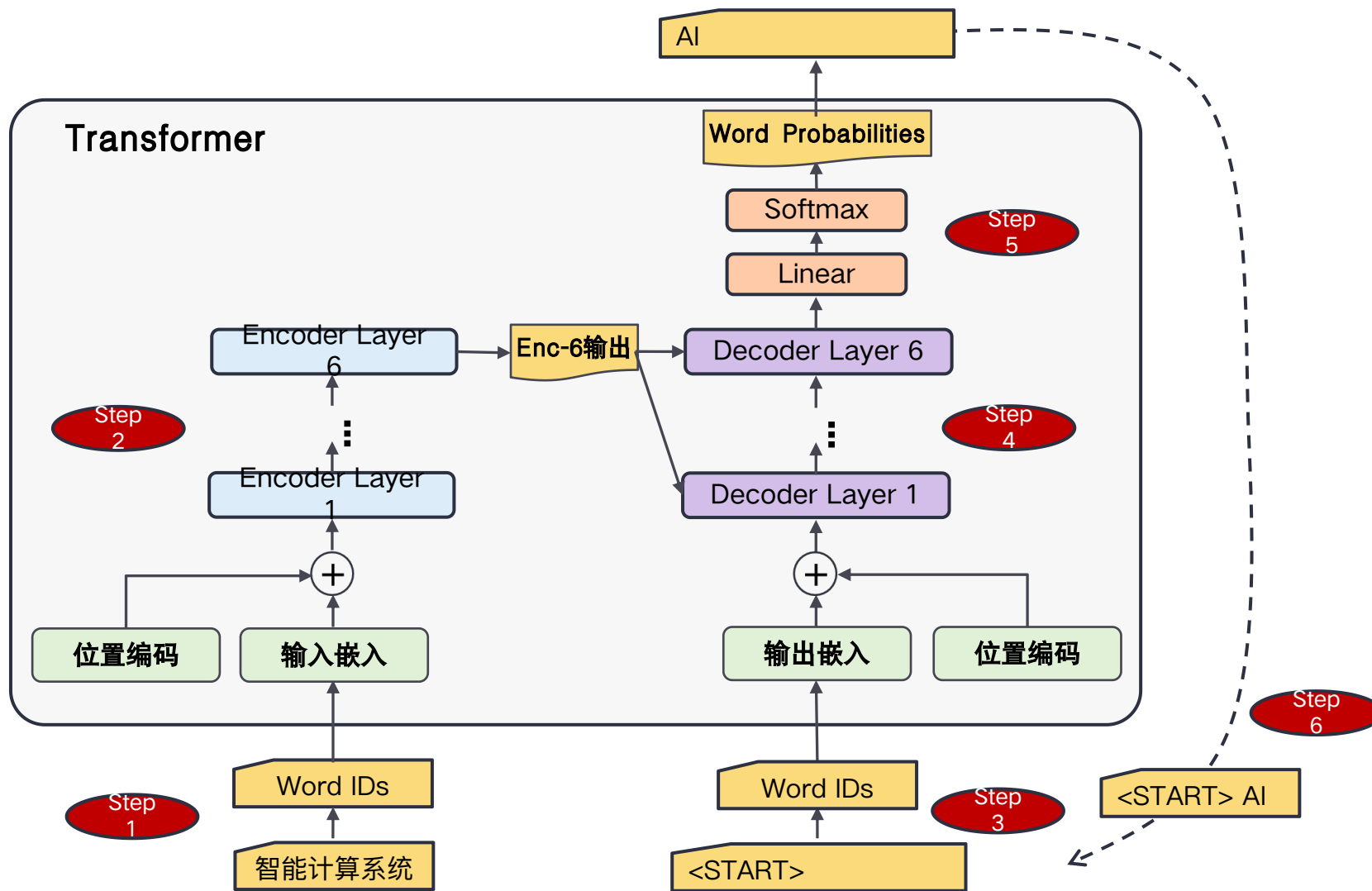
▶ Transformer的输出

- ▶ 先经过一个线性变换（全连接层），将输出映射为词汇表空间
- ▶ 再经过softmax，输出下一个词元的概率分布



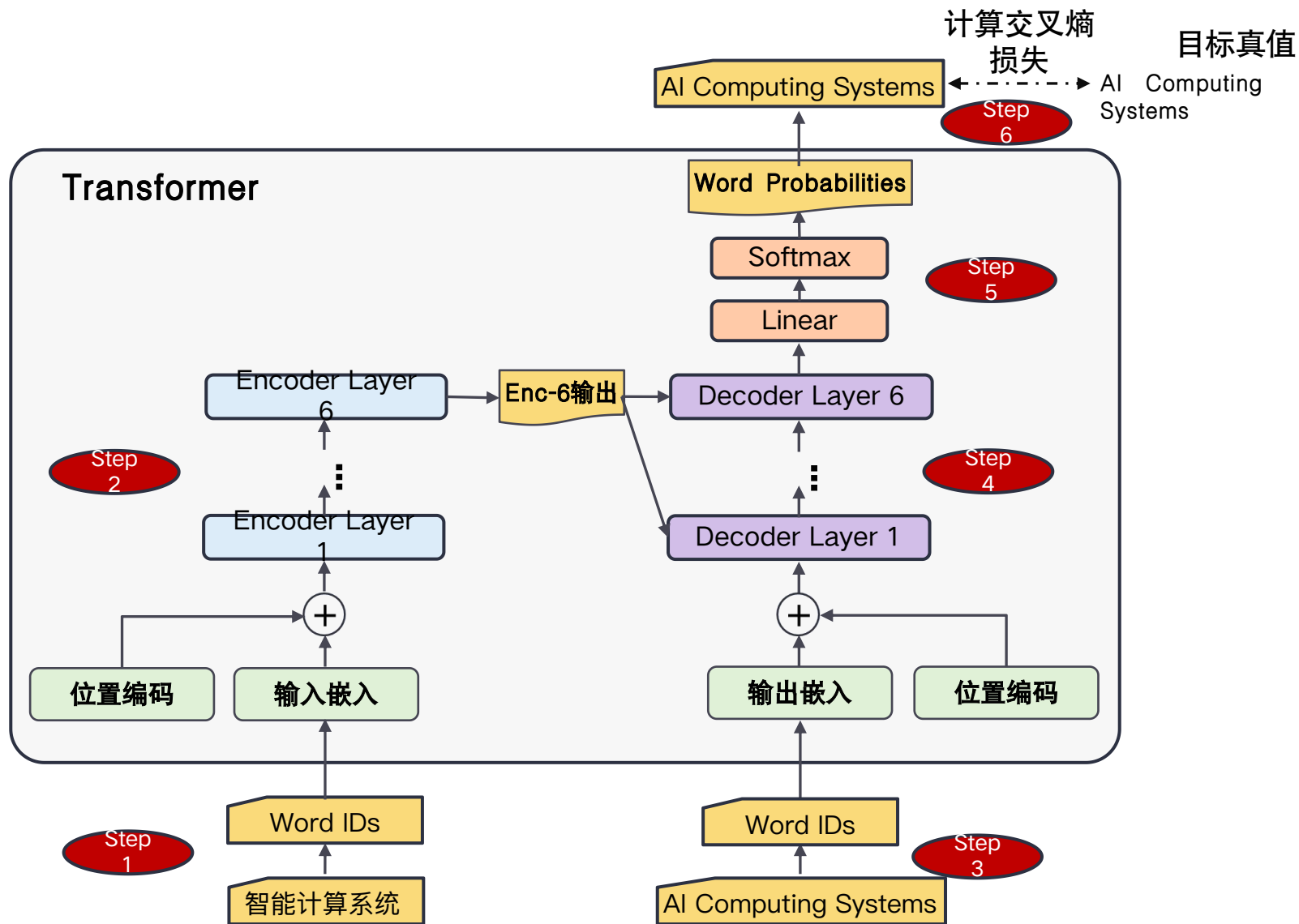
Transformer

推理



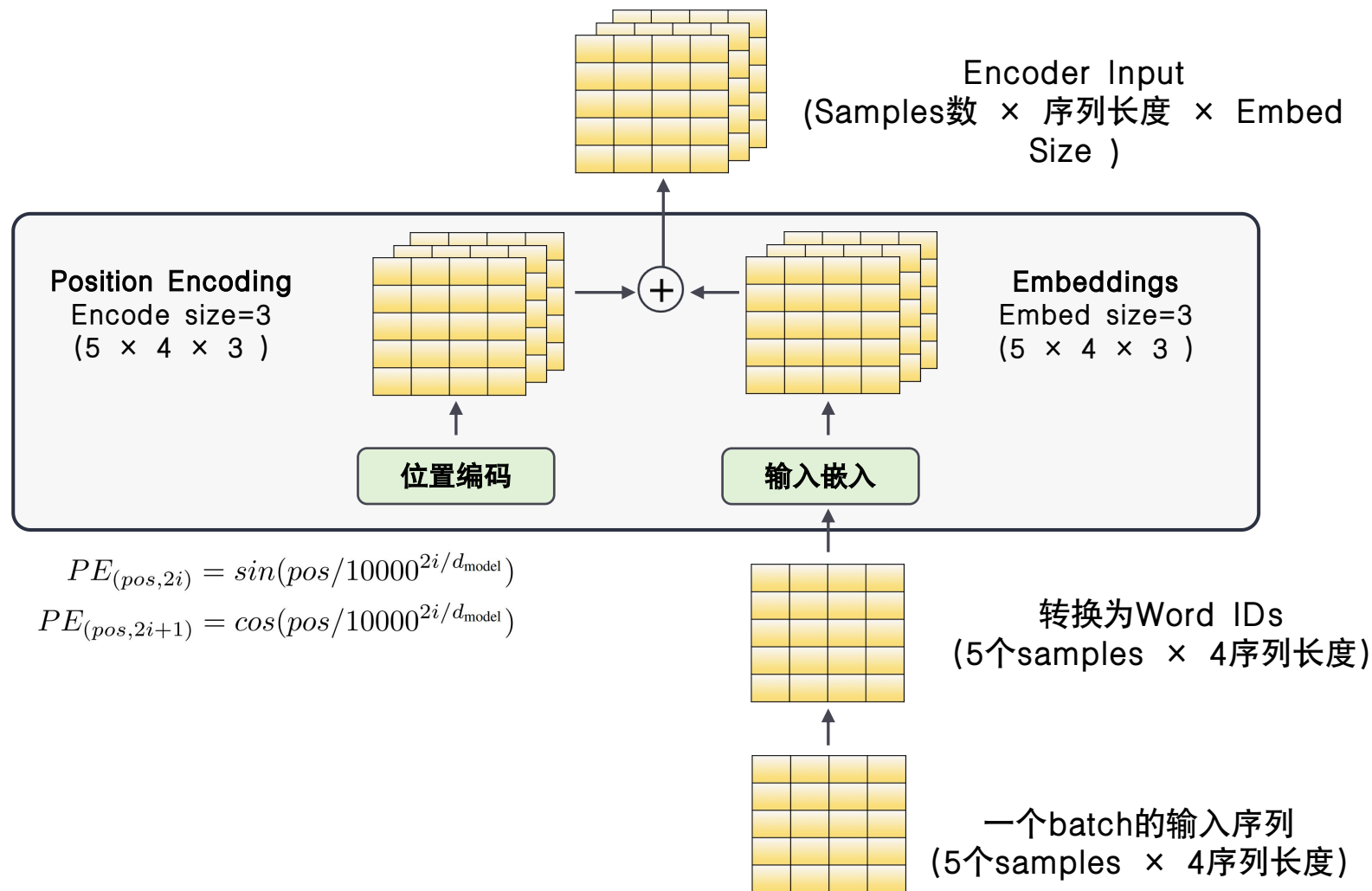
Transformer

► 训练



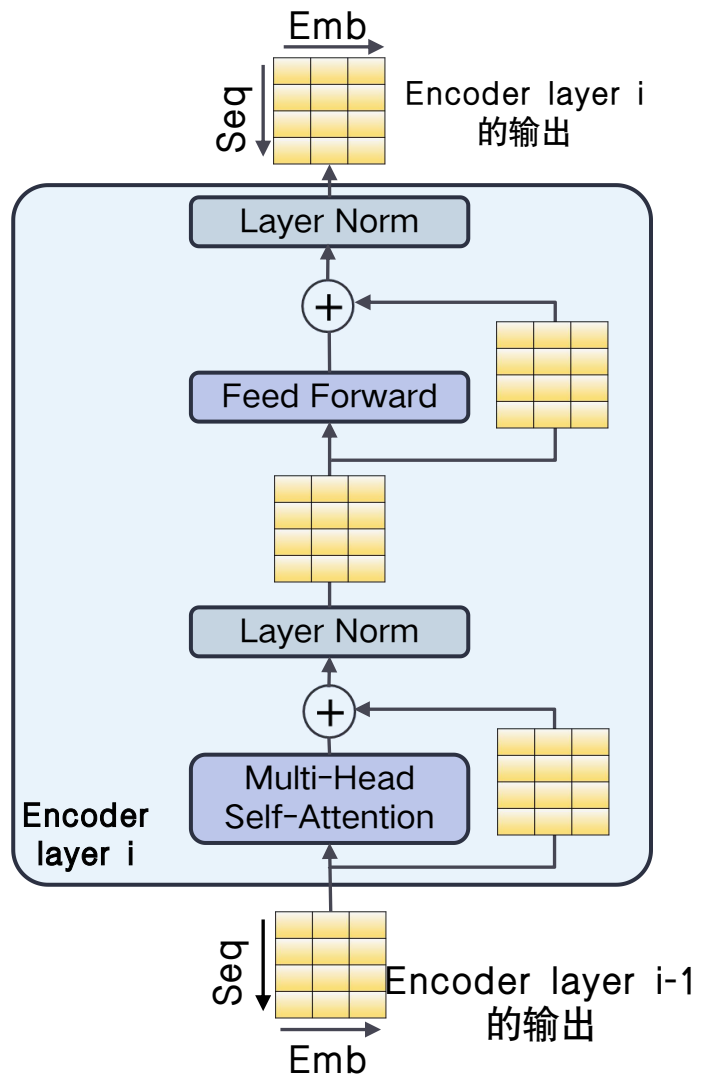
Transformer

▶ 位置编码及嵌入 (Embedding)

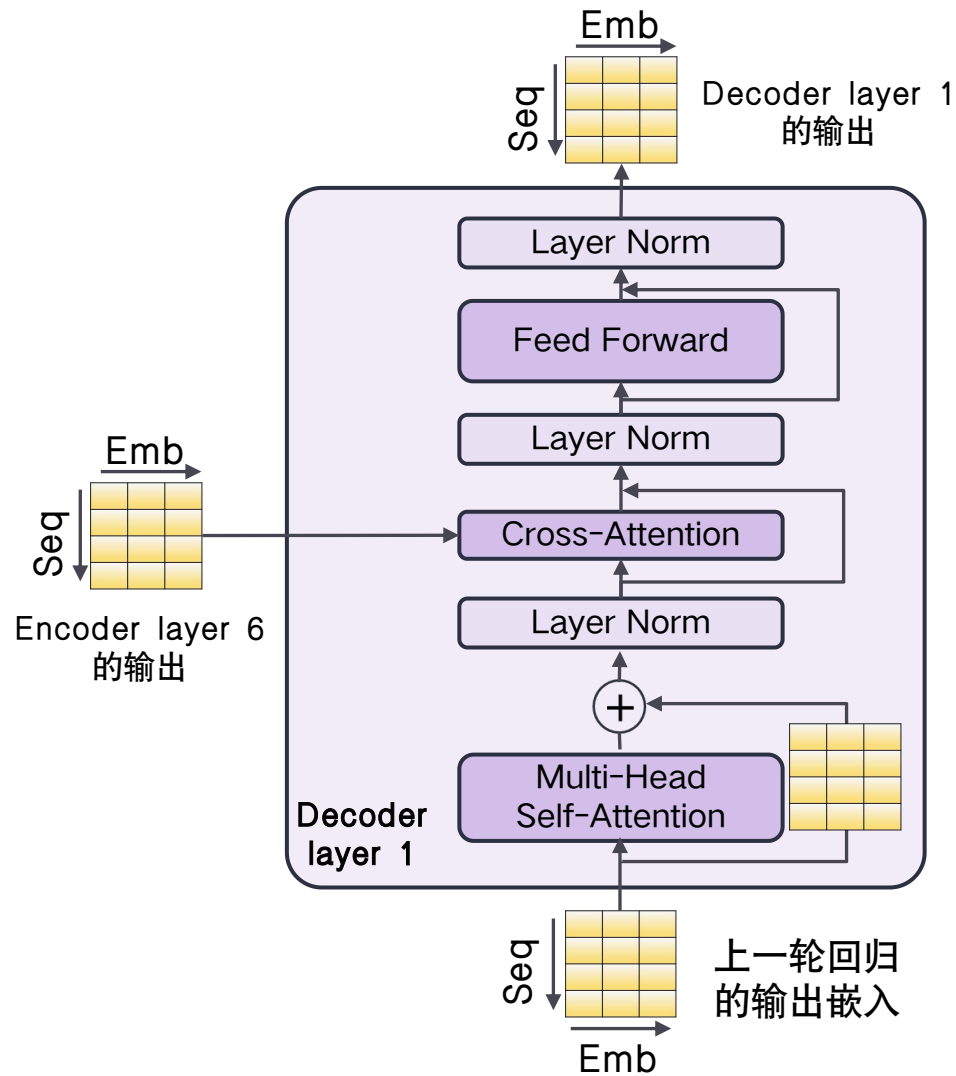


Transformer

► 编码器 (Encoder)

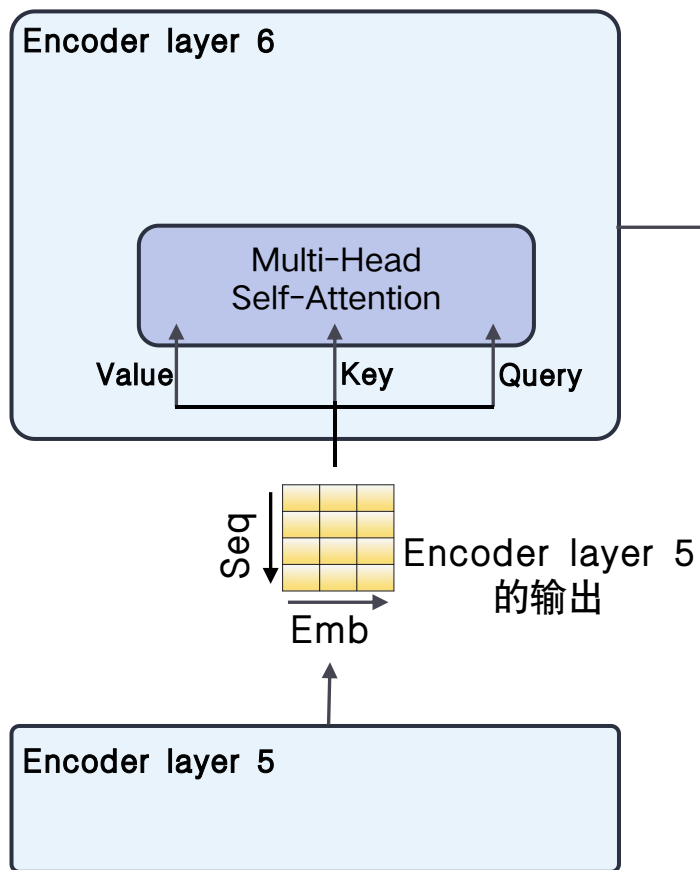


► 解码器 (Decoder)

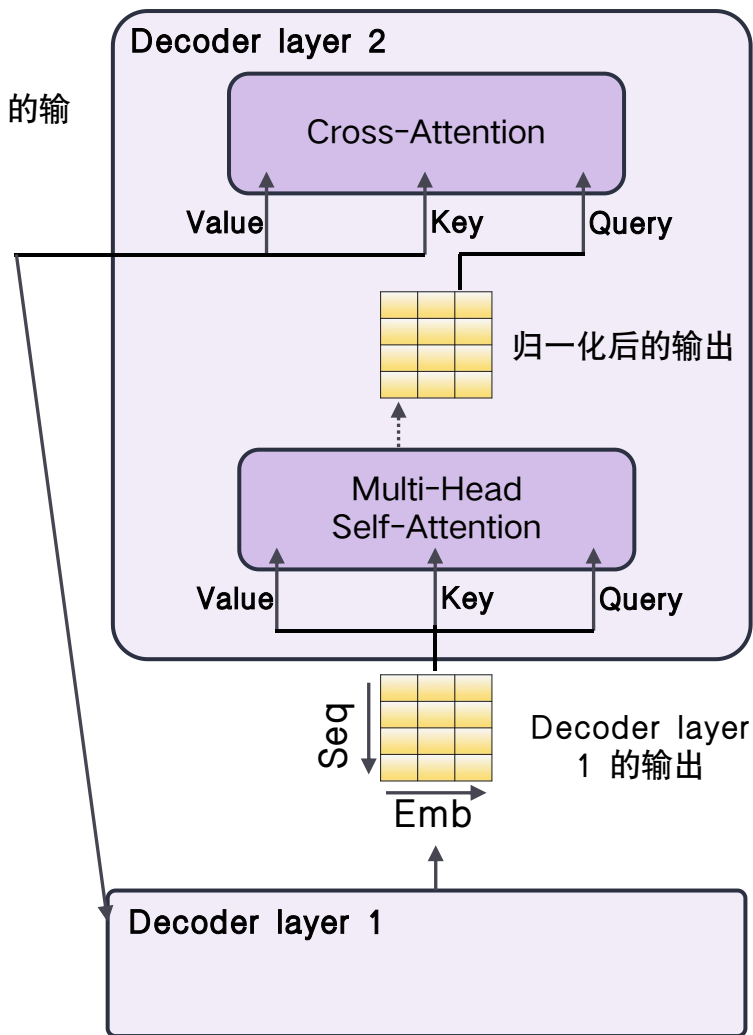


Transformer

► 编码器中注意力

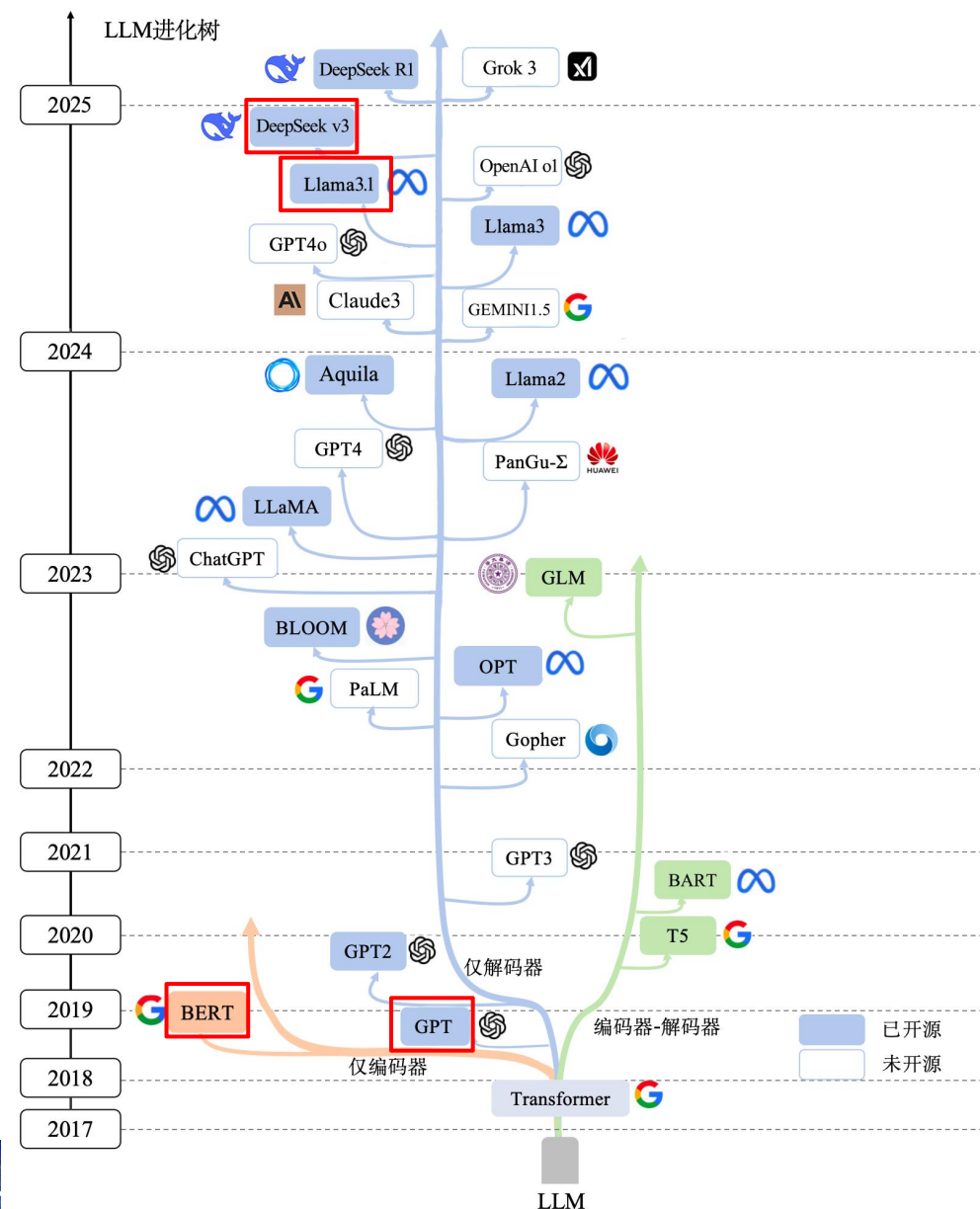


► 解码器中注意力



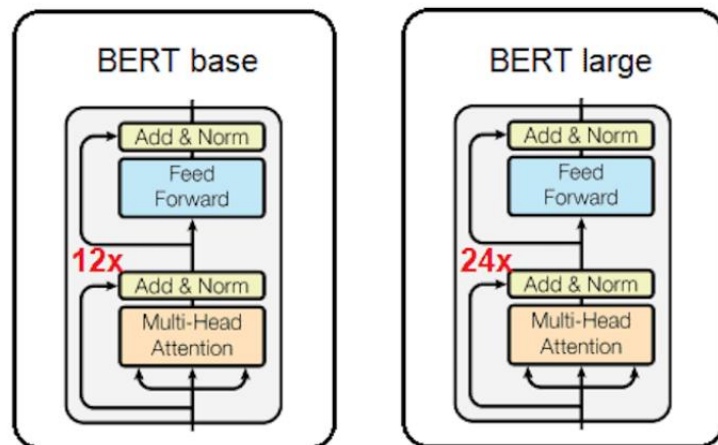
自然语言处理大模型

- ▶ 编码器-解码器 (Encoder-Decoder)
 - ▶ 常用于序列到序列Seq2Seq任务，如机器翻译、生成对话等
 - ▶ 代表：原始Transformer, Google T5, GLM等
- ▶ 仅编码器 (Encoder-Only)
 - ▶ 适用于不需要生成序列，只需要对输入进行编码和处理的单向任务场景，如文本分类、情感分析等
 - ▶ 代表：Google **BERT**
- ▶ 仅解码器 (Decoder-Only)
 - ▶ 通常用于序列生成任务，从输入的编码中生成相应的序列，关注于从已有的信息（开头）扩展出新的内容
 - ▶ 代表：OpenAI **GPT**系列, Meta **LLaMA**系列, **DeepSeek**系列



BERT (Encoder-only)

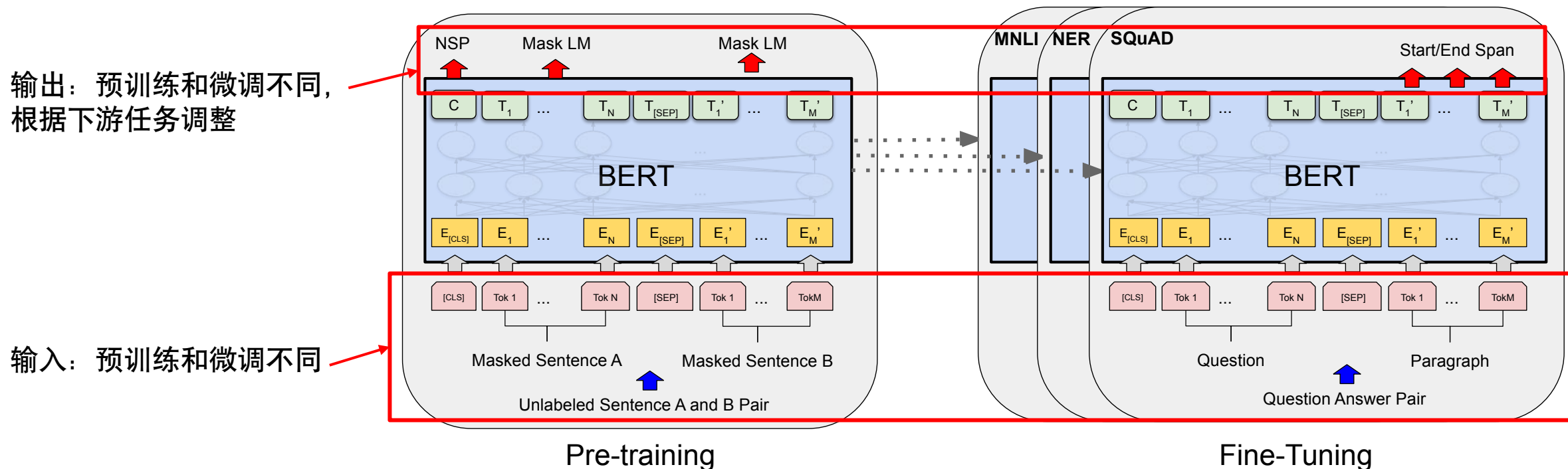
- ▶ BERT (Bidirectional Encoder Representation from Transformer)
 - ▶ 采用Transformer编码器部分，进行双向的Block连接
 - ▶ 编码器结构：每个编码器layer中包含一个多头自注意力（无mask）和FFN
 - ▶ 双向：每个位置可以看到序列前后的位置（无mask）



	层数	隐状态维度	FFN维度	head	dk=dv	参数量
base	12	768	3072	12	64	110M
large	24	1024	4096	16	64	340M

BERT (Encoder-only)

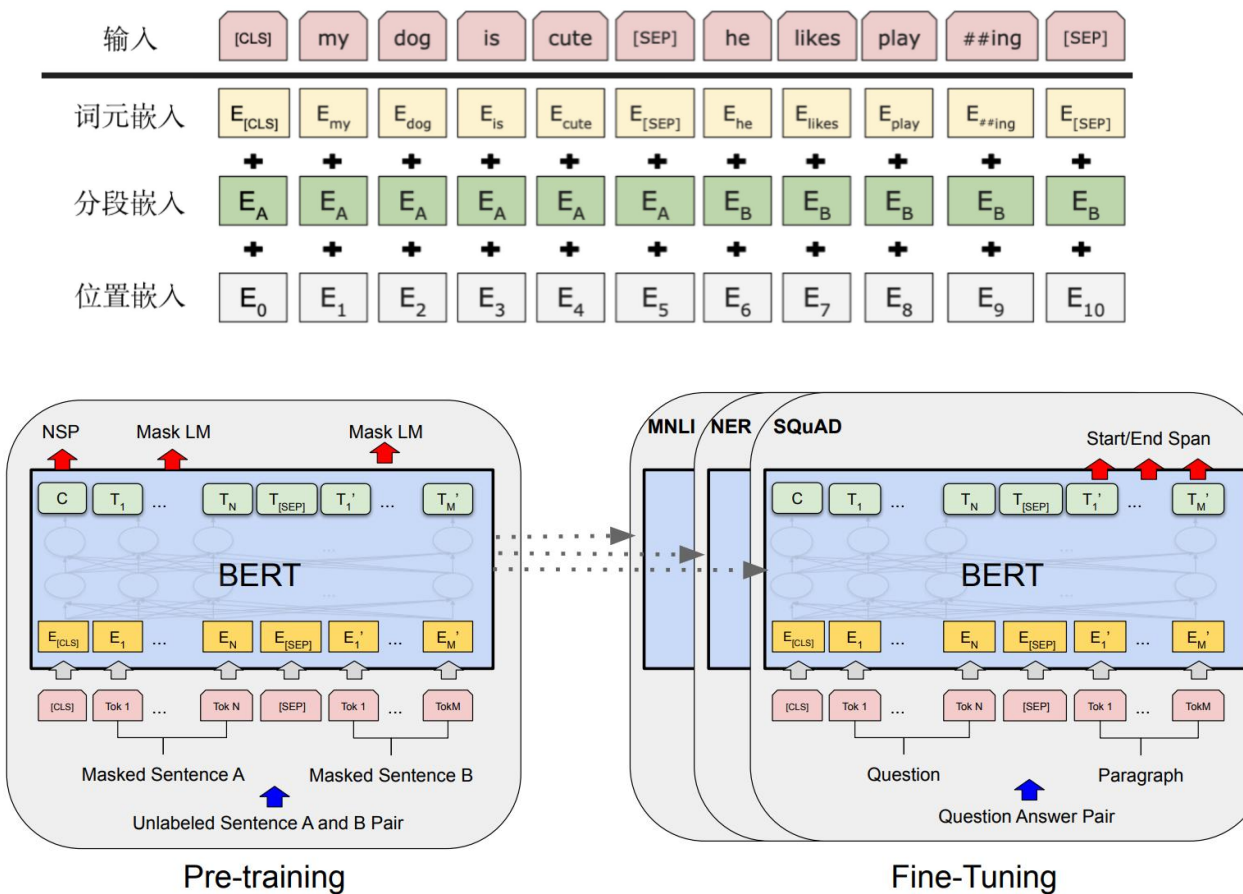
- ▶ 预训练：利用互联网上大量无标签的语料数据进行自监督训练，包括完形填空，以及判断两句话之间是否有上下文关系
- ▶ 微调：利用下游任务的有标签数据进行微调，可使模型快速应用于文本分类、序列标注、信息检索等多种下游任务



Devlin, Jacob and Chang, Ming-Wei and Lee, Kenton and Toutanova, Kristina. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv.1810.04805. 2018

BERT (Encoder-only)

► 输入嵌入



BERT (Encoder-only)

- ▶ BERT和GPT的模型参数量差不多，但是BERT预训练的数据集大约是GPT的4倍，BERT在NLP的很多下游任务上（主要是理解任务）都超越了GPT

	BERT-base	GPT
参数量	110M	117M
数据集大小	BooksCorpus (800M words) + English Wikipedia (2500M words)	BooksCorpus (800M words)

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Encoder-only 架构的优势与不足

▶ 优势

- ▶ 可以在大量的无标签文本语料库上进行无监督预训练
- ▶ 挖掘了预训练阶段的潜力，通过掩码语言模型（MLM）任务获取到上下文相关的双向特征表示
- ▶ 可以通过微调，将预训练好的模型迅速迁移到各种下游任务
- ▶ 适合各种**自然语言理解（NLU）任务**

▶ 不足

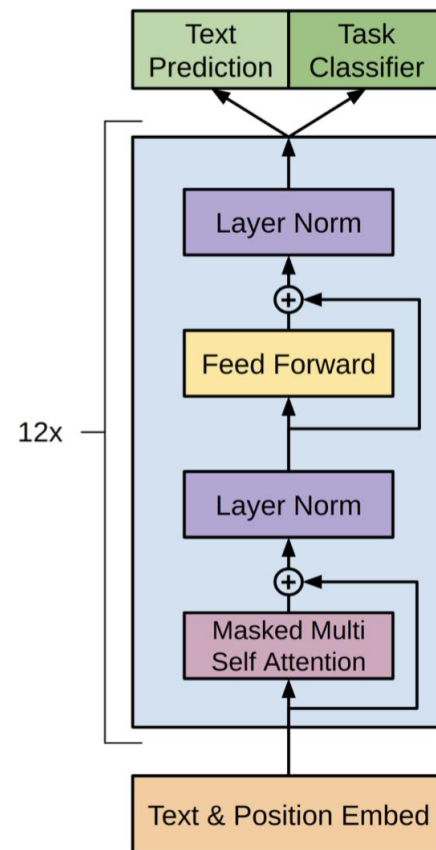
- ▶ 由于只使用了Transformer的编码器部分，没有进行自回归预训练，且架构上缺少解码器结构，因此不适合直接处理**自然语言生成（NLG）任务**，如生成对话、文章摘要、机器翻译
- ▶ 只适合处理句子和段落级别的任务，不适合处理文档级别的任务

Decoder-only 架构

- ▶ Decoder-only 架构
 - ▶ GPT (Generative Pre-trained Transformer)系列: 1-2-3-4-o1-o3
 - ▶ LLAMA (Large Language Model Meta AI)
 - ▶ DeepSeek
- ▶ 为什么 Decoder-only 架构成为主流

GPT (Decoder-only)

- ▶ GPT (Generative Pre-trained Transformer) 提出了NLP领域的预训练-微调统一框架
- ▶ 采用Transformer解码器结构：带mask的自注意力+FFN
 - ▶ 12层，12注意力头，768 维，dff=3072，
 - ▶ Context size=512
- ▶ 预训练：在大规模无标记的语料库上进行预训练，即给定前N-1个词，预测第N个词（无监督学习）
- ▶ 微调：对每个特定下游任务利用有标记数据进行微调
- ▶ 可以广泛应用于各种自然语言处理任务，如文本自动补全、生成对话、文章摘要等



GPT的预训练

▶ 预训练（无监督）

- ▶ 训练方式：自回归，给定前N-1个词，预测第N个词，适用于大量未标注数据的预训练（无监督学习，第N个词成为学习的标签）

- ▶ 目标函数：语言模型

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

P是GPT

用i-k到i-1序列预测第i个词
使用mask保证仅利用i-k到i-1计算

经过词嵌入矩阵 W_e ，加上位置编码 W_p

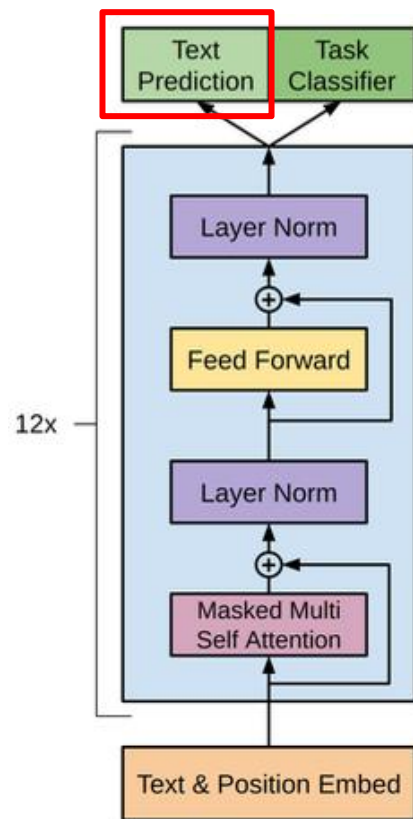
$$h_0 = U W_e + W_p$$

经过n个解码层

$$h_l = \text{transformer_block}(h_{l-1}) \forall i \in [1, n]$$

经过逆词嵌入矩阵和softmax获得输出词的概率

$$P(u) = \text{softmax}(h_n W_e^T)$$



GPT的微调

► 微调（有监督）

- 训练方式：有监督学习，使用下游任务的数据集和标签
- 参数更新：新添加的层从头训练，其他参数微调
- 目标函数：下游任务的目标函数+预训练的语言模型

$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y).$$

其中， x^1 到 x^m 是下游任务序列，标签是 y

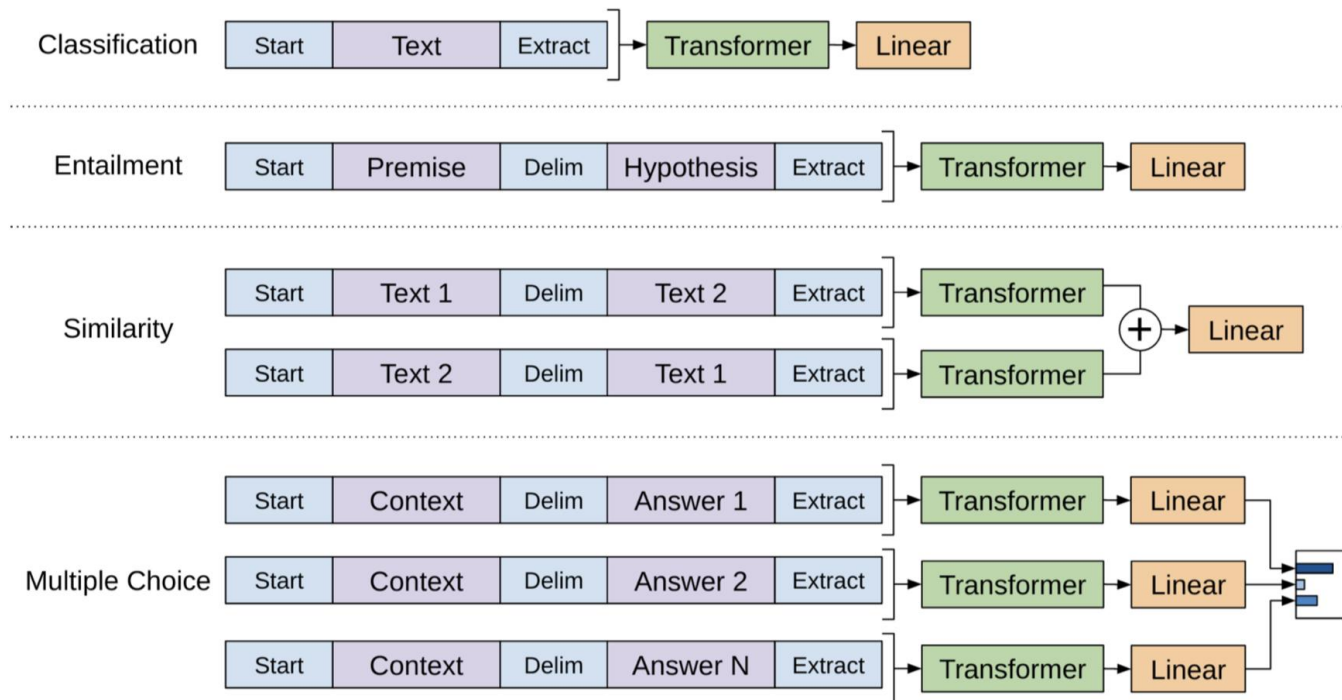
W_y 是微调时添加的线性层，需要从头训练

下游任务的目标函数

$$L_2(C) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m).$$

最终的目标函数：预训练与微调的目标函数加权求和

$$L_3(C) = L_2(C) + \lambda * L_1(C)$$



Radford A, Narasimhan K, Salimans T, Sutskever I. Improving language understanding by generative pre-training. 2018.

GPT-2

- ▶ 核心思想：无需特定任务的监督微调，在预训练阶段使用语言模型建模多种下游任务，实现通用的NLP模型

原始语言模型 $p(s_{n-k}, \dots, s_n | s_1, \dots, s_{n-k-1})$.

语言模型建模下游任务 $p(\text{output} | \text{input}, \text{task})$

任务输出 任务输入 任务描述

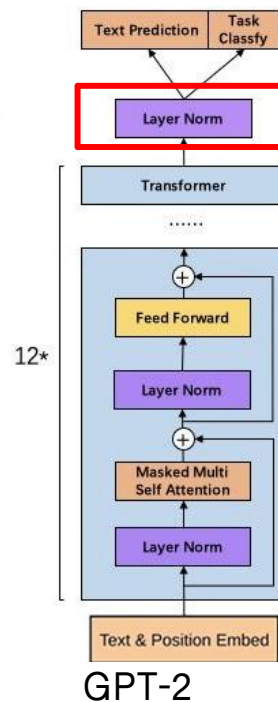
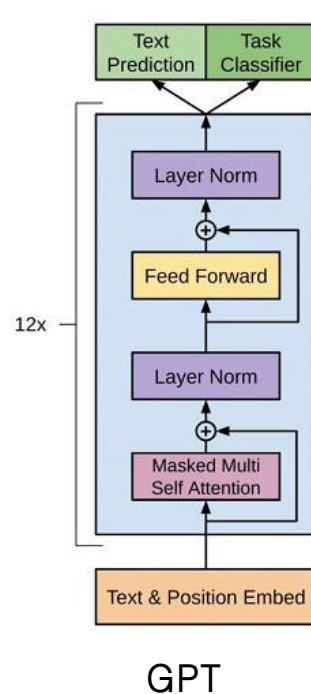
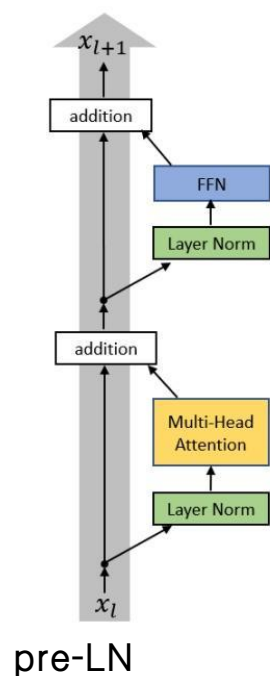
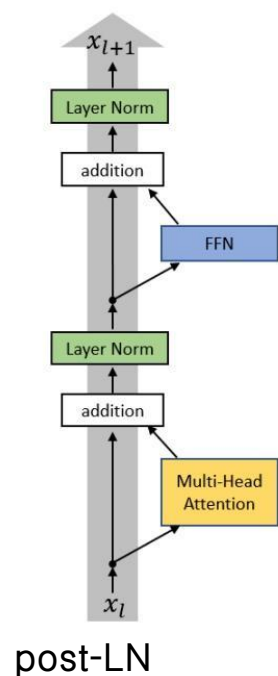
- ▶ 训练样本

- ▶ 机器翻译：(translate to French, English text, French text)
- ▶ 阅读理解：(answer the question, document, question, answer)

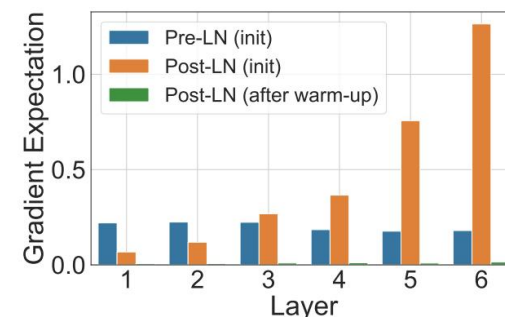
GPT-2

结构调整：采用Pre-LN

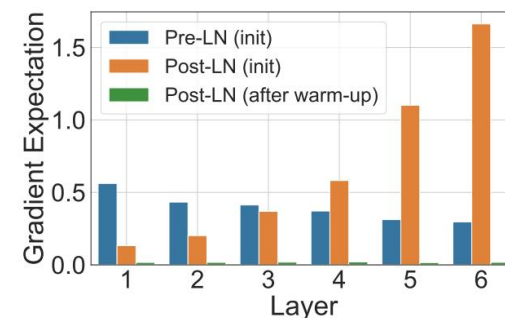
- Post-LN: layer norm放在子层后面，例如Transformer和GPT
- pre-LN : layer norm放在子层前面，使各子层的梯度范数保持不变，有利于稳定优化



最后有一个额外的layer norm



(a) W^1 in the FFN sub-layers



(b) W^2 in the FFN sub-layers

GPT-2

- 更大的参数量，更多的层，更多的训练数据
- 通过海量数据和大量参数训练出来的词向量模型，可以迁移到其它任务中而不需要额外的训练

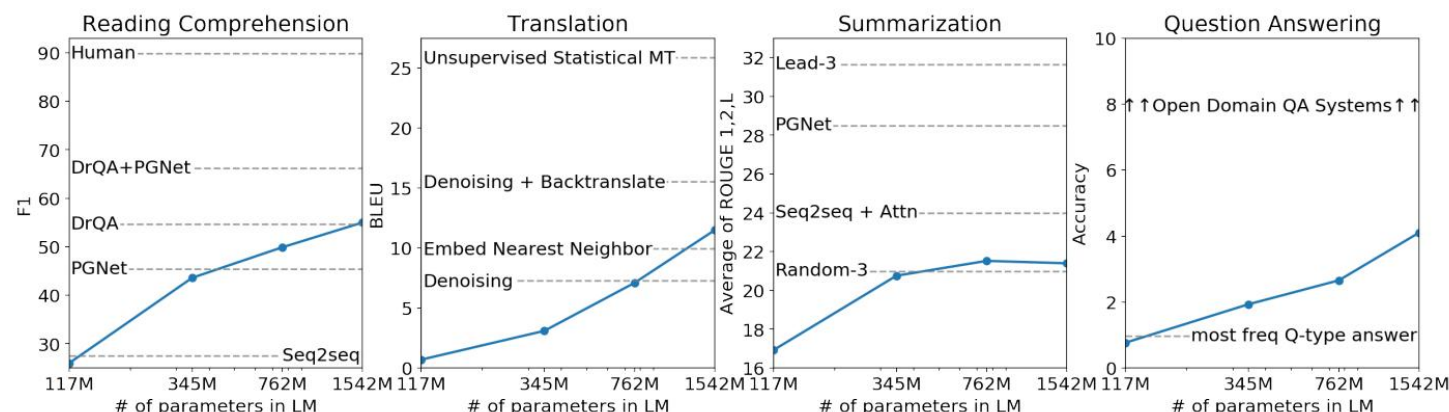
数据集:

GPT: 5GB

GPT-2: 40GB

Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

← 对标GPT
← 对标BERT-large
← 规模提高一个量级



	LAMBADA (PPL)	LAMBADA (ACC)	CBT-CN (ACC)	CBT-NE (ACC)	WikiText2 (PPL)	PTB (PPL)	enwik8 (BPB)	text8 (BPC)	WikiText103 (PPL)	1BW (PPL)
SOTA	99.8	59.23	85.7	82.3	39.14	46.54	0.99	1.08	18.3	21.8
117M	35.13	45.99	87.65	83.4	29.41	65.85	1.16	1.17	37.50	75.20
345M	15.60	55.48	92.35	87.1	22.76	47.33	1.01	1.06	26.37	55.72
762M	10.87	60.12	93.45	88.0	19.93	40.31	0.97	1.02	22.05	44.575
1542M	8.63	63.24	93.30	89.05	18.34	35.76	0.93	0.98	17.48	42.16

GPT-3

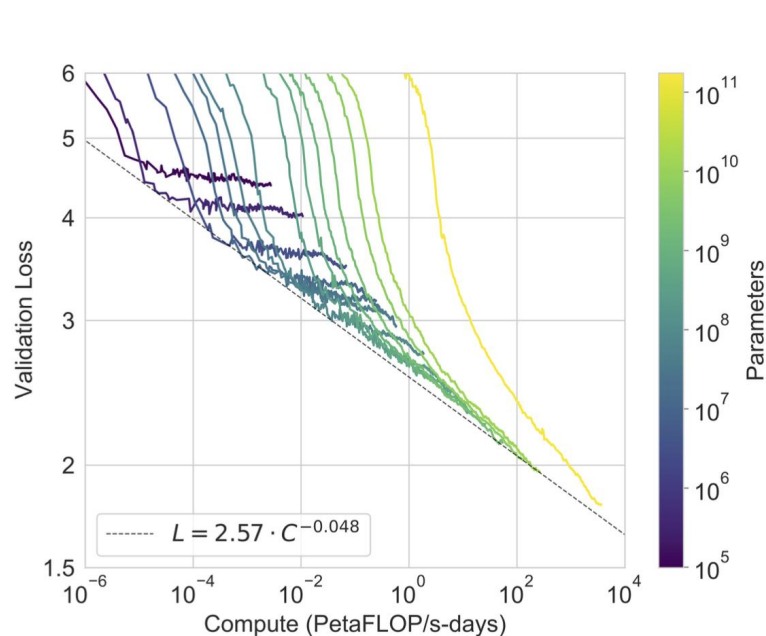
- ▶ GPT-3沿用了GPT-2的结构，在规模上做了很大提升
 - ▶ 更大的参数量，更多的层和更多的训练数据

	GPT	GPT-2	GPT-3
贡献	提出预训练-微调的统一框架	无需微调，预训练用语言模型建模多种任务	无需微调，利用少量示例提升泛化能力
框架	预训练-微调	仅预训练，无需微调	仅预训练，无需微调
结构	12层，12 heads， d=768	48层，d=1600	96层，96 heads，d=12288
参数量	117M	1.5B	175B
数据量	5GB	40GB	45TB
发布时间	2018年6月	2019年2月	2020年5月

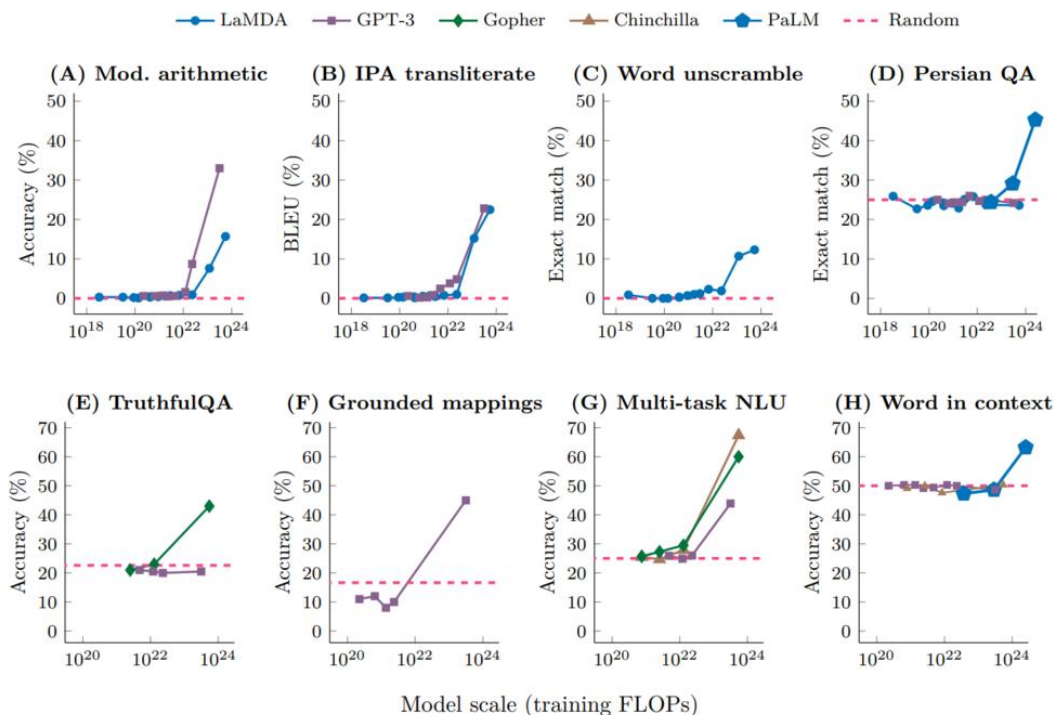
Brown, Tom B. "Language models are few-shot learners." arXiv preprint arXiv:2005.14165 (2020).

GPT-3

- ▶ GPT-3沿用GPT-2的结构，在规模上做了很大提升
- ▶ 更大的参数量，更多的层和更多的训练数据



尺度定律(Scaling law): 当增加模型计算量、训练数据量、模型参数规模，大模型的Loss都会单调降低，模型效果越来越好



涌现能力(Emergent ability): 当模型规模大到一定程度(60B)，模型能力出现飞跃

Brown, Tom B. "Language models are few-shot learners." arXiv preprint arXiv:2005.14165 (2020).

GPT-3

► 无需微调，使用任务提示(prompt)和少量示例处理下游任务 (few-shot)

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

zero-shot: 只有任务描述和提示，没有示例

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

one-shot: 有任务描述提示，和一个示例

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

few-shot: 有任务描述提示，和若干个示例

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



传统微调：样本用于参数更新

InstructGPT

▶ InstructGPT使用来自人类反馈的强化学习 (RLHF) 对预训练大模型进行微调

- ▶ Step1: 人类标注产生数据, 监督方式微调GPT-3
- ▶ Step2: 利用人类标注数据训练一个奖励模型
- ▶ Step3: 利用奖励模型提供reward, 利用PPO微调step1的GPT-3

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A B
Explain gravity... Explain war...
C D
Moon is natural satellite of... People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM
D > C > A = B

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO

Once upon a time...

The reward model calculates a reward for the output.

RM

The reward is used to update the policy using PPO.

r_k

GPT系列后续发展

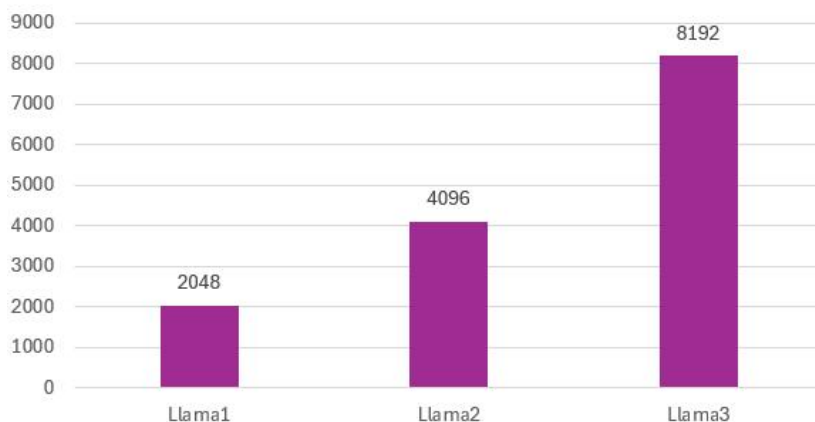
- ▶ 2022.11, ChatGPT: GPT3.5+RLHF, 高质量数据
- ▶ 2023.3, GPT-4: AGI的火花, 比ChatGPT效果更好
- ▶ 2024.5, GPT-4o: 全能, 多模态的输入输出 (文本、音频、图像)
- ▶ 2024.9, OpenAI o1: 通用推理能力, 应用于数学、科学、代码领域
- ▶ 2024.12, OpenAI o3: 通用推理能力, 应用于数学、科学、代码等领域

Llama系列

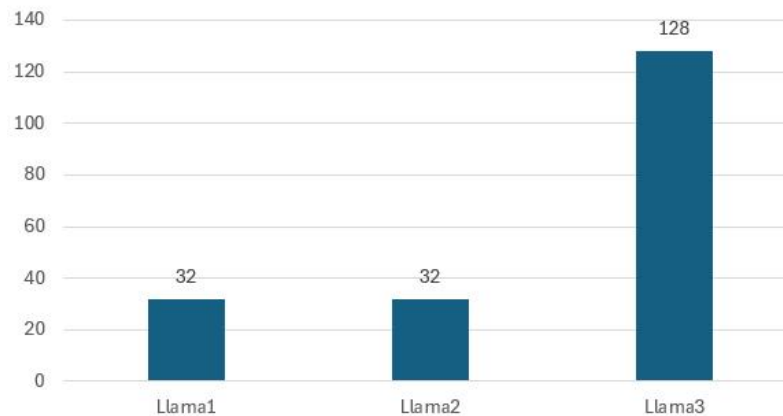
► Meta AI 发布的基础语言模型，开源

	LLaMa	Llama 2	Llama 3
参数量	7B、13B、30B和65B	7B、13B、34B和70B	8B、70B、400B
数据量	1.4T	2T	15T
上下文长度	2048	4096	8192
词表大小	32k	32k	128k
发布时间	2023年2月	2023年7月	2024年4月

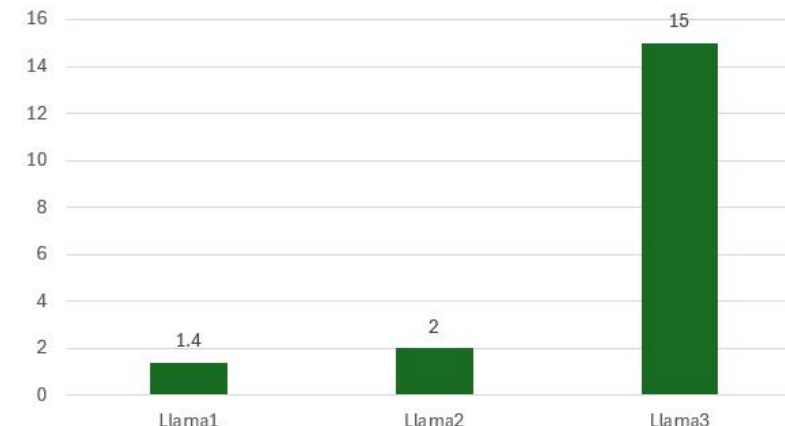
上下文长度



词汇表大小 (K)

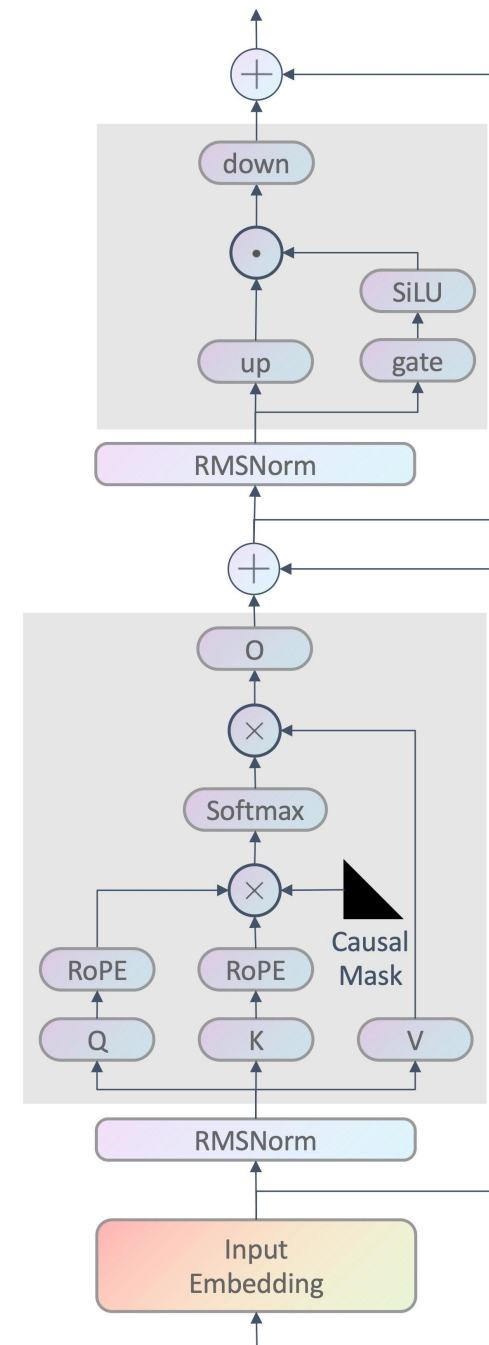


训练数据 (万亿tokens)



Llama

- ▶ Llama模型与GPT类似，也采用decoder-only架构
- ▶ 在原始Transformer解码器的基础上，Llama进行了如下改动：
 - ▶ 采用前置的 RMSNorm 作为层归一化方法，增强训练稳定性
 - ▶ 采用 SwiGLU激活函数，提高模型性能
 - ▶ 采用旋转位置编码RoPE，建模长序列数据
 - ▶ 部分模型采用分组查询注意力机制（Grouped-Query Attention, GQA），平衡效率和性能



Touvron, Hugo, et al. "Llama: Open and efficient foundation language models." *arXiv preprint*

Touvron, Hugo, et al. "Llama 2: Open foundation and fine-tuned chat models." *arXiv preprint*

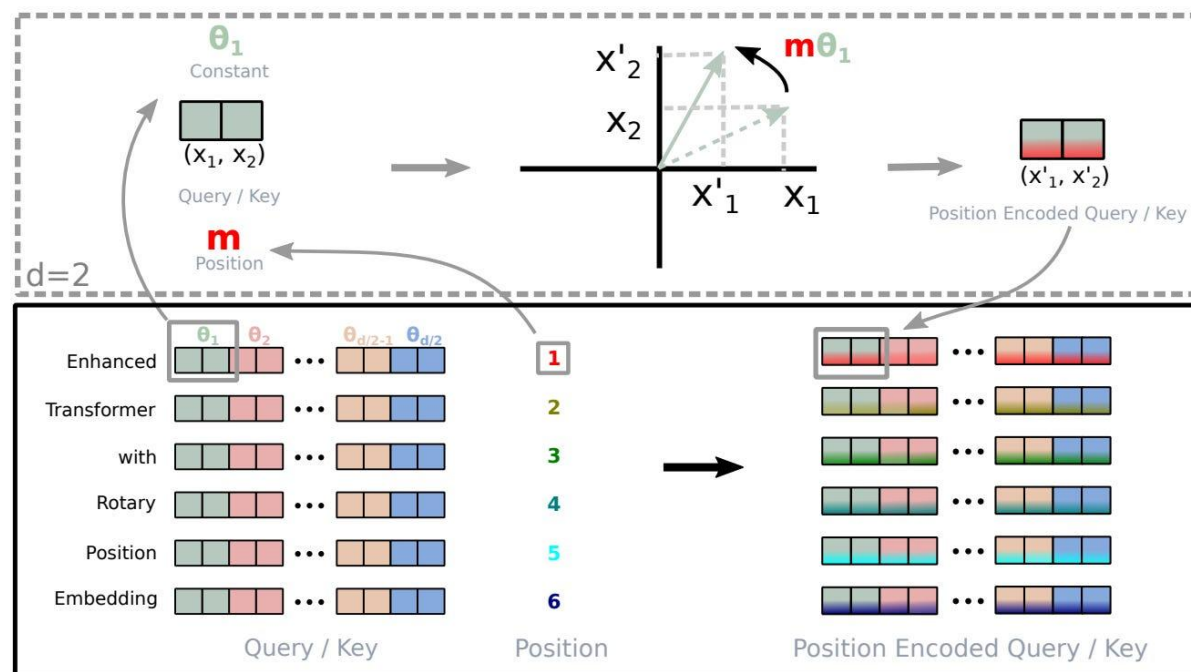
Dubey, Abhimanyu, et al. "The llama 3 herd of models." *arXiv preprint*

arXiv:2407.21785 (2024).

RoPE

► RoPE (Rotary Position Encoding, 旋转位置编码)

- 使用旋转矩阵编码绝对位置
- 在自注意力操作中加入相对位置信息，在每个transformer层中注入位置信息
- 优势：支持长序列，并且随着相对距离增加，词元间依赖下降



$$\text{Attention score}(\mathbf{q}_j, \mathbf{k}_i) = g(\mathbf{q}_j, \mathbf{k}_i, i - j)$$

为了能够表示相对位置，qk的内积表示为qk和其相对位置i-j的函数

Su, L

Figure 1: Implementation of Rotary Position Embedding(RoPE).

bedding." Neurocomputing 568 (2024).

RoPE

► RoPE (rotary position encoding, 旋转位置编码)

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

$$\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\}.$$

任意偶数维的RoPE可以表示为二维形式的拼接

Su, Jianlin, et al. "Roformer: Enhanced transformer with rotary position embedding." Neurocomputing 568 (2024).

大模型中的激活函数

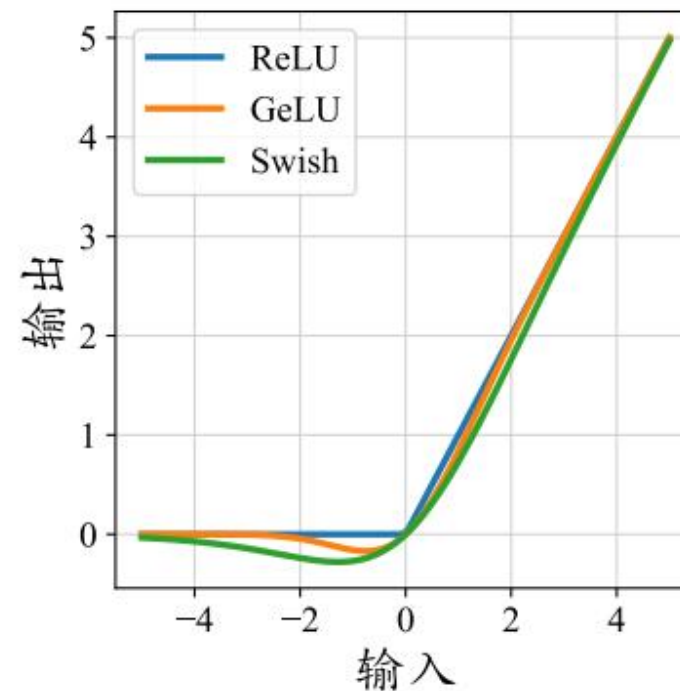
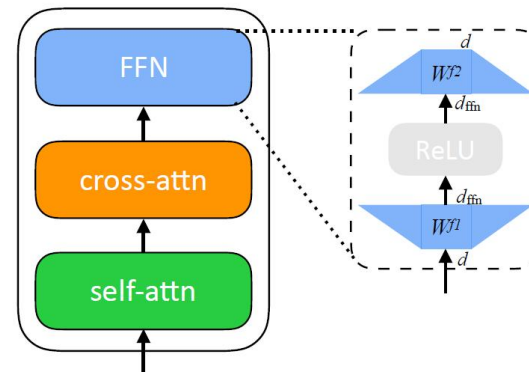
► Swish激活函数

- 更加平滑，可以更好的优化和更快的收敛，减少过拟合提高泛化性

$$\text{Switch}(x) = x \cdot \text{sigmoid}(x)$$

► SwiGLU激活函数 (Llama3, DeepSeek等)

$$\text{SwiGLU}(x, W, V, b, c) = \text{SiLU}(xW + b) \cdot (xV + c)$$



大模型中的layer norm变体

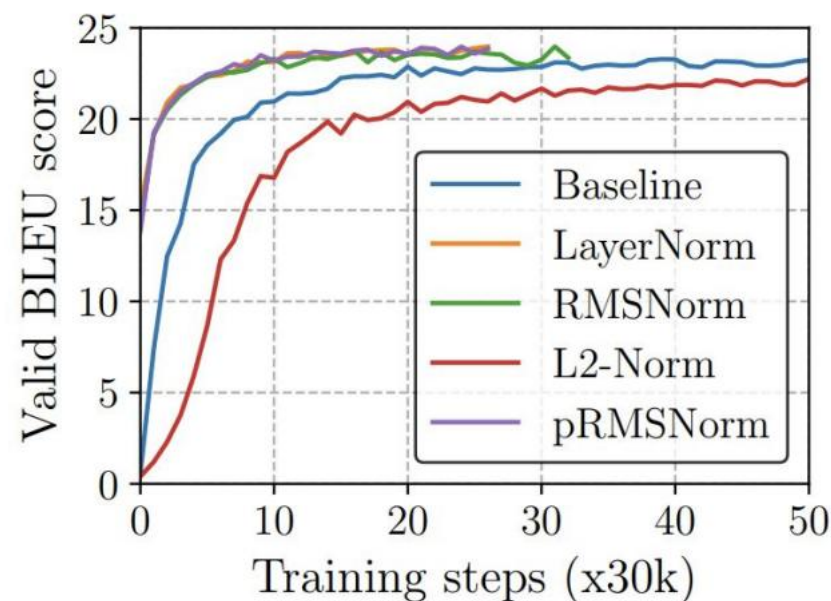
- ▶ RMSNorm: 均方根norm
- ▶ layer norm中包含平移和缩放不变性, RMSnorm去除平移, 只保留缩放
- ▶ 效果相当, 计算更加简单, 更加稳定的

Layer Norm: 平移和缩放

$$\text{LN}(x_i) = \frac{(x_i - \mu)}{\sigma} g_i; \quad \mu = \frac{1}{d} \sum_i x_i, \sigma = \sqrt{\frac{1}{d} \sum_i (x_i - \mu)^2}$$

RMSNorm: 去除平移, 只有缩放

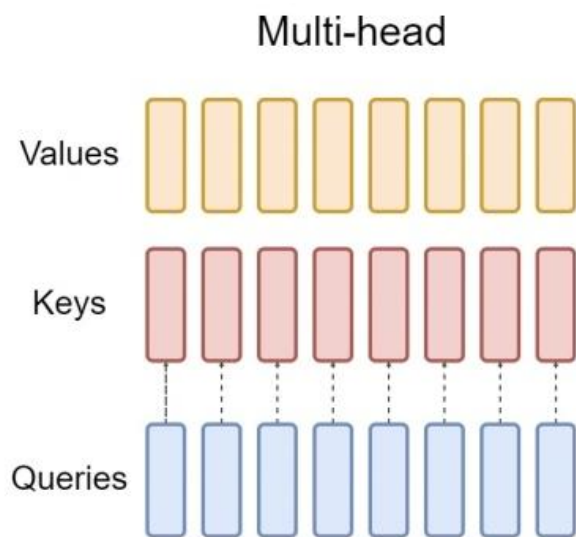
$$\text{LN}(x_i) = \frac{x_i}{\text{RMS}(\mathbf{x})} g_i; \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_i (x_i)^2}$$



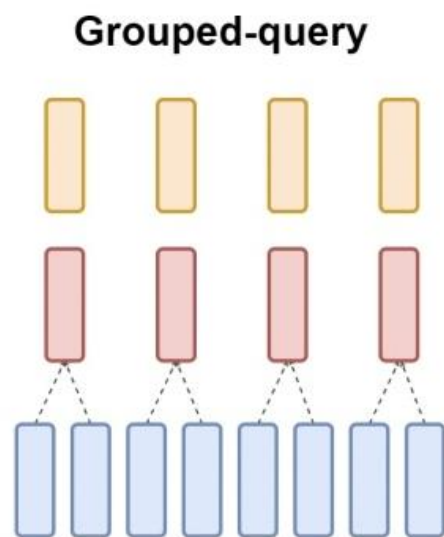
Better Convergence Rate on Machine Translation and Many NLP Tasks [7]

大模型中的attention变体

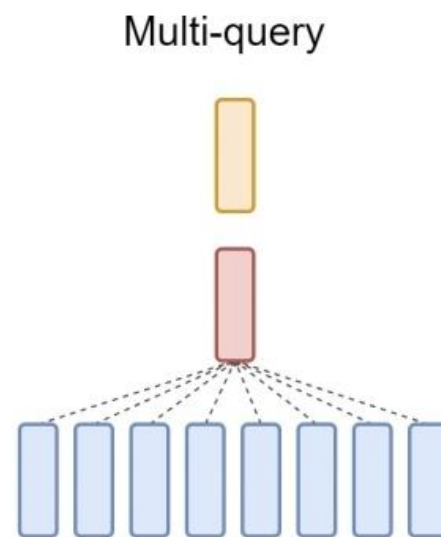
- ▶ GQA: 通过共享KV, 减少计算量, 提升计算速度



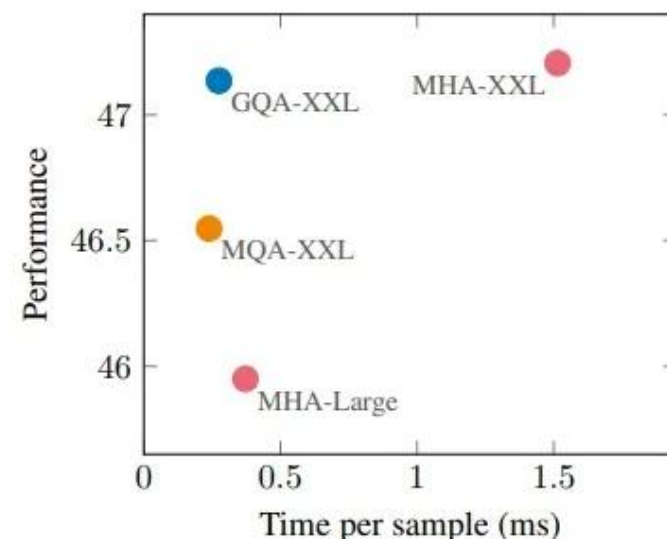
MHA: 每个head有自己单独的KV



GQA: 将Q分成N个组, 每个组共享KV



MQA: 所有Q共享KV

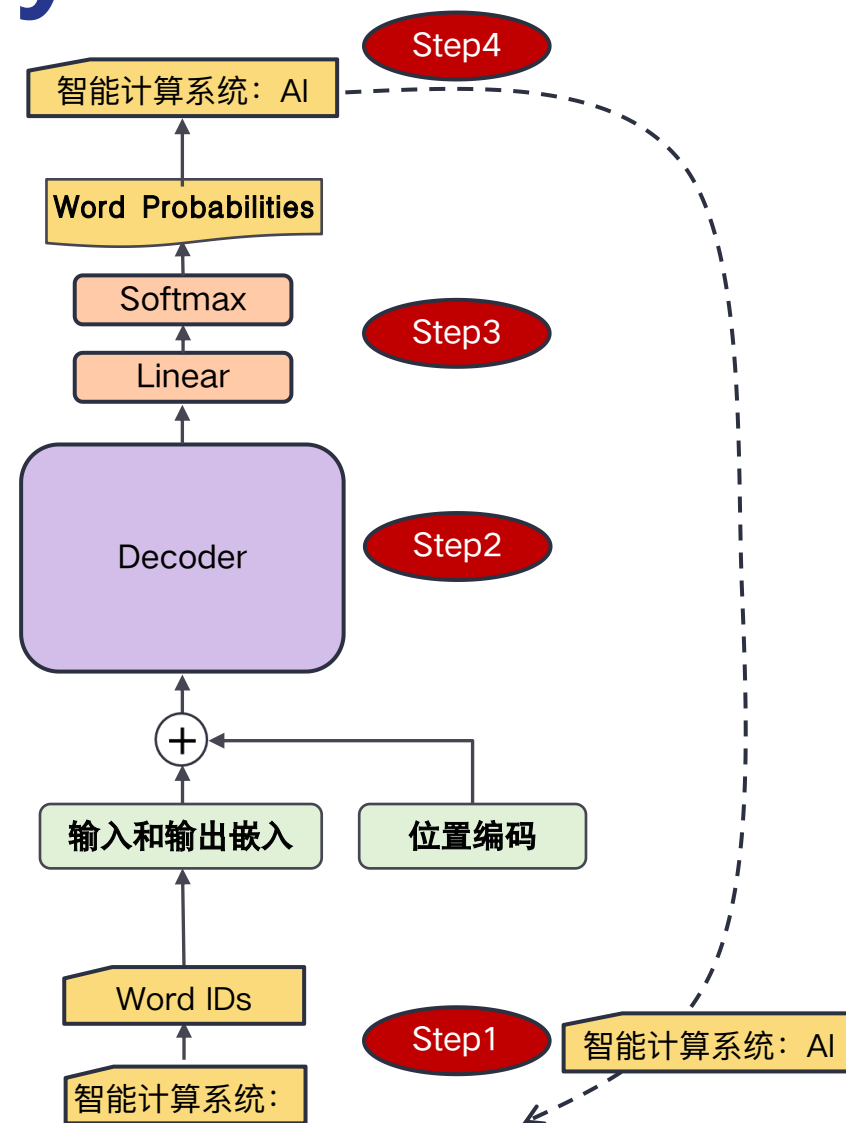


GQA和MHA的效果相当, 取得了performance和速度的trade-off

Decoder-only

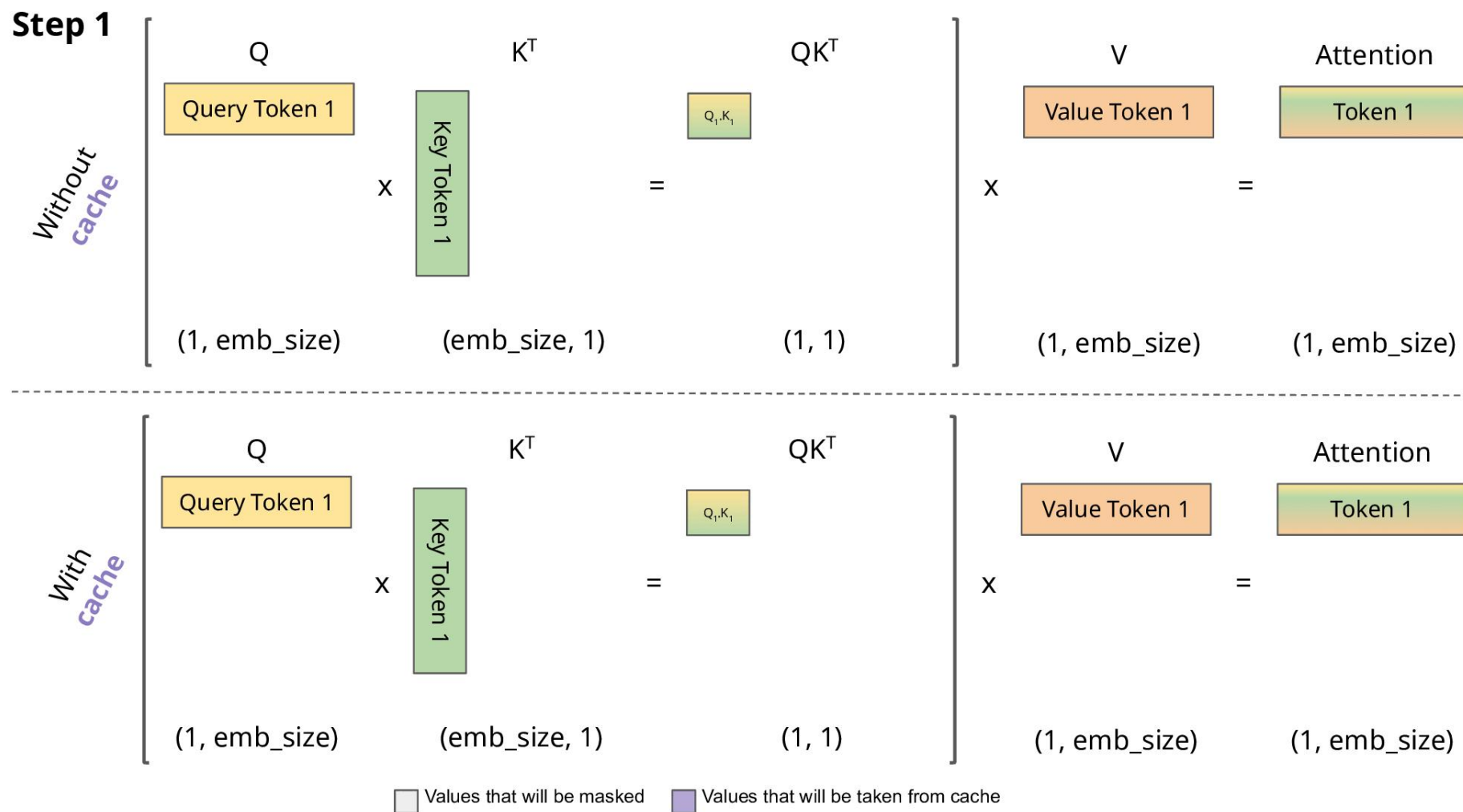
推理

- ▶ Prefill阶段：输入提示词，生成第一个词元。
此过程分析全部的提示词，用时较长，同时
将注意力模块中的K、V缓存下来，方便后续计
算调用（KV-Cache）
- ▶ Decode阶段：自回归地进行生成的过程，直
到生成终止符



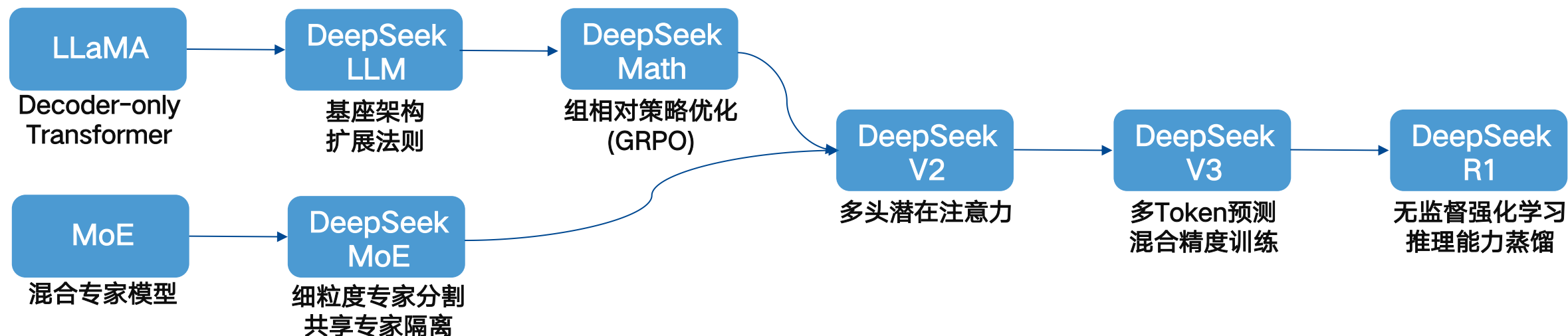
KV cache

- 通过缓存之前的K和V，只需要计算新token的attention



一脉相承的深度求索之路

——DeepSeek系列工作发展历程

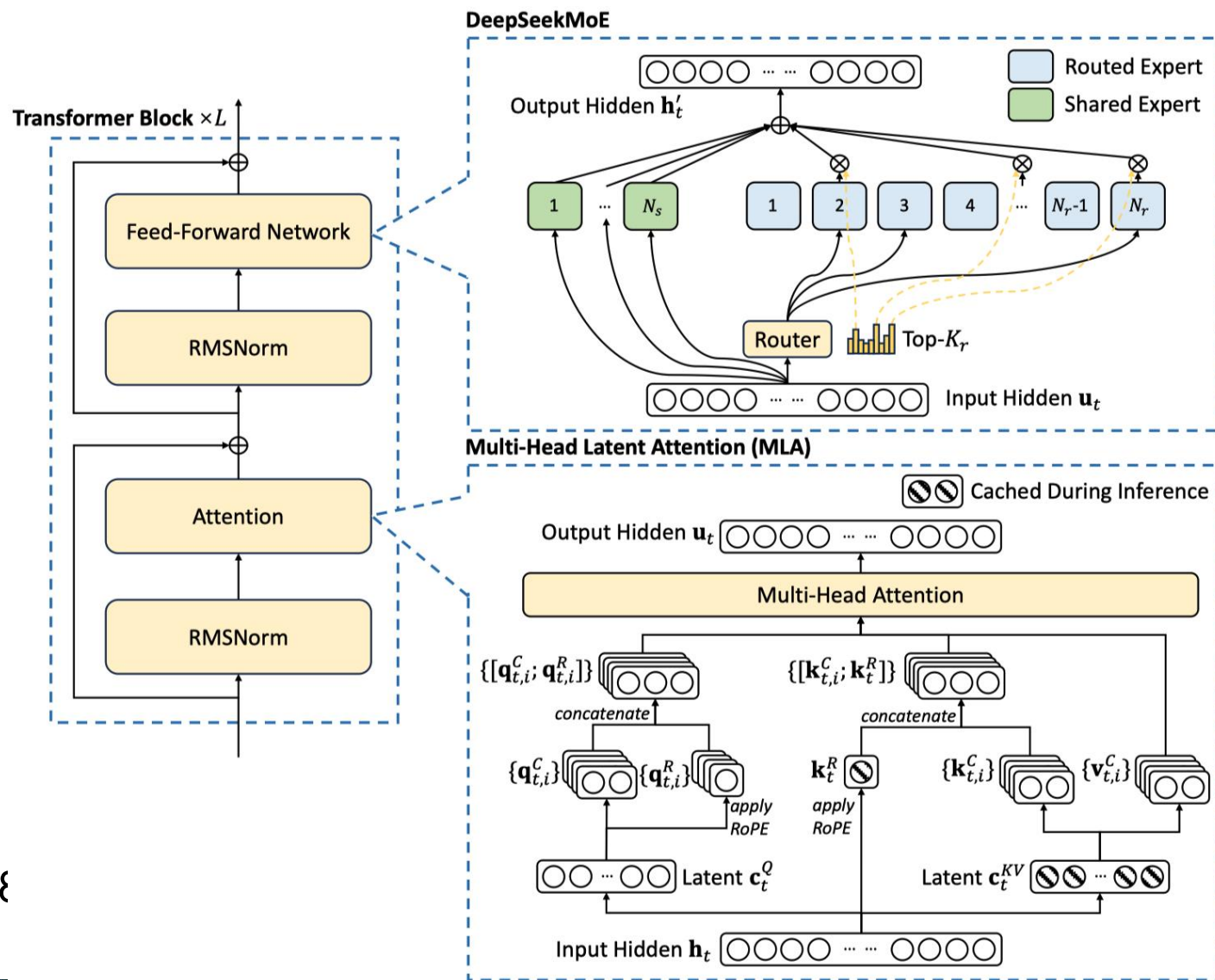


DeepSeek

DeepSeek-V3

671B参数，每token激活37B

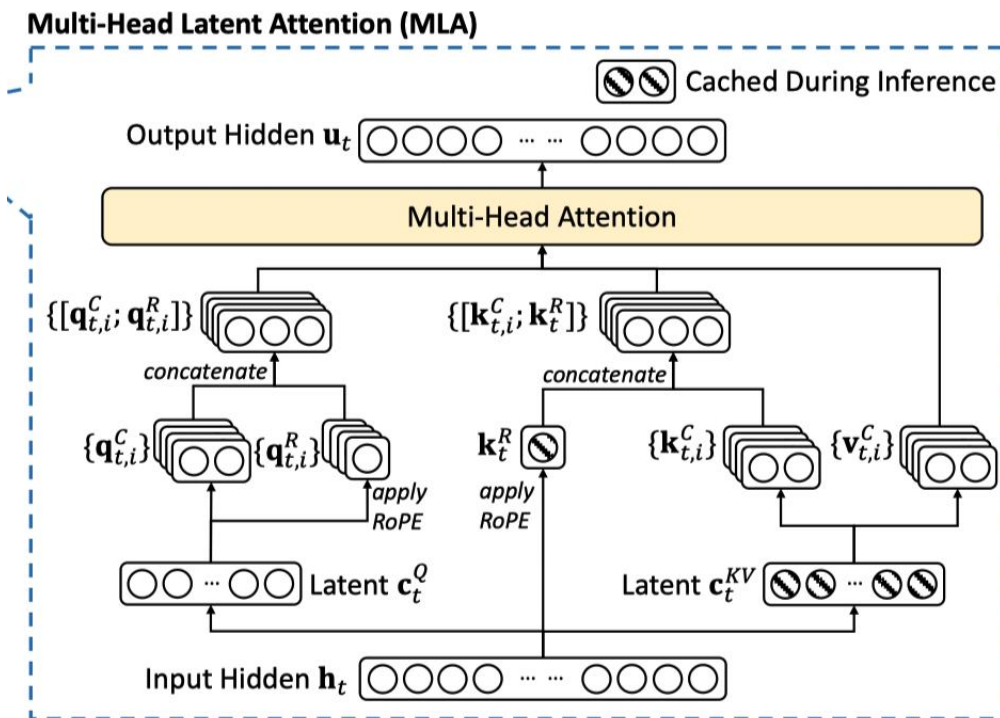
- 61层Transformer layers, 从第4层开始FFN替换为MoE
- hidden dimension=7168
- 注意力头128, 单头维度128
- KV压缩维度512, Q压缩维度1536, 旋转位置编码维度64
- SwiGLU激活函数
- MoE中每个expert的隐藏维度为2048



DeepSeek

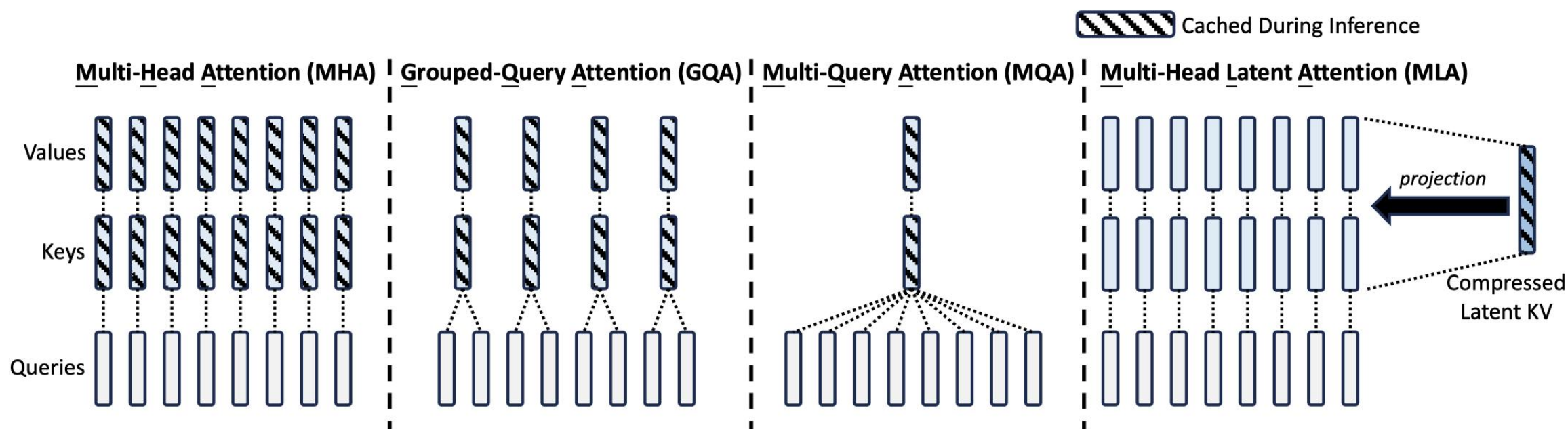
► MLA

$$\begin{aligned}
 \mathbf{c}_t^{KV} &= W^{DKV} \mathbf{h}_t, \\
 [\mathbf{k}_{t,1}^C; \mathbf{k}_{t,2}^C; \dots; \mathbf{k}_{t,n_h}^C] &= \mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV}, \\
 \mathbf{k}_t^R &= \text{RoPE}(W^{KR} \mathbf{h}_t), \\
 \mathbf{k}_{t,i} &= [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R], \\
 [\mathbf{v}_{t,1}^C; \mathbf{v}_{t,2}^C; \dots; \mathbf{v}_{t,n_h}^C] &= \mathbf{v}_t^C = W^{UV} \mathbf{c}_t^{KV}, \\
 \mathbf{o}_{t,i} &= \sum_{j=1}^t \text{Softmax}_j \left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}} \right) \mathbf{v}_{j,i}^C, \\
 \mathbf{u}_t &= W^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}],
 \end{aligned}$$



Attention变体

- ▶ 通过共享KV，减少计算量或存储开销，提升处理速度
- ▶ GQA: 将所有的Q头分成g组，同一组的Q头共享一个K头和一个V头
- ▶ MQA: 所有查询头共享相同的单一键和值头，因此只需要存储共享的K和V



DeepSeek

► MLA

- 性能优于MHA
- KV cache远低于MHA，小MoE模型约14%，大MoE模型约4%

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	37.0
MMLU (Acc.)	5-shot	37.9	41.2	45.2
C-Eval (Acc.)	5-shot	30.0	37.7	42.9
CMMLU (Acc.)	5-shot	34.6	38.4	43.5

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	39.0	46.6	50.7
MMLU (Acc.)	5-shot	48.7	50.0	57.5	59.0
C-Eval (Acc.)	5-shot	51.6	50.9	57.9	59.2
CMMLU (Acc.)	5-shot	52.3	53.4	60.7	62.5

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, arXiv:2405.04434v5, 2024.

DeepSeek

DeepSeek-V3中的MoE

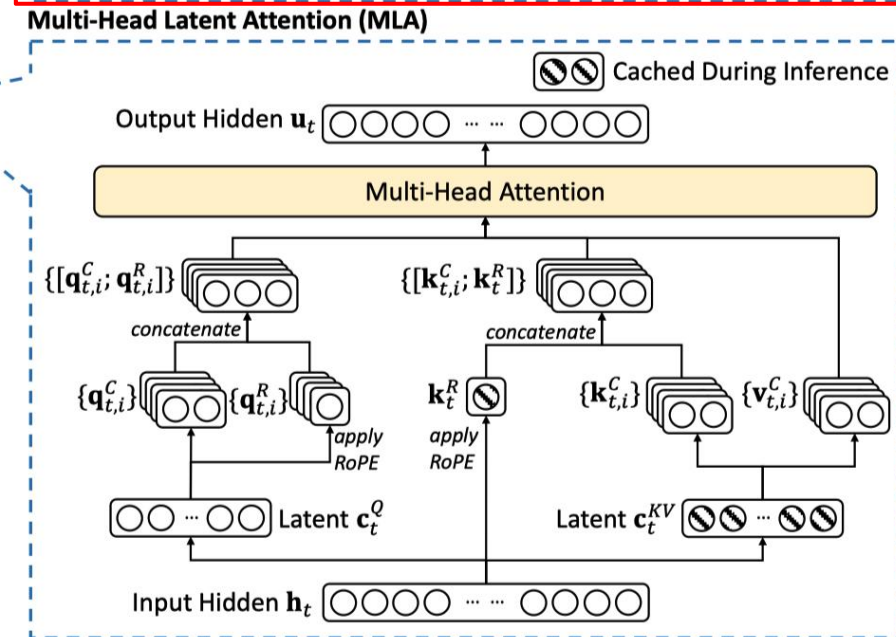
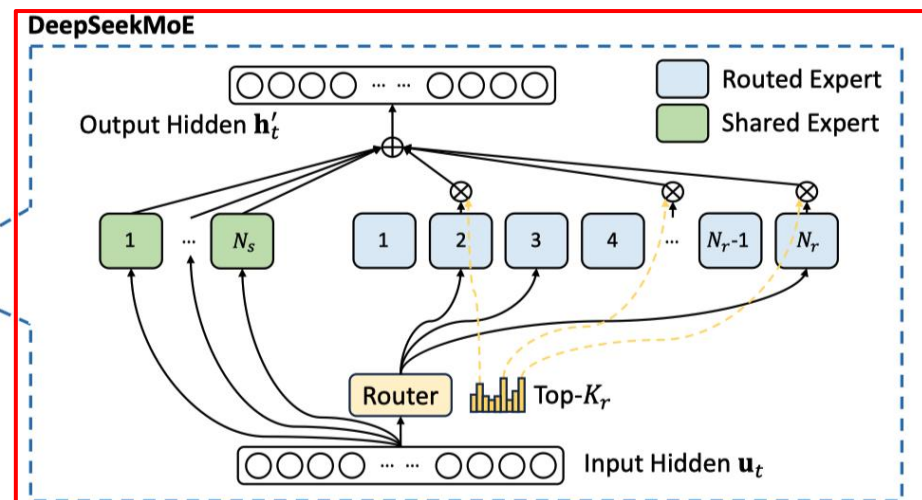
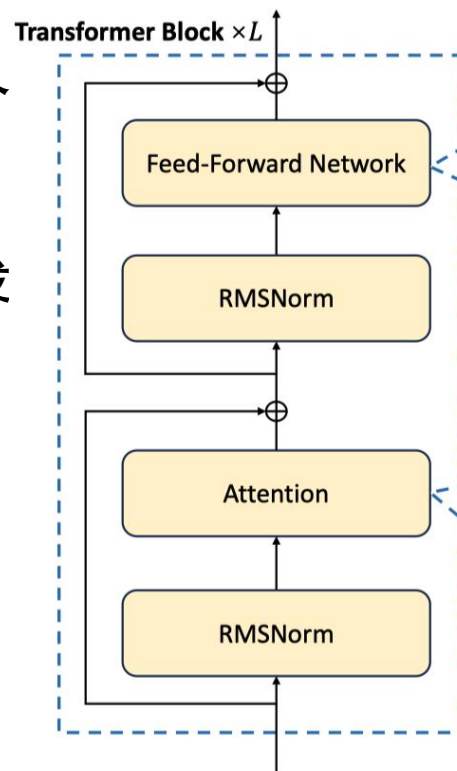
- 每个MoE包含1个shared expert, 256个 routed experts
- 每token激活8个routed experts, 最多发送到4个节点

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}},$$

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise,} \end{cases}$$

$$s_{i,t} = \text{Sigmoid}(\mathbf{u}_t^T \mathbf{e}_i),$$



Decoder-only 架构成为主流的原因探讨

► 一些原因分析

- decoder-only的架构相比encoder-decoder在in-context learning上会更有优势，因为prompt可以更加直接地作用于decoder每一层的参数，无需经过编码器的转换，微调的信号更强，使得decoder-only能够更准确地捕捉和利用输入序列的信息
- 注意力的秩：双向attention的注意力矩阵容易退化为低秩状态，而causal attention的注意力矩阵是下三角矩阵，必然是满秩的，建模能力更强
- causal attention （就是decoder-only的单向attention）具有隐式的位置编码功能

Decoder-only 架构成为主流的原因探讨

▶ 模型复杂度与训练效率

- ▶ decoder-only支持一直复用KV-Cache，计算效率更高
- ▶ Megatron和flash attention等重要工具对causal attention的支持更好

▶ 实验表明

- ▶ decoder-only架构的zero-shot泛化性能更好
- ▶ decoder-only架构的in context learning能力更强，使得few-shot能力更强